

QuadMath: An Analytical Review of 4D and Quadray Coordinates

Daniel Ari Friedman
ORCID: 0000-0001-6232-9096
Email: daniel@activeinference.institute
DOI: 10.5281/zenodo.16887791

October 07, 2026

Contents

1	Introduction	1
1.1	Abstract	1
1.2	Overview	1
1.3	4D Namespace Framework	2
1.4	Contributions	2
1.5	Manuscript Structure	2
1.6	Companion Code and Tests	2
1.7	Reproducibility and Data Availability	3
1.8	Graphical Abstract	3
2	4D Namespaces: Coxeter.4D, Einstein.4D, Fuller.4D	5
2.1	Coxeter.4D (Euclidean E^4)	5
2.2	Einstein.4D (Relativistic spacetime)	5
2.3	Fuller.4D (Synergetics / Quadrays)	5
2.3.1	Directions, not dimensions (language and models)	6
2.3.2	Figures	6
2.3.3	Clarifying remarks	6
2.4	Practical usage guide	6
3	Quadray Analytical Details and Methods	8
3.1	Overview	8
3.2	Mathematical Foundations	8
3.2.1	Framework Distinctions	8
3.2.2	Key Mathematical Principles	8
3.2.3	Implementation Strategy	9
3.3	Framework Integration	9
3.4	Fuller.4D Coordinates and Normalization	9
3.5	Conversions and Vector Operations: Quadray $\leftrightarrow$ Cartesian (Fuller.4D $\leftrightarrow$ Coxeter.4D/XYZ)	9
3.5.1	Integer-coordinate constructions (compact derivation box)	10
3.5.2	Example vertex lists and volume checks (illustrative)	11
3.6	Integer Volume Quantization	11
3.7	Distances and Metrics	12
3.8	XYZ determinant and S3 conversion	13
3.8.1	Alternative Volume Formulas	13
3.9	Fisher Geometry in Quadray Space	13
3.10	Practical Methods	13

3.11	Tetravolumes with Quadrays	13
3.11.1	Bridging and native tetravolume formulas	14
3.11.2	Short Python snippets	14
3.11.3	Random tetrahedra in the IVM (integer volumes)	18
3.11.4	Algebraic precision	18
3.11.5	XYZ determinant and the S3 conversion	18
3.11.6	D^3 vs R^3 : 60° “closing the lid” vs orthogonal “cubing”	18
3.12	Practical Implementation Workflow	19
3.12.11.	Mathematical Formulation	19
3.12.22.	Implementation Strategy	19
3.12.33.	Testing and Validation	19
3.12.44.	Documentation and Reproducibility	19
3.12.55.	Optimization and Extension	19
3.13	Code methods (anchors)	19
3.13.1	Core Quadray operations	19
3.13.2	Quaternion operations	20
3.13.3	Quaternion metrics	20
3.13.4	Volume calculations	22
3.13.5	Coordinate conversions	22
3.13.6	Linear algebra utilities	22
3.13.7	Optimization methods	22
3.13.8	Information geometry methods	22
3.13.9	Metrics and analysis	23
3.13.10	Examples and utilities	23
3.13.11	Symbolic computation	24
3.13.12	Visualization and animation	24
3.13.13	Path and file utilities	24
3.13.14	Additional Nelder-Mead components	24
3.13.15	Discrete variational components	25
3.13.16	Glossary generation	25
3.13.17	Geometry utilities	25
3.13.18	Comprehensive test coverage	25
3.14	Reproducibility checklist	26
4	Optimization in 4D	27
4.1	Overview	27
4.2	Nelder-Mead on Integer Lattice	27
4.2.1	Parameters	27
4.3	Volume-Level Dynamics	27
4.4	Quadray Lattice Optimization Pseudocode	27
4.4.1	Figures	28
4.5	Discrete Lattice Descent (Information-Theoretic Variant)	31
4.6	Convergence and Robustness	31
4.7	Information-Geometric View (Einstein.4D analogy in metric form)	31
4.7.1	Fisher Information as Riemannian Metric	31
4.7.2	4D Framework Integration through Fisher Information	32
4.7.3	Comprehensive Fisher Information Analysis	32
4.7.4	Natural Gradient Descent: Geodesic Motion on Information Manifold	32
4.7.5	Quadray-Specific Considerations	34
4.7.6	Variational Free Energy and Active Inference Integration	34
4.8	Multi-Objective and Higher-Dimensional Notes	34
4.9	External Validation and Computational Context	34
4.10	Results	34
5	Extensions of 4D and Quadrays	37
5.1	Multi-Objective Optimization	37

5.2	Machine Learning and Robustness	37
5.3	Computer Graphics and GPU Acceleration	37
5.4	Active Inference and Free Energy	38
5.5	Complex Systems and Collective Intelligence	38
5.6	Geospatial Intelligence and the World Game	38
5.7	Quadrays, Synergetics (Fuller.4D), and William Blake	38
5.8	Pedagogy and Implementations	39
5.9	Higher Dimensions and Decompositions	39
5.10	Limitations and Future Work	39
6	Discussion	40
6.1	Scope and Limitations	40
6.2	Fisher Information and Curvature	40
6.3	Quadrax Coordinates and 4D Structure (Fuller.4D vs Coxeter.4D vs Einstein.4D)	40
6.4	Integrating FIM with Quadrax Models	41
6.5	Implications for Optimization and Estimation	41
6.5.1	Clarifications on “frequency/time” dimensions	41
6.5.2	On distance-based tetravolume formulas (clarification)	41
6.5.3	Symbolic analysis (bridging vs native) (Results linkage)	41
6.6	Community Ecosystem and Validation	41
7	Resources	43
7.1	Core Concepts and Background	43
7.1.1	Information Geometry and Optimization	43
7.1.2	Active Inference and Free Energy	43
7.1.3	Mathematical Foundations	43
7.2	Quadrays and Synergetics (Core Starting Points)	43
7.2.1	Introductory Materials	43
7.2.2	Historical and Background Materials	43
7.3	4dsolutions Ecosystem: Comprehensive Computational Framework	43
7.3.1	Core Computational Modules	43
7.3.2	Primary Hub: School_of_Tomorrow (Python + Notebooks)	44
7.3.3	Additional Repositories	44
7.3.4	Educational Framework and Curricula	44
7.3.5	Media and Publications	44
7.4	Community Discussions and Collaborative Platforms	45
7.4.1	Active Platforms	45
7.4.2	Historical Archives	45
7.5	Related Projects and Applications	45
7.5.1	Tetrahedral Voxel Engines	45
7.5.2	Academic Publications	45
7.5.3	Context and Integration	45
8	Equations and Math Supplement (Appendix)	46
8.1	Volume of a Tetrahedron (Lattice)	46
8.2	Expanded Ace 5×5 Matrix	46
8.3	Cayley-Menger Determinant (Coxeter.4D)	47
8.4	Piero della Francesca Formula (PdF)	47
8.5	Gerald de Jong Formula (GdJ)	48
8.6	Fisher Information Matrix (FIM)	48
8.7	Empirical Fisher Information Matrix	48
8.8	Natural Gradient	49
8.9	Free Energy (Active Inference)	49
8.10	Expected Free Energy (Active Inference)	49
8.11	Quadrax Normalization (Fuller.4D)	50
8.12	Distance (Embedding Sketch; Coxeter.4D slice)	50

8.13	Minkowski Line Element (Einstein.4D analogy)	50
8.14	High-Precision Arithmetic Note	50
8.14.1	Reproducibility artifacts and external validation	50
8.15	Namespaces summary (notation)	50
9	Appendix: The Free Energy Principle and Active Inference	52
9.1	Overview	52
9.2	Mathematical Formulation and Equation Callouts (Equations linkage)	52
9.3	Four-Fold Partition and Tetrahedral Mapping (Quadrays; Fuller.4D)	52
9.4	How the 4D namespaces relate here	56
9.5	Joint Optimization in the Tetrahedral Framework (Methods linkage)	56
9.6	Implications for AI and Robust Computation	56
9.7	Code, Reproducibility, and Cross-References	56
10	Appendix: Symbols and Glossary	58
10.1	Sets and Spaces	58
10.2	Quadrax Coordinates and Geometry	58
10.3	Optimization and Algorithms	59
10.4	Information Theory and Geometry	59
10.5	Embeddings and Distances	59
10.6	Greek Letters (usage)	60
10.7	Notes (usage and cross-references)	60
10.8	Polyhedra and Synergetic Shapes	60
10.9	Acronyms and abbreviations	60
10.10	API Index (auto-generated; Methods linkage)	61
11	Static IVM Field Learning	81
11.1	Overview	81
11.2	The IVM Lattice in Quadrax Coordinates	81
11.3	The Field Model	81
11.4	Learning: Laplacian-Regularized Kernel-Weighted Least Squares	82
11.5	Recovering Tetrahedral Geometry from Noisy Points	82
11.6	Demonstration	83
11.7	Cross-References	83
12	Dynamics and Learning on the IVM Lattice	84
12.1	Overview	84
12.2	Lattice sites, shells, and the 12-around-one move graph	84
12.3	Heat update and the monotonicity lemma	84
12.4	Majority update on integer states	85
12.5	Learning the coupling from an observed trajectory	85
12.6	Demo figure	85
12.7	Reproducibility and test contract	85
12.8	Cross-references	87
13	Lattice Tooling: Omnidirectional Numbering and Nearest-Site Search	88
13.1	Overview	88
13.2	Omnidirectional Close Packing and Frequency Shells	88
13.2.1	Reachability is enumerative, not congruence-based	88
13.2.2	Site/index mapping	88
13.3	Fast Nearest-Site Queries	89
13.3.1	Exact lattice distances	89
13.3.2	Truncation bound	89
13.3.3	Shell-sweep nearest	89
13.4	API Summary	90
13.5	Verification	90

14	Conversions & Specification	91
14.1	Overview	91
14.2	The embedding (Fuller.4D to Coxeter.4D)	91
14.3	Gram identities and exact distances	91
14.4	Exact canonical inversion	92
14.5	Lattice context	93
14.6	API Summary	93
14.7	Verification	94
15	Learning and Evaluation on the IVM Lattice	95
15.1	Overview	95
15.2	Why held-out evaluation is harder on a lattice	95
15.3	K-fold site splits	95
15.4	Cross-validation and honest model selection	95
15.5	Temporal splits for dynamics identification	96
15.6	Learning curves	97
15.7	Three-way splitting and trainable site fits	97
15.8	Verification	98
15.9	Cross-References	99
16	Lattice Visualization Gallery	100
16.1	Overview	100
16.2	Frequency shells	100
16.3	Lattice-plane slices	100
16.4	Dynamics strips	101
16.5	Deterministic animations and gallery plots	101
16.6	Reproducibility and test contract	104
16.7	Cross-references	105
17	Benchmarks and Statistics	106
17.1	Overview	106
17.2	Timing methodology	106
17.3	Percentile bootstrap	106
17.4	Pooled permutation tests	107
17.5	Effect size, multiplicity, and scaling	107
17.6	Jackknife, FDR, Welch, and circular statistics	107
17.7	A reproducible example	109
17.8	Cross-references	110
18	Statistics and Scaling Gallery	112
18.1	Overview	112
18.2	Latency histogram	112
18.3	Scaling fit	112
18.4	Confidence intervals	112
18.5	Empirical CDF	112
18.6	Reproducibility and test contract	116
18.7	Cross-references	117

1 Introduction

1.1 Abstract

We review a unified analytical framework for four-dimensional (4D) modeling with Quadray coordinates, synthesizing geometric foundations, optimization on tetrahedral lattices, and information geometry. Building on R. Buckminster Fuller’s [Synergetics](#) and Kirby Urner’s computational implementations in the [4dsolutions](#)

ecosystem (Python, Rust, Clojure, and POV-Ray), we show that integer lattice constraints force the volume of any tetrahedron with lattice vertices to be an integer multiple of the unit tetrahedron’s volume — a quantization into discrete “energy levels” that regularizes optimization on the tetrahedral lattice. We adapt the **Nelder-Mead method** to the Quadray lattice, define **Fisher information** in Quadray parameter space, and analyze optimization as geodesic motion on an information manifold via the **natural gradient**. We distinguish three 4D namespaces — Coxeter.4D (Euclidean E^4), Einstein.4D (Minkowski spacetime), and Fuller.4D (synergetics/Quadrays) — and map each to the analytical tools it supports. The result is a cohesive, interpretable approach for robust, geometry-grounded computation in 4D. All source code for the manuscript is available at **QuadMath**.

Keywords: Quadray coordinates, 4D geometry, tetrahedral lattice, integer volume quantization, information geometry, optimization, synergetics, active inference.

1.2 Overview

Quadray coordinates provide a tetrahedral basis for modeling space and computation, standing in contrast to Cartesian cubic frameworks. Originating in Buckminster Fuller’s Synergetics, Quadray coordinates replace right-angle orthonormal assumptions with 60-degree coordination and a unit tetrahedron of volume 1. This reframing yields striking integer relationships among common polyhedra and provides a natural account of space via close-packed spheres and the isotropic vector matrix (IVM).

This paper unifies three threads:

- **Foundations:** Quadray coordinates and their relation to 4D modeling more generally, with explicit namespace usage (Coxeter.4D, Einstein.4D, Fuller.4D) to maintain clarity.
- **Optimization framework:** Leverages integer volume quantization on tetrahedral lattices to achieve robust, discrete convergence.
- **Information geometry:** Tools (e.g., Fisher Information, free-energy minimization) for interpreting optimization as geodesic motion on statistical manifolds.

1.3 4D Namespace Framework

In this synthetic review, we distinguish three internal meanings of “4D,” following a dot-notation that avoids cross-domain confusion. For comprehensive details, see **Section 2: 4D Namespaces**.

- **Coxeter.4D** — four-dimensional Euclidean space (E^4) with mutually orthogonal axes and a positive-definite metric: the setting of classical polytope theory, and explicitly not spacetime (Coxeter, *Regular Polytopes*, Dover ed., p. 119). Lattice packings in four dimensions align with the treatment in Conway & Sloane’s **Sphere Packings, Lattices and Groups**.
- **Einstein.4D** — Minkowski spacetime: three space dimensions plus time, with an indefinite metric of signature $(-, +, +, +)$; the arena of relativistic physics, distinct from Euclidean E^4 .
- **Fuller.4D** — synergetics’ tetrahedral accounting of space: Quadray coordinates (four non-negative coordinates with at least one zero after normalization) on the Isotropic Vector Matrix (IVM) = Cubic Close Packing (CCP) = Face-Centered Cubic (FCC) lattice, with the regular tetrahedron as the natural unit container; it emphasizes angle/shape relations independent of time and energy.

1.4 Contributions

The paper makes the following key contributions:

- **Namespaces mapping:** Coxeter.4D (Euclidean E^4), Einstein.4D (Minkowski spacetime), and Fuller.4D (Quadrays/IVM) $\rightarrow$ analytical tools and examples.
- **Quadray-adapted Nelder-Mead:** Integer-lattice normalization and volume-level tracking.
- **Equations and methods:** Comprehensive supplement with guidance for high-precision computation using `libquadmath`.
- **Discrete optimizer:** Integer-valued variational descent over the IVM (`discrete_ivm_descent`) with animation tooling, connecting lattice geometry to information-theoretic objectives.

1.5 Manuscript Structure

- **Introduction:** motivates Quadrays, clarifies 4D namespaces, and summarizes contributions.
- **Methods:** details coordinate conventions, exact tetravolumes, conversions, and lattice-aware optimization methods (Nelder-Mead and discrete IVM descent).
- **Results:** empirical comparisons and demonstrations are shown inline and saved under `quadmath/output/` (PNG/CSV/NPZ/MP4) for reproducibility.
- **Discussion:** interprets results, limitations, and implications; outlines future work.
- **Appendices:** equations, free-energy background, and a consolidated symbols/glossary with an auto-generated API index.

1.6 Companion Code and Tests

The manuscript is accompanied by a fully-tested Python codebase under `src/` with unit tests under `tests/`. Key artifacts used throughout the paper:

- **Quadray APIs:** `src/quadmath/core/quadray.py` (`Quadray`, `integer_tetra_volume`, `ace_tetravolume_5x5`).
- **Determinant utilities:** `src/quadmath/core/linalg_utils.py` (`bareiss_determinant_int`).
- **Length-based volume:** `src/quadmath/core/cayley_menger.py` (`tetra_volume_cayley_menger`, `ivm_tetra_volume_cayley_menger`).
- **XYZ conversion:** `src/quadmath/lattice/conversions.py` (`urner_embedding`, `quadray_to_xyz`).
- **Examples:** `src/quadmath/core/examples.py` (`example_ivm_neighbors`, `example_volume`, `example_optimize`).

For comprehensive background resources, computational implementations, and related work, see the [Resources](#) section.

1.7 Reproducibility and Data Availability

- The manuscript Markdown and code to generate the PDF are available on the project repository (QuadMath on GitHub, @docxology username). See the repository home page for source, figures, and scripts: [QuadMath repository](#). The repository is also archived on [Zenodo record 16887791](#) with DOI 10.5281/zenodo.16887791.
- The manuscript is licensed under the Apache License 2.0. See the [LICENSE](#) file for details.
- The manuscript is accompanied by a fully-tested Python codebase under `src/` with unit tests under `tests/`, complemented by extensive cross-validation against Kirby Urner's reference implementations in the [4dsolutions ecosystem](#). See the [Resources](#) section for comprehensive details on computational implementations and validation.
- All figures referenced in the manuscript are generated by scripts under `quadmath/scripts/` and saved to `quadmath/output/` with lightweight CSV/NPZ alongside images.
- Tests accompany all methods under `src/` and enforce 100% coverage for `src/`.
- Symbols and notation are standardized across sections; see [Appendix: Symbols and Glossary](#) for a consolidated table of variables and constants used throughout. Equation labels (e.g., Eq. (1) and Eq. (8)) and figure labels are automatically numbered by LaTeX for consistent cross-referencing.
- The manuscript is a work in progress and will be updated as the project progresses. There may be errors and missing references, check all methods and equations for consistency.

1.8 Graphical Abstract

Panel A shows the four Quadray axes (A, B, C, D) under the default symmetric embedding with a tetrahedron wireframe. **Panel B** shows the same vertices as close-packed spheres (IVM = CCP = FCC) sized to kiss along the tetrahedron edges.

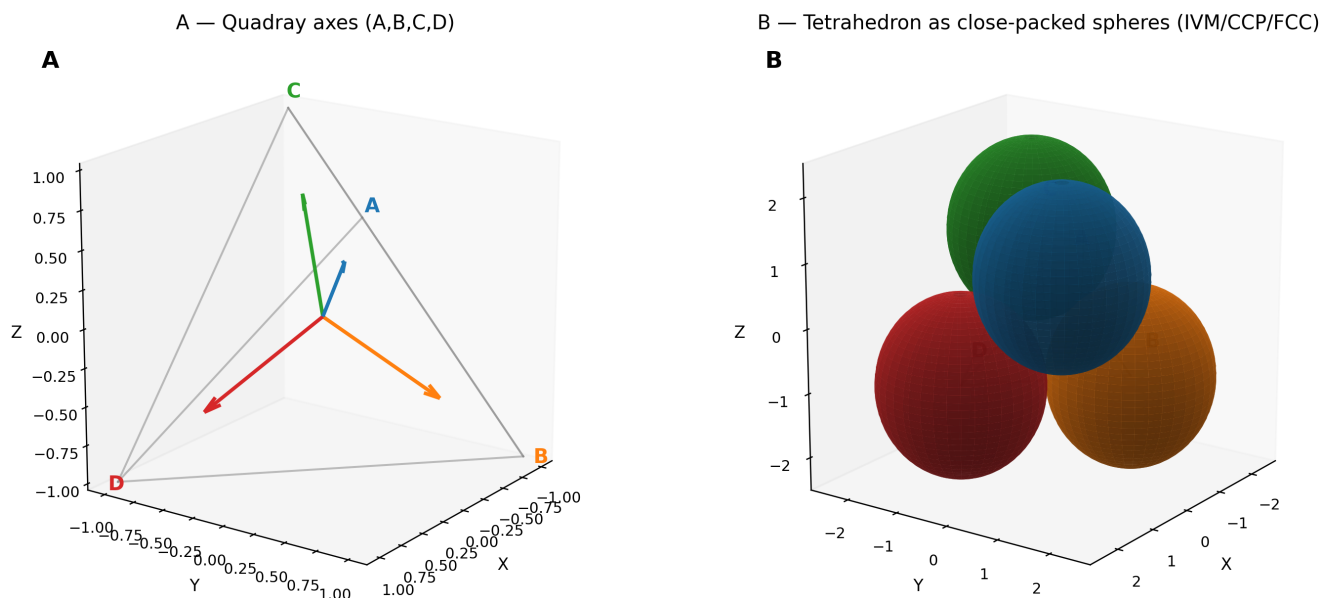


Figure 1: **Quadray coordinate system overview (graphical abstract).** **Panel A:** The four Quadray axes — rays A, B, C, D from the origin to the vertices of a reference regular tetrahedron under the default symmetric embedding — drawn as colored arrows (A=blue, B=orange, C=green, D=red) with axis labels at the vertex endpoints and a light gray wireframe joining the four vertices. Notice that the axes are four canonical directions (spokes) separated by 60° angles, not orthogonal Cartesian dimensions — the defining “directions, not dimensions” character of Fuller.4D. **Panel B:** The same four vertices shown as close-packed spheres, each colored to match its Panel A axis and with radius equal to half the minimum edge length, so neighboring spheres kiss exactly along the tetrahedron edges; a light black wireframe marks those edges. This is the local building block of the Isotropic Vector Matrix (IVM) = Cubic Close Packing (CCP) = Face-Centered Cubic (FCC) correspondence; extending the kissing arrangement across the lattice yields the “twelve around one” coordination central to synergetics. What to notice: one and the same tetrahedral skeleton underlies both the direction-based Quadray coordinate system and the densest sphere packing in 3D, which is why integer Quadray coordinates and integer sphere counts (and hence integer volumes) live on a single lattice. Generated by `quadmath/scripts/graphical_abstract_quadray.py` (output at `quadmath/output/figures/graphical_abstract_quadray.png`).

2 4D Namespaces: Coxeter.4D, Einstein.4D, Fuller.4D

This section provides the definitive reference for the three 4D frameworks used throughout this manuscript. Each namespace represents a distinct mathematical framework with specific applications in our Quadray-based computational system.

2.1 Coxeter.4D (Euclidean E^4)

Definition: Four-dimensional Euclidean space E^4 with mutually orthogonal axes and a positive-definite metric — the native setting of classical regular polytopes, and, per Coxeter (Regular Polytopes, Dover ed., p. 119), explicitly not spacetime. Lattice/packing discussions connect to Conway & Sloane’s systematic treatment of higher-dimensional sphere packings and lattices ([Sphere Packings, Lattices and Groups \(Springer\)](#)).

Usage: Embed Quadray configurations or compare alternative parameterizations when a strictly Euclidean 4D setting is desired.

Simplexes: Simplex structures extend naturally to 4D and beyond (e.g., pentachora).

Mathematical context: This framework is appropriate for standard Euclidean geometry, including the Cayley-Menger determinant for computing volumes from edge lengths.

2.2 Einstein.4D (Relativistic spacetime)

Definition: Minkowski spacetime — three space dimensions joined to one time dimension by an indefinite metric of signature $(-, +, +, +)$ (mostly-plus convention) — the geometric arena of special relativity; the sign of the squared line element classifies separations as timelike, lightlike, or spacelike.

Spacetime: Minkowski metric signature.

Line element (mostly-plus convention; see [Minkowski space](#)): see Eq. (13) in the equations appendix.

Optimization analogy: Metric-aware geodesics generalize to information geometry where the Fisher metric replaces the physical metric. See [Fisher information](#) and [natural gradient](#).

Important note: This namespace is used ONLY as a metric/geodesic analogy when discussing information geometry. Physical constants G , c , Λ do not appear in Quadray lattice methods and should not be mixed with IVM unit conventions.

2.3 Fuller.4D (Synergetics / Quadrays)

Definition: The synergetic account of space built on Quadray coordinates — four non-negative components (A, B, C, D) giving directions from the center of a reference regular tetrahedron to its vertices — together with the IVM = CCP = FCC lattice correspondence, the regular tetrahedron as the unit of volume, and exact integer tetravolumes for lattice tetrahedra; it tracks shape and angle relations among containers, independent of time and energy.

Basis: Four non-negative components A, B, C, D with at least one zero post-normalization, treated as a vector (direction and magnitude), not merely a point. Overview: [Quadray coordinates](#).

Geometry: Tetrahedral; unit tetrahedron volume = 1; integer lattice aligns with close-packed spheres (IVM). Background: [Synergetics](#).

Distances: Computed via appropriate projective normalization; edges align with tetrahedral axes. The IVM = CCP = FCC shortcut allows working in 3D embeddings for visualization while preserving the underlying Fuller.4D tetrahedral accounting.

Implementation heritage: Extensive computational validation through Kirby Urner’s [4dsolutions ecosystem](#). See the [Resources](#) section for comprehensive details on computational implementations and educational materials.

2.3.1 Directions, not dimensions (language and models)

Vector-first framing: Treat Quadrays as four canonical directions (“spokes” to the vertices of a regular tetrahedron from its center), not as four orthogonal dimensions. The methane molecule (CH_4) and caltrop shape are helpful mental models.

Origins outside Synergetics: Quadrays did not originate with Fuller; we adopt the coordinate system within the IVM context. See [Quadray coordinates](#).

Language games: Quadrays and Cartesian are parallel vector languages on the same Euclidean container; teaching them together avoids oscillating between “points now, vectors later.”

2.3.2 Figures

In the previous figure, we show the twelve nearest IVM neighbors with coordination patterns and vector equilibrium geometry; the current figure illustrates random Quadray clouds under several embeddings.

Vector equilibrium (cuboctahedron): The shell formed by the 12 nearest IVM neighbors is the cuboctahedron, also called the vector equilibrium in synergetics. All 12 vertices are equidistant from the origin with equal edge lengths, modeling a balanced local packing. This geometry underlies the “twelve around one” close-packing motif and appears in tensegrity discussions as a canonical balanced structure. See background: [Cuboctahedron \(vector equilibrium\)](#) and synergetics references. Computational demonstrations include related visualizations in the 4dsolutions ecosystem. See the [Resources](#) section for comprehensive details.

2.3.3 Clarifying remarks

“A time machine is not a tesseract.” [KU on synergeo](#) The tesseract is a Euclidean 4D object (Coxeter.4D), while Minkowski spacetime (Einstein.4D) is indefinite and not Euclidean; conflating the two leads to category errors. Fuller.4D, in turn, is a tetrahedral, mereological framing of ordinary space emphasizing shape/angle relations and IVM quantization. Each namespace carries distinct assumptions and should be used accordingly in analysis.

2.4 Practical usage guide

- Use **Fuller.4D** when working with Quadrays, integer tetravolumes, and IVM neighbors (native lattice calculations).
- Use **Coxeter.4D** for Euclidean length-based formulas, higher-dimensional polytopes, or comparisons in E^4 (including Cayley-Menger).
- Use **Einstein.4D** as a metric analogy when discussing geodesics or time-evolution; do not mix with synergetic unit conventions.

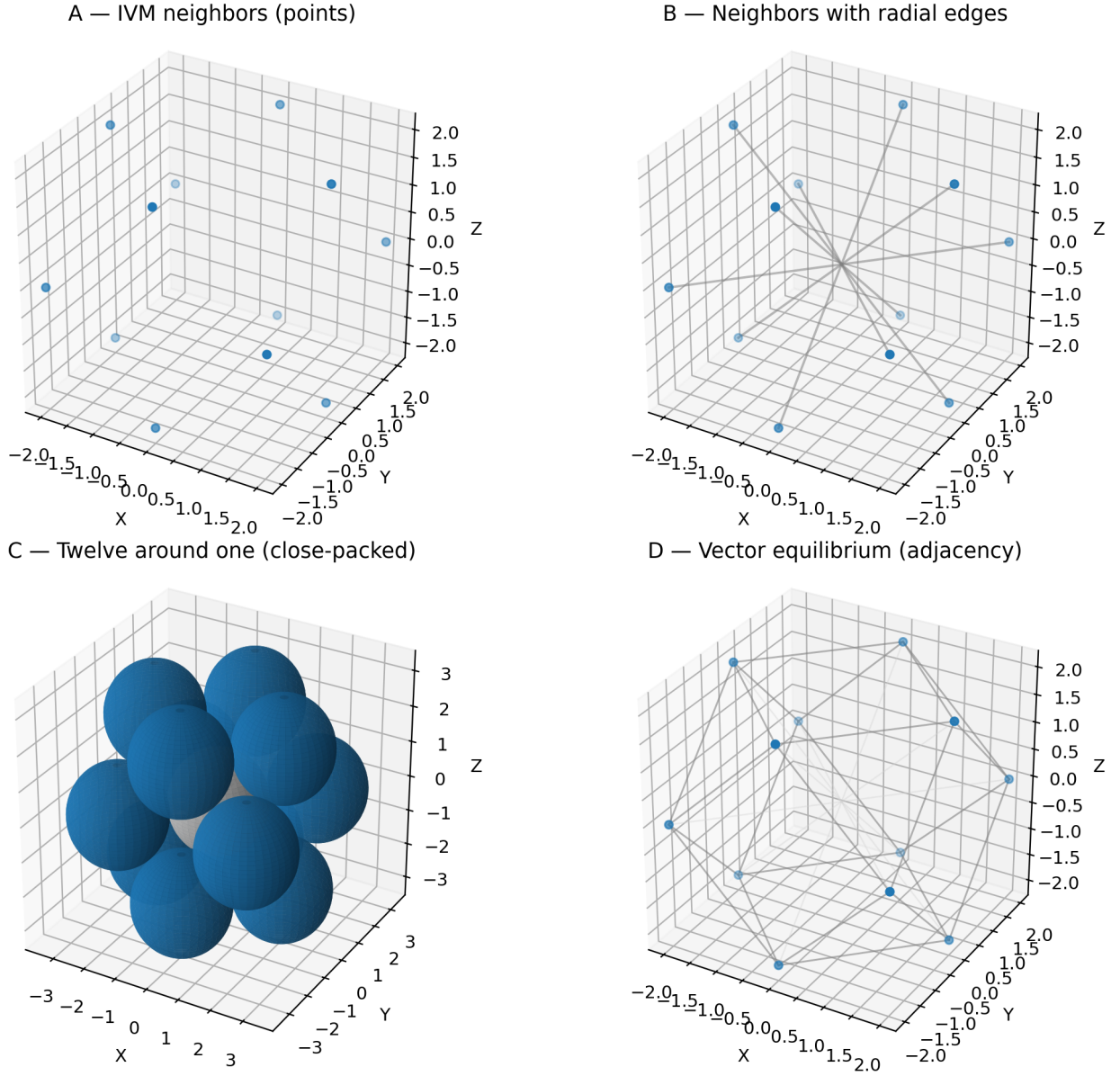


Figure 2: **IVM neighbors and coordination patterns (2×2 panel layout).** **Panel A:** The twelve nearest IVM neighbors plotted as blue points in 3D space under the default embedding, showing the positions corresponding to permutations of the Quadray integer coordinates $\{2,1,1,0\}$. These points form the vertices of a cuboctahedron (vector equilibrium) centered at the origin with uniform radial distances. **Panel B:** The same neighbor points with radial edges (light lines) connecting each neighbor to the central origin, emphasizing the spoke-like radial symmetry and equal distances from center to shell. **Panel C:** Twelve-around-one close-packed spheres configuration where each neighbor position hosts a sphere with radius chosen so neighboring spheres kiss along cuboctahedron edges, illustrating the fundamental CCP/FCC/IVM correspondence. The central gray sphere represents the “one” in Fuller’s “twelve around one” motif. **Panel D:** Adjacency graph showing strut connections (solid lines) between touching neighbor spheres, revealing the cuboctahedron’s edge structure, plus light radial cables to the origin representing a stylized tensegrity interpretation of the vector equilibrium geometry.

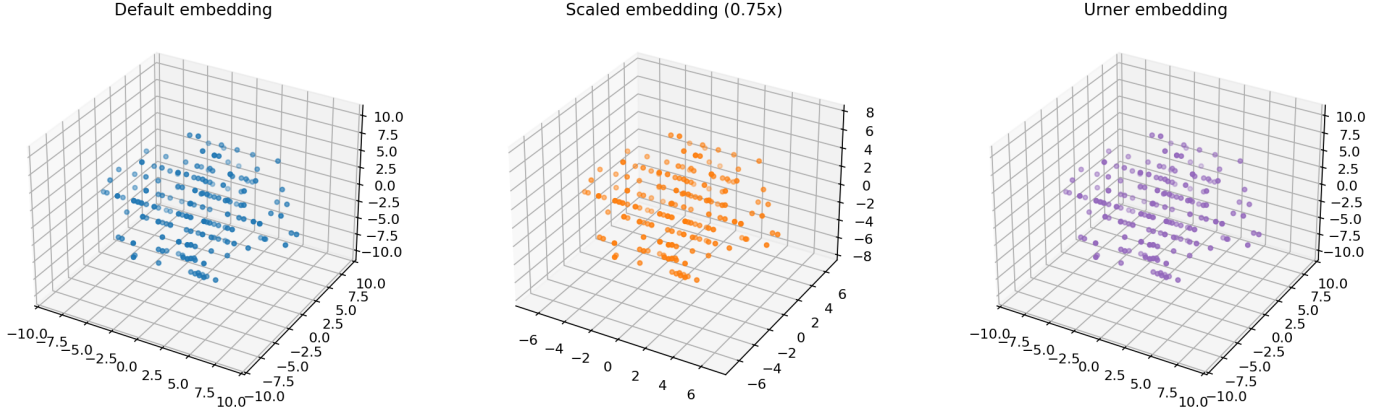


Figure 3: **Random Quadray point clouds under different embeddings (3-panel comparison).** Each panel shows 200 randomly sampled integer Quadray coordinates with components in $\{0,1,2,3,4,5\}$ projected to 3D space using different embedding matrices. **Left panel (Default embedding):** Points (blue) under the default symmetric embedding matrix showing the natural tetrahedral-symmetric distribution of normalized Quadrays in 3D space. **Center panel (Scaled embedding, 0.75x):** The same Quadray points (orange) under a uniformly scaled version of the default embedding, demonstrating how the point cloud structure scales proportionally while preserving relative geometries. **Right panel (Urner embedding):** The same points (purple) projected through the canonical Urner embedding matrix, illustrating how different linear mappings from Fuller.4D to Coxeter.4D (3D slice) affect the spatial distribution while preserving the underlying discrete lattice relationships. This comparison demonstrates the flexibility in choosing embeddings for visualization and analysis while maintaining the fundamental Quadray coordinate relationships.

3 Quadray Analytical Details and Methods

3.1 Overview

This section provides detailed analytical methods for working with Quadray coordinates, including coordinate conventions, volume calculations, and optimization approaches. We emphasize the distinction between different 4D frameworks and provide practical computational methods.

3.2 Mathematical Foundations

The methods presented here rest on three interconnected mathematical frameworks. For comprehensive definitions, see [Section 2: 4D Namespaces](#).

3.2.1 Framework Distinctions

- **Coxeter.4D:** Euclidean 4D geometry with standard metric tensor and volume forms
- **Einstein.4D:** Minkowski spacetime with indefinite metric for information geometry analogies
- **Fuller.4D:** Synergetics/Quadray coordinates with integer lattice constraints and IVM unit conventions

3.2.2 Key Mathematical Principles

- **Rational volume quantization:** Lattice constraints quantize tetrahedral volumes to exact quarter-unit ($1/4$ -grain) rationals in IVM units — integral for unit-tetra tilings, e.g. $\frac{1}{4}$ for the primitive tetrahedron
- **Coordinate system bridges:** Linear transformations between Fuller.4D and Coxeter.4D preserve geometric relationships
- **Information geometry:** Fisher metric provides Riemannian structure for optimization on parameter manifolds

- **Exact arithmetic:** Bareiss algorithm ensures determinant calculations remain exact for integer inputs

3.2.3 Implementation Strategy

All mathematical concepts are implemented with: - **Exact arithmetic** where possible (integer determinants, symbolic computation) - **Numerical stability** for floating-point operations (ridge regularization, condition number monitoring) - **Cross-validation** between different formulations (Ace 5×5 vs Cayley-Menger, native vs bridging approaches) - **Comprehensive testing** ensuring mathematical correctness across edge cases

3.3 Framework Integration

The three 4D frameworks serve distinct but complementary roles in our implementation:

- **Coxeter.4D provides the container:** Euclidean 4D space serves as the mathematical foundation for volume calculations, distance metrics, and geometric transformations. When we compute volumes via Cayley-Menger determinants or XYZ coordinate determinants, we operate in this framework.
- **Einstein.4D provides the analogy:** The Minkowski metric structure inspires our information geometry approach, where the Fisher information matrix acts as a Riemannian metric on parameter space. Natural gradient descent follows geodesics on this information manifold, analogous to how particles follow geodesics in spacetime.
- **Fuller.4D provides the constraints:** The Quadray coordinate system and IVM lattice impose integer constraints that enable exact arithmetic and discrete optimization. The synergetics unit conventions (regular tetrahedron volume = 1) create a quantized geometry where volumes are exact rationals on the $1/4$ grid — integral for tetrahedra tiling unit IVM tetras, $\frac{1}{4}$ -grain otherwise.

This multi-framework approach allows us to: 1. Use standard Euclidean methods for volume calculations where relevant or already in use (Coxeter.4D) 2. Apply information geometry principles for geodesic optimization (Einstein.4D analogy)

3. Maintain exact arithmetic in the Isotropic Vector Matrix (IVM) setting through integer lattice constraints (Fuller.4D) 4. Bridge and swap among frameworks via coordinate transformations and unit conversions

3.4 Fuller.4D Coordinates and Normalization

- Quadray vector $q = (a,b,c,d)$, $a,b,c,d \geq 0$, with at least one coordinate zero under normalization.
- Projective normalization can add/subtract (k,k,k,k) without changing direction; choose k to enforce non-negativity and one zero minimum.
- Isotropic Vector Matrix (IVM): integer quadrays describe CCP sphere centers; the 12 permutations of $\{2,1,1,0\}$ form the cuboctahedron (vector equilibrium).
 - Integer-coordinate models: assigning unit IVM tetravolume to the regular tetrahedron yields integer coordinates for several familiar polyhedra (inverse tetrahedron, cube, octahedron, rhombic dodecahedron, cuboctahedron) when expressed as linear combinations of the four quadray basis vectors. See overview: [Quadray coordinates](#).

3.5 Conversions and Vector Operations: Quadray $\leftrightarrow$ Cartesian (Fuller.4D $\leftrightarrow$ Coxeter.4D/XYZ)

- **Embedding conventions** determine the linear maps between Quadray (Fuller.4D) and Cartesian XYZ (a 3D slice or embedding aligned with Coxeter.4D conventions).
- **References:** Urner provides practical conversion write-ups and matrices; see:
 - Quadrays and XYZ: [Urner - Quadrays and XYZ](#)
 - Introduction with examples: [Urner - Quadray intro](#)
- **Implementation:** choose a fixed tetrahedral embedding; construct a 3×4 matrix M that maps (a,b,c,d) to (x,y,z) , respecting A,B,C,D directions to tetra vertices. The inverse map can be defined up to projective normalization (adding (k,k,k,k)). When comparing volumes, use the $s_3 = \sqrt{9/8}$ scale to convert XYZ (Euclidean) volumes to IVM (Fuller.4D) units.

- **Vector view:** treat q as a vector with magnitude and direction; define dot products and norms by pushing to XYZ via M .

3.5.1 Integer-coordinate constructions (compact derivation box)

- Under the synergetics convention (unit regular tetrahedron has tetravolume 1), many familiar solids admit Quadray integer coordinates. For example, the octahedron at the same edge length has tetravolume 4, and its vertices can be formed as integer linear combinations of the four axes A,B,C,D subject to the Quadray normalization rule.
- The cuboctahedron (vector equilibrium) arises as the shell of the 12 nearest IVM neighbors given by the permutations of $(2, 1, 1, 0)$. The rhombic dodecahedron (tetravolume 6) is the Voronoi cell of the FCC/CCP packing centered at the origin under the same embedding.
- See the following figure for a schematic summary of these relationships.

Object	Quadray construction (sketch)	IVM volume
Regular tetrahedron	Vertices $o=(0,0,0,0)$, $p=(2,1,0,1)$, $q=(2,1,1,0)$, $r=(2,0,1,1)$	1
Cube (same edge)	Union of 3 mutually orthogonal rhombic belts wrapped on the tetra frame; edges tracked by XYZ embedding; compare the following figure	3
Octahedron (same edge)	Convex hull of mid-edges of the tetra frame (pairwise axis sums normalized)	4
Rhombic dodecahedron	Voronoi cell of FCC/CCP packing at origin (dual to cuboctahedron)	6
Cuboctahedron (vector equilibrium)	Shell of the 12 nearest IVM neighbors: permutations of $(2,1,1,0)$	20
Truncated octahedron	Archimedean solid with 6 square and 8 hexagonal faces; space-filling tiling	20

Small coordinate examples (subset):

- Cuboctahedron neighbors (representatives): $(2,1,1,0)$, $(2,1,0,1)$, $(2,0,1,1)$, $(1,2,1,0)$; the full shell is all distinct permutations.
- Tetrahedron: $[(0,0,0,0)$, $(2,1,0,1)$, $(2,1,1,0)$, $(2,0,1,1)]$.

Short scripts:

```
1 python3 quadmath/scripts/polyhedra_quadray_constructions.py
```

Programmatic check (neighbors, equal radii, adjacency):

```
1 import numpy as np
2 from quadmath.core.examples import example_cuboctahedron_vertices_xyz
3
4 xyz = np.array(example_cuboctahedron_vertices_xyz())
5 r = np.linalg.norm(xyz[0])
6 assert np.allclose(np.linalg.norm(xyz, axis=1), r)
7
8 # Touching neighbors have separation 2r
9 touch = []
10 for i in range(len(xyz)):
```

```

11     for j in range(i+1, len(xyz)):
12         d = np.linalg.norm(xyz[i] - xyz[j])
13         if abs(d - 2*r) / (2*r) < 0.05:
14             touch.append((i, j))
15 assert len(touch) > 0

```

3.5.2 Example vertex lists and volume checks (illustrative)

The following snippets show executable decompositions for standard synergetics volumes. Each tetra volume is computed via `ace_tetravolume_5x5` and summed.

Octahedron ($V = 4$): the octahedron with vertices $\pm 2\hat{e}_i$ (edge $2\sqrt{2}$, matching the unit IVM tetra) decomposes into eight origin-apex orthant tetras, each of volume $\frac{1}{2}$:

```

1 from quadmath.core.quadray import Quadray, ace_tetravolume_5x5
2
3 o = Quadray(0,0,0,0)
4 axes = [
5     Quadray(1,0,0,1), Quadray(0,1,1,0), # +x, -x
6     Quadray(1,1,0,0), Quadray(0,0,1,1), # +y, -y
7     Quadray(1,0,1,0), Quadray(0,1,0,1), # +z, -z
8 ]
9 V_oct = sum(
10     ace_tetravolume_5x5(o, x, y, z)
11     for x in axes[0:2] for y in axes[2:4] for z in axes[4:6]
12 ) # 8 * 1/2 = 4

```

Cube ($V = 3$): the cube with XYZ vertices $(\pm 1, \pm 1, \pm 1)$ (edge 2) decomposes into the inscribed tetra on four alternating cube vertices ($V = 1$) plus four corner tetras ($V = \frac{1}{2}$ each):

```

1 from quadmath.core.quadray import Quadray, ace_tetravolume_5x5
2
3 inner = ace_tetravolume_5x5(
4     Quadray(1,0,0,0), Quadray(0,1,0,0), Quadray(0,0,1,0), Quadray(0,0,0,1),
5 ) # 1
6 corner = ace_tetravolume_5x5(
7     Quadray(0,1,1,1), Quadray(0,0,0,1), Quadray(0,1,0,0), Quadray(0,0,1,0),
8 ) # 1/2
9 V_cube = inner + 4 * corner # 1 + 4*(1/2) = 3

```

Notes.

- The octahedron and cube decompositions above execute exactly as shown (verified against `ace_tetravolume_5x5`). Other equivalent tilings are possible.
- Volumes are invariant to adding (k, k, k, k) to each vertex of a tetra (projective normalization), which the 5×5 determinant respects.

3.6 Integer Volume Quantization

For a tetrahedron with vertices $P_0..P_3$ in the Quadray integer lattice (Fuller.4D), see Eq. (1) in the equations appendix.

- With integer quadray coordinates the projected determinant is an exact integer, and the IVM tetravolume is the exact rational $|\det|/4$ (see Eq. (7)), implemented with `Fractions.Fraction` in `integer_tetra_volume` and `ace_tetravolume_5x5`. Volumes are integral for tetrahedra that tile into unit IVM tetras (determinant divisible

- by 4) and fractional otherwise: the primitive tetrahedron spanned by $(0, 0, 0, 0)$, $(1, 0, 0, 0)$, $(0, 1, 0, 0)$, $(0, 0, 1, 0)$ has determinant 1 and exact volume $\frac{1}{4}$.
- Unit conventions: the unit IVM tetrahedron (origin plus three IVM neighbor moves, permutations of $(2, 1, 1, 0)$) has determinant 4 and volume exactly 1 (synergetics).

Notes.

- $P_0, \dots, P_3$ are tetrahedron vertices in Quadray coordinates.
- V_{xyz} in Eq. (1) is the Euclidean (XYZ) volume; converting it to IVM tetra-units requires the synergetics scale factor $S3 = \sqrt{9/8}$ in sphere-radius units (Eq. (4)). For lattice tetrahedra, the direct quadray formula Eq. (7) yields the IVM volume exactly, without unit conversion.
- Background and variations are discussed under Tetrahedron volume formulas: [Tetrahedron - volume](#).

Tom Ace 5×5 determinant (tetravolume directly from quadrays), see Eq. (2) in the equations appendix.

This returns the same exact tetravolumes as `integer_tetra_volume` for every lattice tetrahedron: $|\det_{5 \times 5}|$ always equals the magnitude of the projected 3×3 determinant, so both implementations agree exactly (both return `fractions.Fraction` values $|\det|/4$).

Notes.

- Rows correspond to the Quadray 4-tuples of the four vertices with a final affine column of ones; the last row enforces projective normalization.
- The factor $\frac{1}{4}$ returns tetravolumes in IVM units consistent with synergetics. See also [Quadray coordinates](#).

Equivalently, define the 5×5 matrix of quadray coordinates augmented with an affine 1 as shown in Eq. (3) in the equations appendix.

Points vs vectors: subtracting points is shorthand for forming edge vectors. We treat quadray 4-tuples as vectors from the origin; differences like $(P_1 - P_0)$ mean “edge vectors,” avoiding ambiguity between “points” and “vectors.”

Equivalently via Cayley-Menger determinant (Coxeter.4D/Euclidean lengths) ([Cayley-Menger determinant](#)), see Eq. (5) in the equations appendix.

References: [Cayley-Menger determinant](#), lattice tetrahedra discussions in geometry texts; see also [Tetrahedron - volume](#). Code: `integer_tetra_volume`, `ace_tetravolume_5x5`.

Notes.

- **Pairwise distances:** d_{ij} are Euclidean distances between vertices P_i and P_j .
- **Length-only formulation:** Cayley-Menger provides a length-only formula for simplex volumes, here specialized to tetrahedra; see the canonical reference above.

Table 1: Polyhedra tetravolumes in IVM units (edge length equal to the unit tetra edge).

Polyhedron (edge = tetra edge)	Volume (tetra-units)
Regular Tetrahedron	1
Cube	3
Octahedron	4
Rhombic Dodecahedron	6
Cuboctahedron (Vector Equilibrium)	20
Truncated Octahedron	20

3.7 Distances and Metrics

Distance definitions depend on the chosen embedding and normalization. For cross-references to information geometry, see [Eq. \(FIM\)](#) and [natural gradient](#) in the Equations appendix.

3.8 XYZ determinant and S3 conversion

Given XYZ coordinates of tetrahedron vertices (x_i, y_i, z_i), the Euclidean volume is computed as shown in Eq. (4) in the equations appendix.

Convert to IVM units via $V_{ivm} = S3 \cdot V_{xyz}$ with $S3 = \sqrt{9/8}$. See background discussion under [Tetrahedron - volume](#).

3.8.1 Alternative Volume Formulas

- **Piero della Francesca (PdF) formula**, which consumes edge lengths and returns Euclidean volumes. See Eq. (6) in the equations appendix.

Convert to IVM units via $V_{ivm} = S3 \cdot V_{xyz}$ with $S3 = \sqrt{9/8}$. See background discussion under [Tetrahedron - volume](#).

- **Gerald de Jong (GdJ) formula**, which natively returns tetravolumes. See Eq. (7) in the equations appendix.

In Quadray coordinates, one convenient native form uses edge-vector differences and an integer-preserving determinant (agreeing with Ace 5×5):

where each column is formed from Quadray component differences of $P_1 - P_0, P_2 - P_0, P_3 - P_0$ projected via $\pi(P) = (a - d, b - d, c - d)$ to the synergetics 3D slice (matching `integer_tetra_volume`); integer arithmetic is exact and the factor $\frac{1}{4}$ produces IVM tetravolumes. See de Jong's Quadray notes and Urner's implementations for derivations ([Quadray coordinates](#)).

- Euclidean embedding distance via appropriate linear map from quadray to R^3 .
- Information geometry metric: Fisher Information Matrix (FIM)
 - $FIM[i, j] = \mathbb{E}[\partial_{\theta_i} \log p(x; \theta) \partial_{\theta_j} \log p(x; \theta)]$
 - Acts as Riemannian metric; natural gradient uses $FIM^{-1} \nabla \theta$. See [Fisher information](#).

3.9 Fisher Geometry in Quadray Space

- Symmetries of quadray lattices often induce near block-diagonal FIM.
- Determinant and spectrum characterize conditioning and information concentration.

3.10 Practical Methods

3.11 Tetravolumes with Quadrays

- The tetravolume of a tetrahedron with vertices given as Quadrays a, b, c, d can be computed directly from their 4-tuples via the Tom Ace 5×5 determinant; see Eq. (2) for the canonical form.
- Unit regular tetrahedron from origin: with $o=(0,0,0,0)$, $p=(2,1,0,1)$, $q=(2,1,1,0)$, $r=(2,0,1,1)$, we have $V_{ivm}(o, p, q, r)=1$. Doubling each vector scales volume by 8, as expected.
- Equivalent length-based formulas agree with the 5×5 determinant:

- Cayley-Menger: $288 V^2 = \det \begin{pmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & d_{01}^2 & d_{02}^2 & d_{03}^2 \\ 1 & d_{10}^2 & 0 & d_{12}^2 & d_{13}^2 \\ 1 & d_{20}^2 & d_{21}^2 & 0 & d_{23}^2 \\ 1 & d_{30}^2 & d_{31}^2 & d_{32}^2 & 0 \end{pmatrix}.$

- Piero della Francesca (PdF) Heron-like formula (converted to IVM via $S3 = \sqrt{9/8}$).

Let edge lengths meeting at a vertex be a, b, c , and the opposite edges be d, e, f . The Euclidean volume satisfies

Convert to IVM units via $V_{ivm} = S3 \cdot V_{xyz}$ with $S3 = \sqrt{9/8}$. See background discussion under [Tetrahedron - volume](#).

- Gerald de Jong (GdJ) formula, which natively returns tetravolumes.

In Quadray coordinates, one convenient native form uses edge-vector differences and an integer-preserving determinant (agreeing with Ace 5×5):

where each column is formed from Quadray component differences of $P_1 - P_0, P_2 - P_0, P_3 - P_0$ projected via $\pi(P) = (a - d, b - d, c - d)$ to the synergetics 3D slice (matching `integer_tetra_volume`); integer arithmetic is exact and the factor $\frac{1}{4}$ produces IVM tetravolumes. See de Jong's Quadray notes and Urner's implementations for derivations ([Quadray coordinates](#)).

3.11.1 Bridging and native tetravolume formulas

- **Lengths (bridging):** PdF and Cayley-Menger (CM) consume Cartesian lengths (XYZ) and produce Euclidean volumes; convert to IVM units via $S3 = \sqrt{9/8}$.
- **Quadray-native:** Gerald de Jong (GdJ) returns IVM tetravolumes directly (no XYZ bridge). Tom Ace's 5×5 coordinate formula is likewise native IVM. All agree numerically with CM+S3 on shared cases.

References and discussion: [Urner - Flickr diagram](#). For computational implementations and educational materials, see the [Resources](#) section.

Figure: automated comparison (native Ace 5×5 vs CM+S3) across small examples (see script `sympy_formalisms.py`). The figure and source CSV/NPZ are in `quadmath/output/`.

3.11.2 Short Python snippets

```
1 from quadmath.core.quadray import Quadray, ace_tetravolume_5x5
2
3 o = Quadray(0,0,0,0)
4 p = Quadray(2,1,0,1)
5 q = Quadray(2,1,1,0)
6 r = Quadray(2,0,1,1)
7 assert ace_tetravolume_5x5(o,p,q,r) == 1 # unit IVM tetra
```

```
1 import numpy as np
2 from quadmath.core.cayley_menger import ivm_tetra_volume_cayley_menger
3
4 # Example: regular tetrahedron with edge length 1 (XYZ units)
5 d2 = np.ones((4,4)) - np.eye(4) # squared distances
6 V_ivm = ivm_tetra_volume_cayley_menger(d2) # = 1/8 in IVM tetra-units
```

```
1 # SymPy implementation of Tom Ace 5x5 (symbolic determinant)
2 from sympy import Matrix
3
4 def qvolume(q0, q1, q2, q3):
5     M = Matrix([
6         q0 + (1,),
7         q1 + (1,),
8         q2 + (1,),
9         q3 + (1,),
10        [1, 1, 1, 1, 0],
11    ])
12    return abs(M.det()) / 4
```

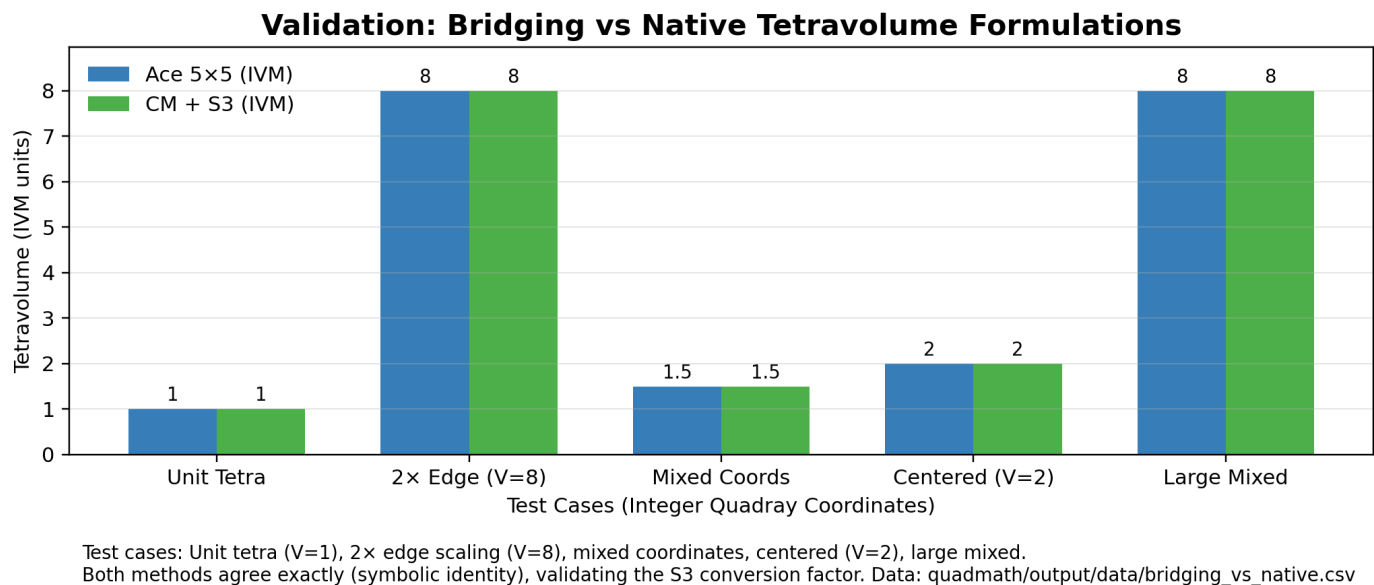


Figure 4: **Validation of bridging vs native tetravolume formulations across canonical examples.** This bar chart compares IVM tetravolumes computed via two independent methods: the “bridging” approach using Cayley-Menger determinants on Euclidean edge lengths converted to IVM units via the synergetics factor $S3 = \sqrt{9/8}$, versus the “native” approach using Tom Ace’s 5×5 determinant formula that operates directly on Quadray coordinates without XYZ intermediates. **Test cases:** Unit tetrahedron (V=1), 2× edge scaling (V=8), mixed coordinate tetrahedron, centered tetrahedron (V=3), and large mixed tetrahedron, all using integer Quadray coordinates. **Results:** The overlapping bars demonstrate numerical agreement at machine precision between the length-based Coxeter.4D approach (Cayley-Menger + S3 conversion) and the coordinate-based Fuller.4D approach (Ace 5×5), confirming the mathematical equivalence of these formulations under synergetics unit conventions. Raw numerical data saved as `bridging_vs_native.csv` for reproducibility and further analysis.

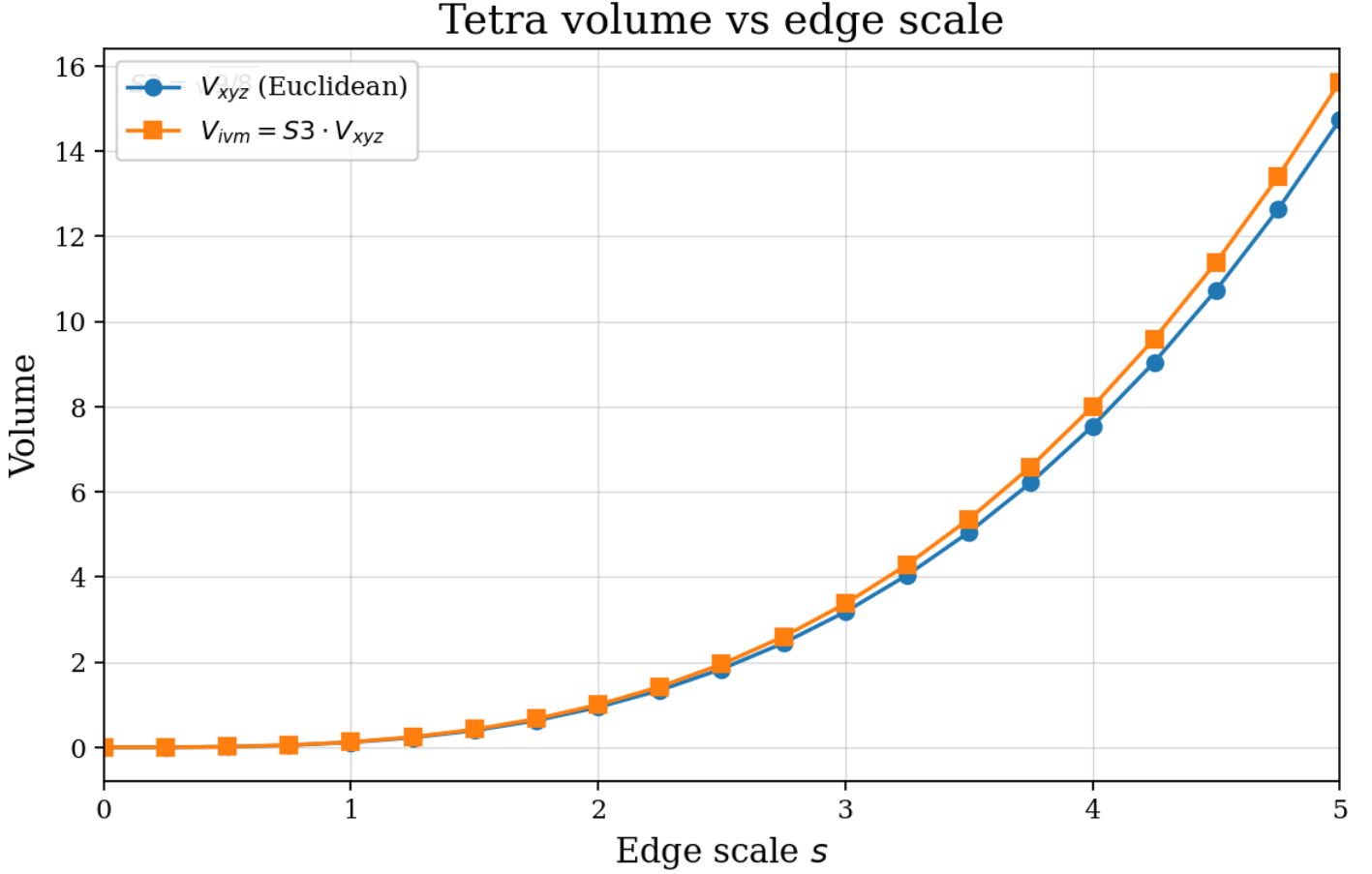


Figure 5: **Tetrahedron volume scaling relationships: Euclidean vs IVM unit conventions.** This plot demonstrates the mathematical relationship between edge length scaling and tetravolume under both Euclidean (XYZ) and IVM (synergetics) unit conventions. **X-axis:** Edge length scaling factor (0 to 5.0). **Y-axis:** Tetrahedron volume in respective units. **Blue line (Euclidean):** Volume scales as the cube of edge length, following the standard $V = \frac{\sqrt{2}}{12} \cdot L^3$ relationship for regular tetrahedra. **Orange line (IVM):** Volume scales as the cube of edge length but in IVM tetra-units, following $V_{ivm} = \frac{1}{8} \cdot L^3$ where the regular tetrahedron with unit edge has volume 1/8. **Key insight:** The ratio between these two scaling laws is the synergetics factor $S3 = \sqrt{9/8} \approx 1.06066$, which converts between Euclidean and IVM volume conventions. **Mathematical foundation:** This scaling relationship demonstrates how both conventions preserve the cubic scaling relationship, but with different fundamental units reflecting the different geometric assumptions of Coxeter.4D (Euclidean) versus Fuller.4D (synergetics) frameworks. The plot provides the theoretical foundation for understanding volume conversions and scaling behavior in the IVM system.

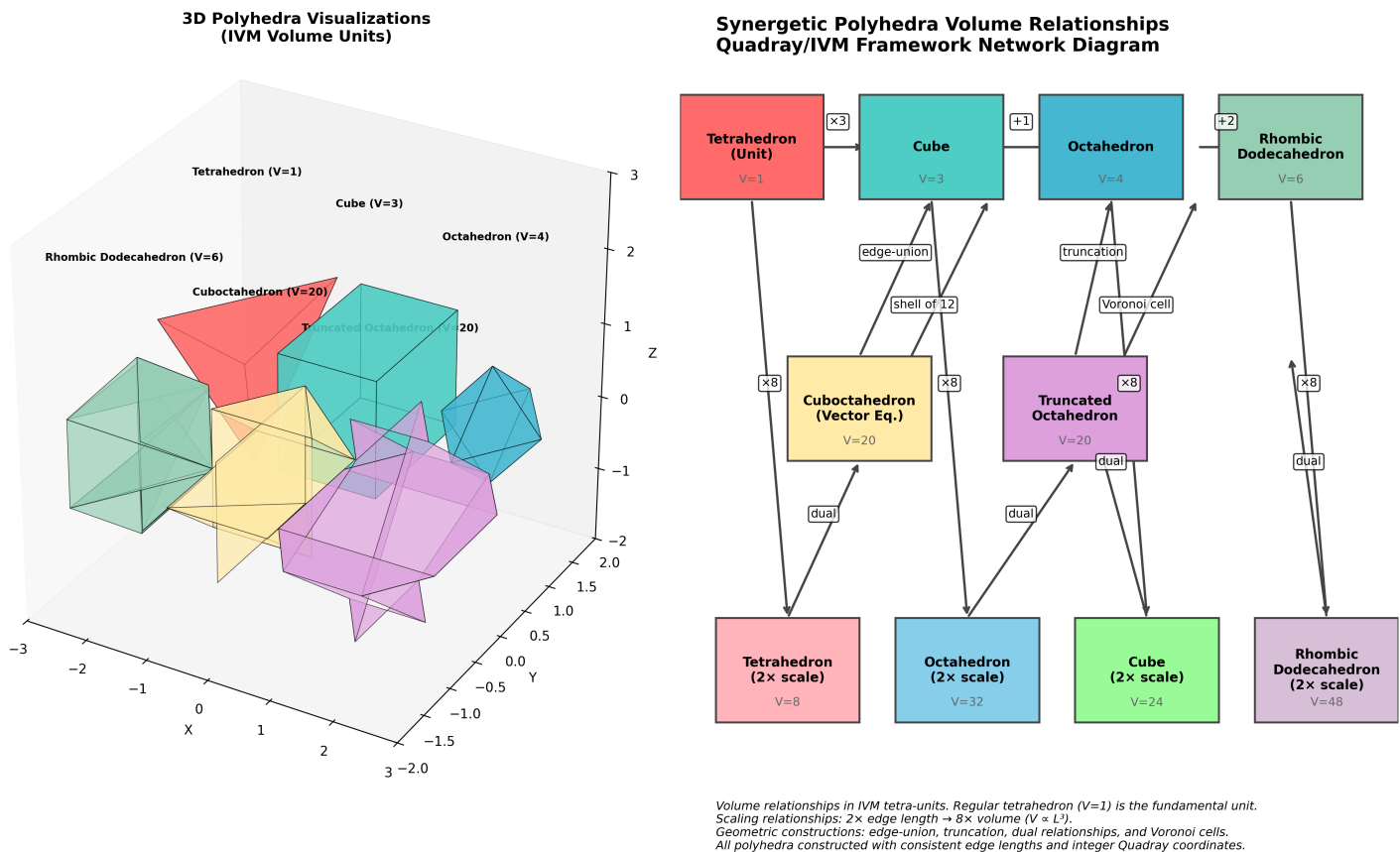


Figure 6: Synergetic polyhedra volume relationships in the Quadray/IVM framework (comprehensive visualization). This figure combines 3D polyhedra visualizations with an extended network diagram showing integer volume relationships among key synergetic polyhedra. **Left panel (3D visualizations):** Color-coded polyhedra including regular tetrahedron ($V=1$, fundamental unit), cube ($V=3$), octahedron ($V=4$), rhombic dodecahedron ($V=6$), cuboctahedron ($V=20$), and truncated octahedron ($V=20$), all constructed with consistent edge lengths and proper geometric faces. **Right panel (network diagram):** Extended volume relationships showing fundamental shapes ($V=1, 3, 4, 6$), complex constructions ($V=20$), and scaling relationships ($2 \times$ edge length $\rightarrow 8 \times$ volume). **Additional polyhedra:** Includes truncated octahedron ($V=20$) and scaled variants demonstrating the “third power” volume scaling law $V \propto L^3$ in IVM units. **Geometric constructions:** Edge-union relationships, truncation operations, dual polyhedra, and Voronoi cell constructions. **Fuller.4D significance:** These integer volume ratios reflect the quantized nature of space-filling in synergetics, where the regular tetrahedron provides a natural unit container and other polyhedra emerge as integer multiples, supporting discrete geometric computation and exact lattice-based optimization methods. All constructions respect the IVM unit convention where the regular tetrahedron has tetravolume 1.

```

1 # Symbolic variant with SymPy (exact radicals)
2 from sympy import Matrix, sqrt, simplify
3 from quadmath.core.symbolic import cayley_menger_volume_symbolic, convert_xyz_volume_to_ivm_symbolic
4
5 d2 = Matrix([[0,1,1,1],[1,0,1,1],[1,1,0,1],[1,1,1,0]])
6 V_xyz_sym = cayley_menger_volume_symbolic(d2) # sqrt(2)/12
7 V_ivm_sym = simplify(convert_xyz_volume_to_ivm_symbolic(V_xyz_sym)) # 1/8

```

3.11.3 Random tetrahedra in the IVM (integer volumes)

- The 12 CCP directions are the permutations of (2, 1, 1, 0). Random walks on this move set generate integer-coordinate Quadrays; resulting tetrahedra have integer tetravolumes.

```

1 from itertools import permutations
2 from random import choice
3 from quadmath.core.quadray import Quadray, ace_tetravolume_5x5
4
5 moves = [Quadray(*p) for p in set(permutations((2,1,1,0)))]
6
7 def random_walk(start: Quadray, steps: int) -> Quadray:
8     cur = start
9     for _ in range(steps):
10         m = choice(moves)
11         cur = Quadray(cur.a+m.a, cur.b+m.b, cur.c+m.c, cur.d+m.d)
12     return cur
13
14 A = random_walk(Quadray(0,0,0,0), 1000)
15 B = random_walk(Quadray(0,0,0,0), 1000)
16 C = random_walk(Quadray(0,0,0,0), 1000)
17 D = random_walk(Quadray(0,0,0,0), 1000)
18 V = ace_tetravolume_5x5(A,B,C,D) # exact IVM volume as a Fraction

```

3.11.4 Algebraic precision

- Determinants via floating-point introduce rounding noise. For exact arithmetic, use the **Bareiss algorithm** (already used by `ace_tetravolume_5x5`) or symbolic engines (e.g., `sympy`). For random-walk examples with integer quadray inputs, volumes are exact rationals (`fractions.Fraction`): integral when the tetrahedron decomposes into unit IVM tetras, fractional (in quarters) otherwise.
- When computing via XYZ determinants, high-precision floats (e.g., `gmpy2.mpfr`) or symbolic matrices avoid vestigial errors; round at the end if the underlying result is known to be integral.

3.11.5 XYZ determinant and the S3 conversion

- Using XYZ coordinates of the four vertices: see Eq. (4) for the determinant form and the S3 conversion to IVM units.

3.11.6 D^3 vs R^3 : 60° “closing the lid” vs orthogonal “cubing”

- **IVM (D^3) heuristic:** From a 60–60–60 corner, three non-negative edge lengths A, B, C along quadray directions enclose a tetrahedron by “closing the lid.” In synergetics, the tetravolume scales as the simple product ABC under IVM conventions (unit regular tetra has volume 1). By contrast, in the orthogonal (R^3) habit, one constructs a full parallelepiped (12 edges); the tetra occupies one-sixth of the triple product of edge vectors. The IVM path is more direct for tetrahedra.

- **Pedagogical note:** Adopt a vector-first approach. Differences like $(P_i - P_0)$ denote edge vectors; Quadrays and Cartesian can be taught in parallel as vector languages on the same Euclidean container.

Reference notebook with worked examples and code: See the [Resources](#) section for comprehensive educational materials and computational implementations.

See implementation: `tetra_volume_cayley_menger`.

- **Lattice projection:** round to nearest integer quadray; renormalize to maintain non-negativity and a minimal zero.

3.12 Practical Implementation Workflow

The methods described in this document follow a unified workflow that ensures mathematical correctness, computational efficiency, and reproducibility:

3.12.1 1. Mathematical Formulation

- **Theoretical foundations:** Establish mathematical relationships between frameworks (Coxeter.4D $\leftrightarrow$ Fuller.4D $\leftrightarrow$ Einstein.4D)
- **Coordinate transformations:** Define linear maps between coordinate systems with exact arithmetic
- **Volume formulations:** Implement multiple approaches (Ace 5×5, Cayley-Menger, symbolic) for cross-validation

3.12.2 2. Implementation Strategy

- **Exact arithmetic:** Use Bareiss algorithm for integer determinants, symbolic computation for exact results
- **Numerical stability:** Implement ridge regularization, condition number monitoring, and error handling
- **Cross-validation:** Ensure different formulations produce identical results on test cases

3.12.3 3. Testing and Validation

- **Unit tests:** 100% coverage of all mathematical functions with edge case handling
- **Integration tests:** Validate coordinate transformations and volume calculations across frameworks
- **Numerical validation:** Compare floating-point implementations with exact symbolic results

3.12.4 4. Documentation and Reproducibility

- **Code-documentation sync:** All mathematical concepts have corresponding implementations in `src/`
- **Figure generation:** Scripts in `quadmath/scripts/` generate all figures from source code
- **Data export:** CSV/NPZ files accompany all figures for complete reproducibility

3.12.5 5. Optimization and Extension

- **Lattice constraints:** Integer volume quantization enables discrete optimization methods
- **Information geometry:** Fisher metric guides natural gradient descent on parameter manifolds
- **Active inference:** Free energy minimization drives both perception and action updates

This workflow ensures that mathematical theory, computational implementation, and practical application remain coherent and verifiable throughout the development process.

3.13 Code methods (anchors)

3.13.1 Core Quadray operations

Quadray Source: `src/quadmath/core/quadray.py` — Quadray vector class with non-negative components and at least one zero (Fuller.4D).

DEFAULT_EMBEDDING Source: `src/quadmth/core/quadray.py` — canonical 3×4 symmetric embedding matrix for Quadray to XYZ conversion.

to_xyz Source: `src/quadmth/core/quadray.py` — map quadray to $\mathbb{R}^3$ via a 3×4 embedding matrix (Fuller.4D $\rightarrow$ Coxeter.4D slice).

magnitude Source: `src/quadmth/core/quadray.py` — return the Euclidean magnitude of q under the given embedding (vector norm).

dot Source: `src/quadmth/core/quadray.py` — return Euclidean dot product $\langle q_1, q_2 \rangle$ under the given embedding.

3.13.2 Quaternion operations

qmul Source: `src/quadmth/core/quadray.py` — Hamilton product of two quaternions in scalar-first (w, x, y, z) order ($q = w + x i + y j + z k$), an independent representation from the Quadray lattice components (a, b, c, d) and from the XYZ triples produced by `to_xyz`.

qconjugate Source: `src/quadmth/core/quadray.py` — conjugate $(w, -x, -y, -z)$ of a quaternion; for a unit quaternion the conjugate is the multiplicative inverse, so $q * qconjugate(q)$ equals $(1, 0, 0, 0)$ up to floating-point error.

grotate Source: `src/quadmth/core/quadray.py` — Rodrigues rotation of a 3-vector by a unit quaternion via $v' = q(0, v)q^*$, composed with `qmul` and `qconjugate`; q must be unit (within $1e-9$) and the rotation magnitude it encodes, $2 \operatorname{atan2}(\|(x, y, z)\|, w)$, must match $|\text{angle}|$ modulo 2π within $1e-9$, so a mismatched (q, angle) pair raises instead of silently rotating by the wrong amount.

slerp Source: `src/quadmth/core/quadray.py` — shortest-arc spherical linear interpolation between unit quaternions: when $\langle q_a, q_b \rangle < 0$ the second quaternion is negated first (q and $-q$ encode the same rotation) so the path always takes the shorter arc on the rotation sphere; nearly parallel inputs (including exactly antipodal pairs after sign alignment) fall back to normalized lerp to avoid dividing by $\sin \theta \approx 0$, and the endpoints are exact. The site path traced by these interpolations is visualized by `plot_slerp_path` in `src/quadmth/viz/plots.py` (figure below).

rotate_about_axis Source: `src/quadmth/core/quadray.py` — axis-angle convenience wrapper: builds the unit quaternion $q = (\cos(\theta/2), \sin(\theta/2) \hat{u})$ from the normalized rotation axis (any non-zero scale accepted; the zero vector is rejected) and delegates to `grotate`, reducing the angle to $(-\pi, \pi]$ first so rotation is right-handed about the axis and `grotate`'s encoded-angle validation stays exact.

The property checks in `src/quadmth/validate/validate.py` — normalization, conjugate-inverse, double cover, slerp midpoint, and associativity — consume these operations over quaternion sequences via `run_validation`, which collects the resulting deterministic validation reports.

3.13.3 Quaternion metrics

angle_error Source: `src/quadmth/core/metrics.py` — geodesic rotation angle $2 \arccos(|\langle q_1, q_2 \rangle|)$ between two quaternions, in radians and confined to $[0, \pi]$ by clamping the `acos` argument; the metric is symmetric in its arguments and invariant under $q \rightarrow -q$ (the quaternion double cover), with inputs normalized internally so non-unit but non-zero quaternions are accepted.

Slerp rotation path of site (2, 0, 0, 0)

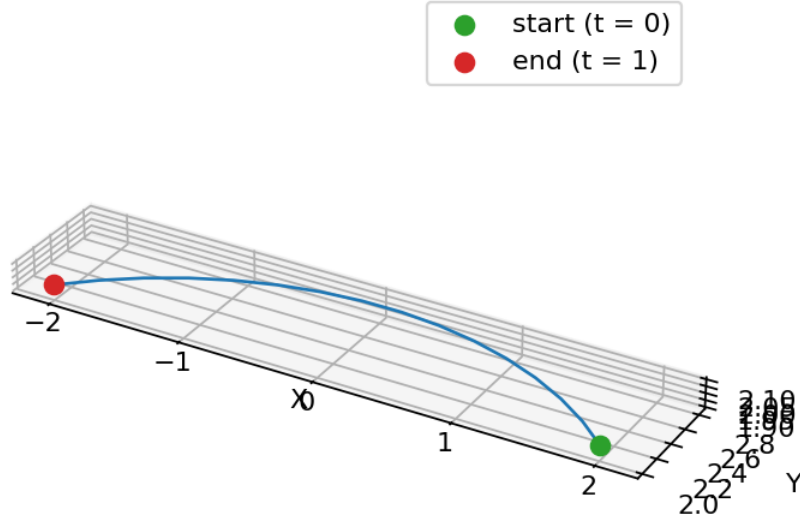


Figure 7: **Shortest-arc slerp geodesic from the identity to a $\pi/2$ rotation about the z axis.** The path traced by the canonical $(2, 0, 0, 0)$ quadray axis point (embedded at $(2, 2, 2)$ under `DEFAULT_EMBEDDING`) as the rotation quaternion is interpolated from the identity $(1, 0, 0, 0)$ to $(\cos \frac{\pi}{4}, 0, 0, \sin \frac{\pi}{4})$ — a right-handed $\pi/2$ rotation about the z axis — over 16 samples of $t \in [0, 1]$, rendered by `plot_slerp_path` in `src/quadmth/viz/plots.py` via the script `quadmth/scripts/quaternion_gallery.py`. Because `slerp` negates the second quaternion whenever $\langle q_0, q_1 \rangle < 0$, the interpolation always follows the shorter arc of the geodesic on the rotation sphere: the rotation angle grows linearly from 0 to $\pi/2$, so the site sweeps the quarter circle of radius $2\sqrt{2}$ about the z axis at constant height $z = 2$ (blue line), from the start marker at $t = 0$ (green) to the end marker at $t = 1$ (red). Reproduce with `uv run python quadmath/scripts/quaternion_gallery.py`.

quat_log_euclidean_dispersion Source: `src/quadmath/core/metrics.py` — root-mean-square chordal dispersion of a quaternion set about its normalized component-wise mean, $\sqrt{\text{mean}_i \|q_i - m\|^2}$ in $\mathbb{R}^4$, with signs aligned to the first quaternion before averaging because q and $-q$ encode the same rotation; returns 0 for a single quaternion and raises if the sign-aligned mean vanishes.

3.13.4 Volume calculations

integer_tetra_volume Source: `src/quadmath/core/quadray.py` — exact projected 3×3 determinant over 4, i.e. the IVM tetravolume $|\det|/4$ as a `fractions.Fraction`.

ace_tetravolume_5x5 Source: `src/quadmath/core/quadray.py` — Tom Ace 5×5 determinant in IVM units.

tetra_volume_cayley_menger Source: `src/quadmath/core/cayley_menger.py` — length-based formula (XYZ units).

ivm_tetra_volume_cayley_menger Source: `src/quadmath/core/cayley_menger.py` — Cayley-Menger volume converted to IVM units.

3.13.5 Coordinate conversions

urner_embedding Source: `src/quadmath/lattice/conversions.py` — canonical XYZ embedding.

quadray_to_xyz Source: `src/quadmath/lattice/conversions.py` — apply embedding matrix to map Quadray to XYZ.

3.13.6 Linear algebra utilities

bareiss_determinant_int Source: `src/quadmath/core/linalg_utils.py` — exact integer Bareiss determinant.

3.13.7 Optimization methods

nelder_mead_quadray Source: `src/quadmath/optimize/nelder_mead_quadray.py` — Nelder-Mead optimization adapted to the integer quadray lattice.

discrete_ivm_descent Source: `src/quadmath/optimize/discrete_variational.py` — greedy integer-valued descent over the IVM using canonical neighbor moves; returns a `DiscretePath` with visited Quadrays and objective values.

neighbor_moves_ivm Source: `src/quadmath/optimize/discrete_variational.py` — return the 12 canonical IVM neighbor moves as Quadray deltas.

apply_move Source: `src/quadmath/optimize/discrete_variational.py` — apply a lattice move and normalize to the canonical representative.

3.13.8 Information geometry methods

fisher_information_matrix Source: `src/quadmath/inference/information.py` — empirical outer-product estimator.

fisher_information_quadray Source: `src/quadmath/inference/information.py` — compute Fisher information matrix in both Cartesian and Quadray coordinates.

natural_gradient_step Source: `src/quadmath/inference/information.py` — damped inverse-Fisher step.

free_energy Source: `src/quadmath/inference/information.py` — discrete-state variational free energy.

expected_free_energy Source: `src/quadmath/inference/information.py` — canonical expected free energy G for Active Inference (epistemic KL + ambiguity – pragmatic; see the equations appendix).

active_inference_step Source: `src/quadmath/inference/information.py` — joint perception-action update step in Active Inference.

information_geometric_distance Source: `src/quadmath/inference/information.py` — compute information-geometric distance between two points.

perception_update Source: `src/quadmath/inference/information.py` — continuous-time perception update: $d\mu/dt = D\mu - dF/d\mu$.

action_update Source: `src/quadmath/inference/information.py` — continuous-time action update: $da/dt = -dF/da$.

finite_difference_gradient Source: `src/quadmath/inference/information.py` — compute numerical gradient of a scalar function via central differences.

3.13.9 Metrics and analysis

shannon_entropy Source: `src/quadmath/core/metrics.py` — Shannon entropy $H(p)$ for a discrete distribution.

information_length Source: `src/quadmath/core/metrics.py` — path length in information space via gradient-weighted arc length.

fim_eigenspectrum Source: `src/quadmath/core/metrics.py` — eigen-decomposition of a Fisher information matrix.

fisher_condition_number Source: `src/quadmath/core/metrics.py` — compute the condition number of the Fisher information matrix.

fisher_curvature_analysis Source: `src/quadmath/core/metrics.py` — comprehensive analysis of Fisher information matrix curvature.

fisher_quadray_comparison Source: `src/quadmath/core/metrics.py` — compare Fisher information matrices between coordinate systems.

3.13.10 Examples and utilities

example_ivm_neighbors Source: `src/quadmath/core/examples.py` — return the 12 nearest IVM neighbors as permutations of $\{2,1,1,0\}$ (neighbor-move-set role).

example_cuboctahedron_neighbors Source: `src/quadmath/core/examples.py` — return twelve-around-one IVM neighbors (vector-equilibrium-shell role). Same set as `example_ivm_neighbors`; both public names are documented API sharing one sorted, deterministic implementation.

example_cuboctahedron_vertices_xyz Source: `src/quadmath/core/examples.py` — return XYZ coordinates for the twelve-around-one neighbors.

example_partition_tetra_volume Source: `src/quadmath/core/examples.py` — construct a tetrahedron from the four-fold partition and return the exact IVM tetravolume (`fractions.Fraction`).

3.13.11 Symbolic computation

cayley_menger_volume_symbolic Source: `src/quadmath/core/symbolic.py` — return symbolic Euclidean tetrahedron volume from squared distances.

convert_xyz_volume_to_ivm_symbolic Source: `src/quadmath/core/symbolic.py` — convert a symbolic Euclidean volume to IVM tetravolume via S3.

3.13.12 Visualization and animation

animate_discrete_path Source: `src/quadmath/viz/visualize.py` — animate a `DiscretePath` to MP4; saves CSV/NPZ trajectory to `quadmath/output/`.

plot_ivm_neighbors Source: `src/quadmath/viz/visualize.py` — scatter the 12 IVM neighbor points in 3D.

plot_partition_tetrahedron Source: `src/quadmath/viz/visualize.py` — plot the four-fold partition as a labeled tetrahedron in 3D.

animate_simplex Source: `src/quadmath/viz/visualize.py` — animate simplex evolution across iterations.

plot_simplex_trace Source: `src/quadmath/viz/visualize.py` — plot per-iteration diagnostics for Nelder-Mead.

3.13.13 Path and file utilities

get_repo_root Source: `src/quadmath/paths.py` — heuristically find repository root by walking up from start.

get_output_dir Source: `src/quadmath/paths.py` — return `quadmath/output` path at the repo root and ensure it exists.

get_data_dir Source: `src/quadmath/paths.py` — return `quadmath/output/data` path and ensure it exists.

get_figure_dir Source: `src/quadmath/paths.py` — return `quadmath/output/figures` path and ensure it exists.

3.13.14 Additional Nelder-Mead components

SimplexState Source: `src/quadmath/optimize/nelder_mead_quadray.py` — optimization trajectory state containing vertices, values, volume, and history.

order_simplex Source: `src/quadmath/optimize/nelder_mead_quadray.py` — sort vertices by objective value ascending and return paired lists.

centroid_excluding Source: `src/quadmath/optimize/nelder_mead_quadray.py` — integer centroid of three vertices, excluding the specified index.

project_to_lattice Source: `src/quadmath/optimize/nelder_mead_quadray.py` — project a quadray to the canonical lattice representative via `normalize`.

compute_volume Source: `src/quadmath/optimize/nelder_mead_quadray.py` — exact IVM tetra-volume (`fractions.Fraction, |det|/4`) from the first four vertices.

3.13.15 Discrete variational components

DiscretePath Source: `src/quadmath/optimize/discrete_variational.py` — optimization trajectory on the integer quadray lattice.

3.13.16 Glossary generation

build_api_index Source: `src/quadmath/tools/glossary_gen.py` — build API index from source directory.

generate_markdown_table Source: `src/quadmath/tools/glossary_gen.py` — generate markdown table from API entries.

inject_between_markers Source: `src/quadmath/tools/glossary_gen.py` — inject payload between markers in markdown text.

3.13.17 Geometry utilities

minkowski_interval Source: `src/quadmath/core/geometry.py` — return the Minkowski interval squared ds^2 (Einstein.4D).

Relevant tests (`tests/`):

- `test_quadray.py` (unit IVM tetra, primitive tetra = $1/4$, exact scaled volume, Ace vs. integer agreement)
- `test_quadray_cov.py` (Ace determinant basic check)
- `test_cayley_menger.py` (regular tetra volume in XYZ units)
- `test_linalg_utils.py` (Bareiss determinant behavior)
- `test_examples.py`, `test_examples_cov.py` (neighbors, examples)
- `test_metrics.py`, `test_metrics_cov.py`, `test_information.py`, `test_paths.py`, `test_paths_cov.py`

3.13.18 Comprehensive test coverage

The test suite provides 100% coverage of all source modules with the following focus areas:

Core functionality tests

- `test_quadray.py`: Quadray class operations, normalization, volume calculations, vector operations
- `test_cayley_menger.py`: Cayley-Menger determinant validation, IVM conversion accuracy
- `test_linalg_utils.py`: Bareiss algorithm correctness, edge cases, numerical stability

Optimization and variational methods

- `test_nelder_mead_visual.py`: Nelder-Mead adaptation to quadray lattice, simplex evolution
- `test_discrete_variational.py`: IVM neighbor moves, discrete descent algorithms, path tracking
- `test_active_inference.py`: Free energy minimization, perception-action updates

Information geometry and metrics

- `test_information.py`: Fisher information matrix computation, natural gradient steps
- `test_metrics.py`: FIM eigendecomposition, curvature analysis, coordinate system comparisons
- `test_information_cov.py`: Extended coverage of information geometry functions

Examples and utilities

- `test_examples.py`: IVM neighbor constructions, polyhedra volume relationships
- `test_examples_cov.py`: Extended example function coverage
- `test_paths.py`: Repository structure utilities, output directory management

Visualization and symbolic computation

- `test_visualize.py`: Plotting functions, animation generation, data export
- `test_visualize_cov.py`: Extended visualization coverage, edge case handling
- `test_symbolic.py`: SymPy symbolic volume calculations, exact arithmetic
- `test_symbolic_cov.py`: Symbolic computation error handling
- `test_sympy_formalisms.py`: End-to-end symbolic workflow validation

API and documentation generation

- `test_glossary_gen.py`: Automatic API index generation, markdown table formatting
- `test_glossary_gen_cov.py`: Extended glossary generation coverage

3.14 Reproducibility checklist

- **Complete implementation coverage**: All formulas and methods discussed in the paper are implemented in `src/` modules and verified by comprehensive test suites in `tests/`.
- **Exact arithmetic for integer inputs**: Determinants are computed using the Bareiss algorithm for exact integer arithmetic; floating-point paths are used only where appropriate and results are converted (e.g., via S3) as specified.
- **Deterministic random experiments**: Random-walk experiments use fixed seeds and produce integer volumes; Ace 5×5 determinant agrees with length-based methods across all test cases.
- **Volume tracking and convergence**: Integer simplex volume monitoring detects convergence plateaus; face/edge analyses interpret sensitivity along edges and enable subspace searches across faces.
- **Test-driven development**: All source code follows TDD principles with 100% coverage requirements enforced via `.coveragerc` configuration.
- **Figure and data generation**: All figures referenced in documentation are generated by scripts in `quadmath/scripts/` that import from `src/` modules, ensuring code-documentation coherence.
- **Cross-platform compatibility**: Headless matplotlib backend (`MPLBACKEND=Agg`) ensures CI compatibility; deterministic RNG seeds guarantee reproducible outputs.
- **Dependency management**: All dependencies managed through `uv` and `pyproject.toml` with exact version pinning via `uv.lock`.
- **Path resolution**: No hardcoded paths; all output directories resolved via `paths.py` utilities for cross-platform compatibility.
- **Data export formats**: Figures saved alongside CSV/NPZ data for complete reproducibility; all generation scripts print output paths for manifest collection.

All source code, tests, and documentation are available in the docxology/QuadMath repository, ensuring complete transparency and reproducibility of the methods described herein.

4 Optimization in 4D

4.1 Overview

This section describes optimization methods adapted to the integer Quadray lattice: a discrete Nelder-Mead simplex method with lattice projection, a greedy 12-neighbor descent, and Fisher-metric (natural-gradient) guidance for continuous parameter spaces. The integer lattice supplies natural quantization of both steps and simplex volumes, which the convergence criteria below exploit; higher-dimensional extensions are developed in Section 5 (`05_extensions.md`).

4.2 Nelder-Mead on Integer Lattice

- **Adaptation:** the four standard simplex operations — reflection (α), expansion (γ), contraction (ρ), and shrink (σ) — applied to a simplex of four lattice points, with the objective f evaluated on the embedded coordinates (x, y, z) of each vertex.
- **Projection:** candidate moves are formed with per-component integer truncation of the scaled offsets (the centroid of the best three uses floor division), then mapped back to the canonical lattice representative by projective normalization, so the entire trajectory stays on the lattice.
- **Volume tracking:** monitor the exact IVM tetravolume (absolute quadray determinant divided by 4, kept as an exact Fraction; Section 3) as a convergence diagnostic; discrete steps create stable volume plateaus.
- **Degenerate-simplex restart:** a zero-volume (collinear/coplanar) simplex cannot leave its own affine subspace, since every NM move is an affine combination of the vertices. When the simplex collapses (zero volume with spread below tolerance), the algorithm probes ± 1 and ± 2 lattice steps along each quadray axis from the best vertex; if any probe improves the best value, it re-seeds a full-volume simplex along the three most promising axes (a CVP-style restart, consuming one iteration), otherwise the collapse is accepted as convergence and the run terminates.

4.2.1 Parameters

- **Reflection** $\alpha \approx 1$
- **Expansion** $\gamma \approx 2$
- **Contraction** $\rho \approx 0.5$
- **Shrink** $\sigma \approx 0.5$

References: original Nelder-Mead method and common parameterizations in optimization texts and survey articles; see overview: [Nelder-Mead method](#).

4.3 Volume-Level Dynamics

- Simplex tetravolume decreases in discrete steps ($|\det|/4$ values such as 0.5, 0.25, 0 in IVM units), producing plateaus (“energy levels”) between accepted moves.
- Termination: a collapsed simplex (zero volume) with objective spread below tolerance τ is checked by axis probes (± 1 , ± 2 lattice steps along each quadray axis from the best vertex); if no probe improves the best value, the run terminates, and if one does, a CVP-style restart re-seeds a full-volume simplex along the most promising axes. A degenerate simplex (zero volume) with spread still above τ simply continues — its affine moves remain confined to the collapse subspace while the spread shrinks toward collapse.
- Monitoring: record best/worst objective, spread, and exact IVM volume at each iteration (see `simplex_trace.png` below).

4.4 Quadray Lattice Optimization Pseudocode

```
1 while not converged:
2     order vertices by objective
3     centroid of best three
4     propose reflected (then possibly expanded/contracted) point
5     accept per standard tests; else shrink toward best
6     if simplex volume is zero and spread is below tolerance:
```

```

7 probe +/-1 and +/-2 lattice steps along each quadray axis from best
8 if any probe improves best: restart (full-volume simplex along 3 best axes); else stop
9 update integer volume and function spread trackers

```

4.4.1 Figures

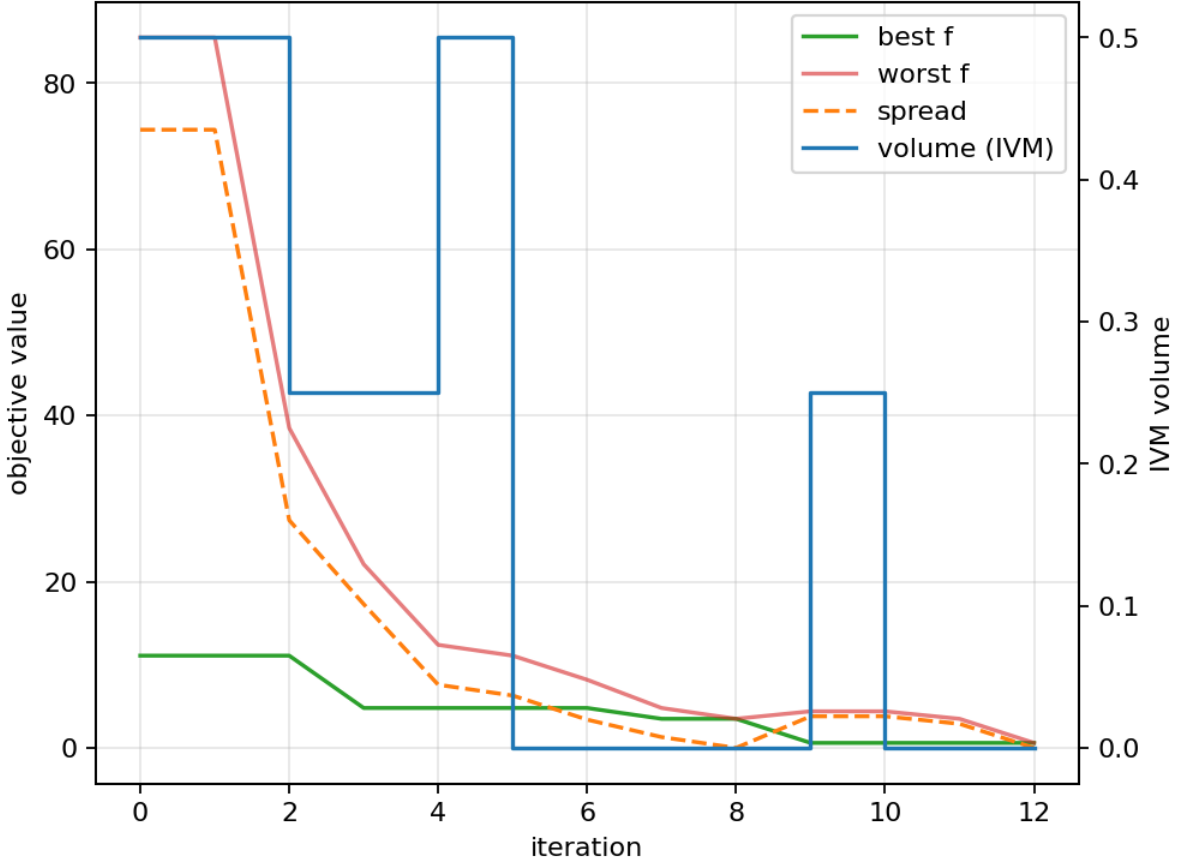


Figure 8: **Per-iteration diagnostics for discrete Nelder-Mead on the integer Quadray lattice.** Time series of the 13 recorded states (iterations 0-12, i.e. 12 optimization iterations after the initial simplex), plotted by `plot_simplex_trace` (`src/quadmath/viz/visualize.py`) from the run in `quadmath/scripts/simplex_animation.py` for the penalized objective $f(q) = (x-2)^2 + (y-2)^2 + (z-2)^2 + 0.1|x+y+z|$, plus +5 in the quadrant $x < 0 \wedge y < 0$ and +10 when any of $|x|, |y|, |z|$ exceeds 4, evaluated at the embedded Cartesian coordinates (x, y, z) from `to_xyz` with `DEFAULT_EMBEDDING`. **Left axis** (dimensionless objective units): best value $\min_i f(v_i)$ (green), worst value $\max_i f(v_i)$ (faint red), and spread $\max_i f(v_i) - \min_i f(v_i)$ (orange, dashed); only the best value is monotone — the restart at iteration 9 raises the worst value from 3.5 to 4.4. **Right axis** (blue step line): exact tetravolume in IVM units ($|\det|/4$, i.e. 0.5, 0.25, 0). The best value descends in plateaus (11.1 on 0-2, 4.8 on 3-6, 3.5 on 7-8, 0.6 on 9-12); the spread reaches zero at iterations 8 and 12, and the iteration-8 collapse triggers the CVP-style restart visible at iteration 9 (spread 0 $\rightarrow$ 3.8, volume 0 $\rightarrow$ 0.25). Regenerate with `uv run python quadmath/scripts/simplex_animation.py`; raw data in `quadmath/output/data/simplex_trace.csv/.npz`.

The 2×2 panel below shows the simplex itself at iterations 0, 3, 6, and 9; the plateau structure of the trace above appears as iterations where the simplex repositions without improving its best value.

Raw artifacts: the full trajectory animation `simplex_animation.mp4` and per-frame vertices (`simplex_animation_vertices.csv/.npz`) are available in `quadmath/output/`.

Nelder-Mead Simplex Evolution on Integer Quadray Lattice

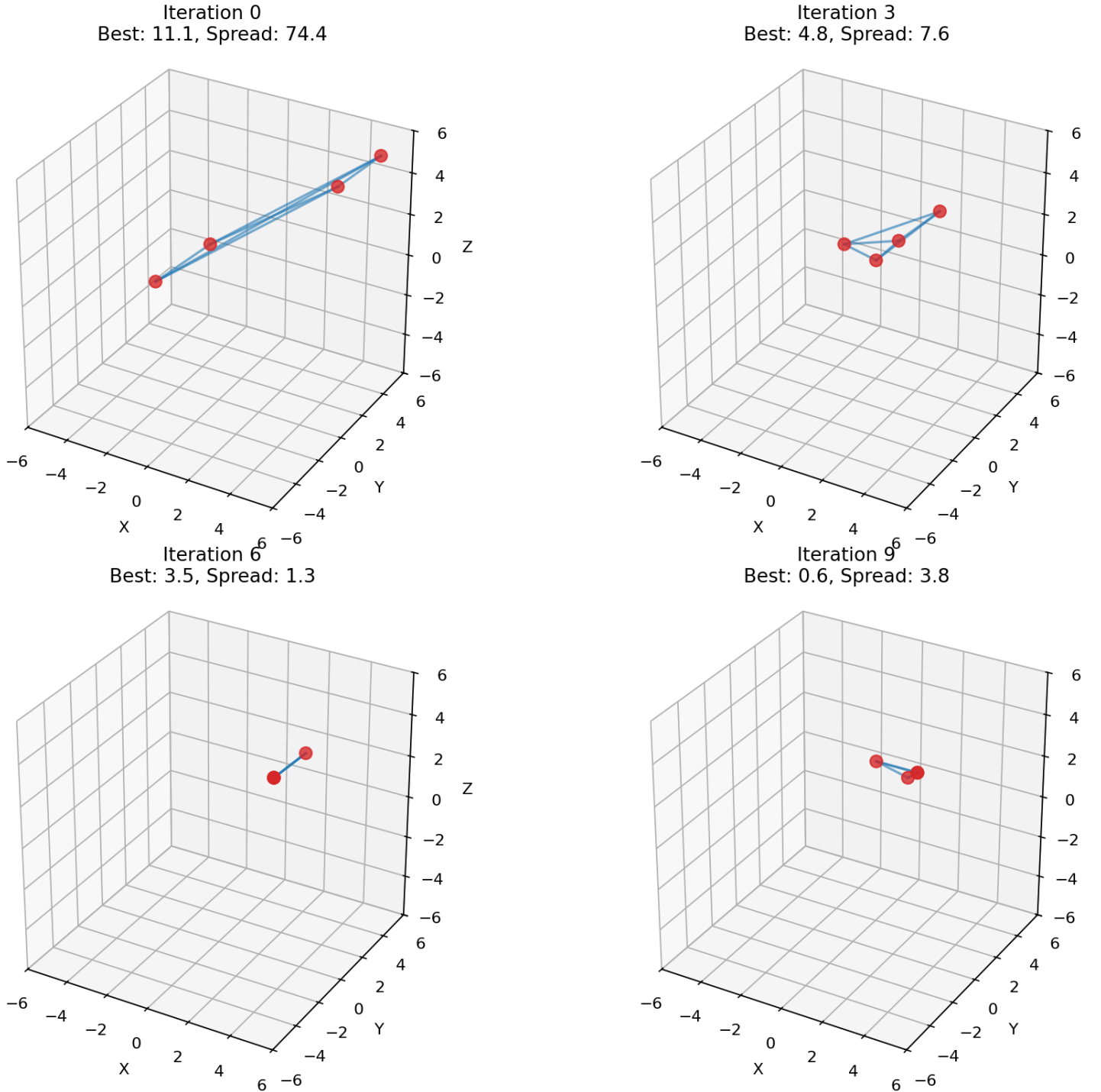


Figure 9: **Nelder-Mead simplex evolution on the integer Quadray lattice (2×2 panel).** The four-vertex simplex (vertices as red spheres, the six tetrahedral edges as blue lines) at iterations 0, 3, 6, and 9 of the same run as the trace figure, plotted in the embedded (x, y, z) space produced by `to_xyz` with `DEFAULT_EMBEDDING`; each axis spans $[-6, 6]$, and each panel title reports that iteration's best objective value and spread. **Top-left (iteration 0):** the widely dispersed initial simplex (Quadray(5,0,0,0), Quadray(4,1,0,0), Quadray(0,4,1,0), Quadray(1,1,1,0)). **Top-right (iteration 3):** contraction toward the main basin. **Bottom-left (iteration 6):** near-collapsed simplex (spread 1.3) just before the degenerate restart. **Bottom-right (iteration 9):** the re-seeded simplex contracting again (spread 3.8); the run terminates at iteration 11 with all four vertices coinciding at Quadray(2,0,0,0), i.e. embedded (2, 2, 2). Generated by `quadmath/scripts/simplex_animation.py`.

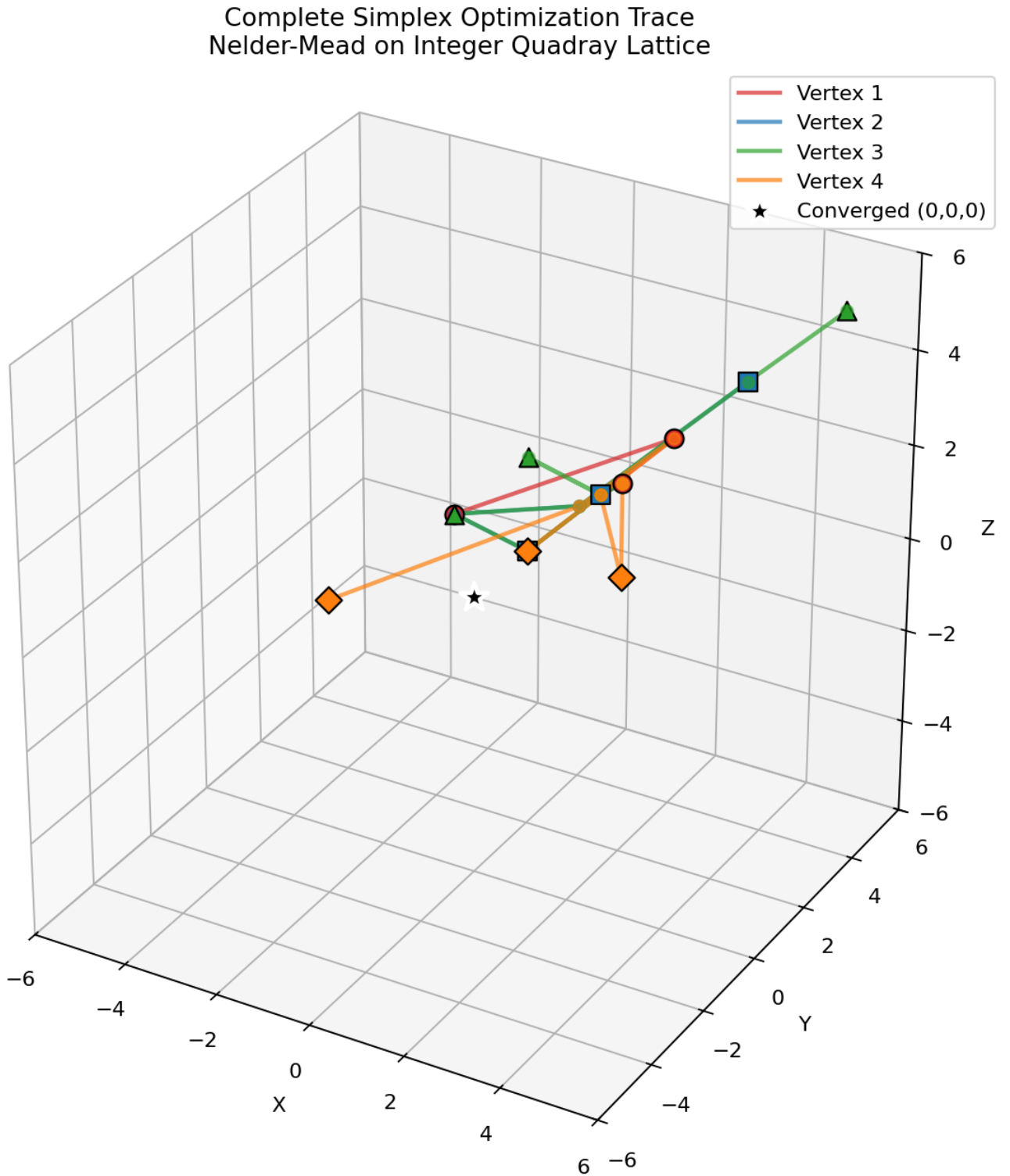


Figure 10: **Complete simplex vertex trajectories (3D)**. Full paths of the four simplex vertices across all 12 recorded iterations of the same run, in the same embedded (x, y, z) axes ($[-6, 6]$ per direction) as the panel above. Each vertex trace uses a fixed color and marker (red circle, blue square, green triangle, orange diamond); large black-edged markers flag iterations 0, 4, and 8. The traces show coordinated contraction toward the converged point $(2, 2, 2)$, with the degenerate-simplex restart visible as the outward jump near iteration 8. The script also draws a black star labeled “Converged $(0,0,0)$ ” at the embedded origin; the vertices themselves converge to $(2, 2, 2)$, so read the star as a decorative marker, not the optimum. Generated by `quadmath/scripts/simplex_animation.py`.

4.5 Discrete Lattice Descent (Information-Theoretic Variant)

- Integer-valued greedy descent over the IVM: from the current lattice point q , evaluate f at the 12 nearest neighbors (the distinct permutations of $(2, 1, 1, 0)$ in quadray offsets), then move to the minimizing neighbor.
- Each accepted move strictly decreases f , so the objective is monotone along the path; the walk terminates at a local minimum of f over the IVM adjacency, i.e. when no neighbor improves on the current value.
- The objective may be geometric (e.g. Euclidean distance in an embedding) or information-theoretic (e.g. a local free-energy proxy).
- API: `discrete_ivm_descent` in `src/quadmath/optimize/discrete_variational.py`. Animation helper: `animate_discrete_path` in `src/quadmath/viz/visualize.py`.

Short snippet (paper reproducibility):

```
1 from quadmath.core.quadray import Quadray, DEFAULT_EMBEDDING, to_xyz
2 from quadmath.optimize.discrete_variational import discrete_ivm_descent
3 from quadmath.viz.visualize import animate_discrete_path
4
5 def f(q: Quadray) -> float:
6     x, y, z = to_xyz(q, DEFAULT_EMBEDDING)
7     return (x - 0.5)**2 + (y + 0.2)**2 + (z - 0.1)**2
8
9 path = discrete_ivm_descent(f, Quadray(6,0,0,0))
10 animate_discrete_path(path)
```

4.6 Convergence and Robustness

- Discrete steps reduce numerical drift; improved stability vs. unconstrained Cartesian.
- Natural regularization from volume quantization; fewer wasted evaluations.
- Compatible with Gauss-Newton/Natural Gradient guidance using FIM for metric-aware steps (Amari, natural gradient).

4.7 Information-Geometric View (Einstein.4D analogy in metric form)

The Fisher Information Matrix (FIM) provides a fundamental bridge between the three 4D frameworks, establishing a Riemannian metric on parameter space that guides optimization through information geometry. This section demonstrates how the FIM connects Coxeter.4D (Euclidean parameter space), Einstein.4D (information-geometric flows), and Fuller.4D (tetrahedral structure) in a unified optimization framework.

4.7.1 Fisher Information as Riemannian Metric

The empirical Fisher Information Matrix F_{ij} quantifies the local curvature of the log-likelihood surface around parameter estimates, providing a natural metric for parameter space geometry. This fundamental concept in information geometry establishes a Riemannian structure on the statistical manifold, where distances and angles are measured according to the intrinsic geometry of the probability distributions rather than the extrinsic Euclidean geometry of the parameter space.

For a model with parameter vector θ and per-sample scores $\partial_{\theta_i} \log p(x_n; \theta)$, the empirical FIM is the average outer product of score functions, $F_{i,j} = \frac{1}{N} \sum_{n=1}^N \partial_{\theta_i} \log p(x_n; \theta) \partial_{\theta_j} \log p(x_n; \theta)$ (Eq. (9) in the equations appendix). Diagonal entries quantify parameter sensitivity and off-diagonal entries pairwise parameter interactions. The running example below keeps the parameter names $\mathbf{w} = (w_0, w_1, w_2)$ used by `information_demo.py`; the two notations coincide.

The Fisher Information Matrix serves as the natural metric tensor $g_{ij} = F_{ij}$ on the statistical manifold, replacing the Euclidean metric δ_{ij} with a data-dependent metric that reflects the actual curvature structure of the objective function. This geometric interpretation enables the application of differential geometry concepts to

optimization problems, where geodesics (locally distance-minimizing paths) follow the natural gradient direction $F^{-1}\nabla L$ rather than the standard gradient ∇L .

The theoretical foundation of this approach stems from the work of Rao (1945) and Amari (1985), who established information geometry as a framework for analyzing statistical models through differential geometry. The FIM naturally arises as the Hessian of the Kullback-Leibler divergence between nearby probability distributions, making it the canonical choice for measuring distances on the statistical manifold.

In the context of optimization, the FIM provides several key advantages:

1. **Invariance to parameterization:** The natural gradient $F^{-1}\nabla L$ is invariant to smooth, invertible parameter transformations, unlike the standard gradient which depends on the choice of coordinate system.
2. **Optimal step sizing:** The FIM automatically determines appropriate step sizes in different parameter directions, scaling updates according to local curvature.
3. **Geometric consistency:** Optimization follows geodesics on the statistical manifold, respecting the intrinsic geometry of the parameter space rather than imposing an artificial Euclidean structure.

This geometric approach to optimization is particularly powerful in the context of the 4D frameworks, where it provides a unified mathematical language for describing optimization dynamics across different geometric paradigms.

4.7.2 4D Framework Integration through Fisher Information

Coxeter.4D (Euclidean): In standard Euclidean parameter space, the metric tensor is simply δ_{ij} , providing uniform scaling in all directions. The FIM F_{ij} generalizes this to capture the actual curvature structure of the objective function.

Einstein.4D (Minkowski analogy): the Fisher metric $g_{ij} = F_{ij}$ replaces the flat metric, and geodesic motion on the statistical manifold corresponds to the natural gradient update $\theta \leftarrow \theta - \eta F(\theta)^{-1}\nabla_{\theta}L(\theta)$ (Eq. (10) in the equations appendix) — steepest descent measured in the Fisher metric rather than straight-line motion in parameter space.

Fuller.4D (Synergetics): The tetrahedral structure of Quadray coordinates naturally encodes the four-fold partition of optimization problems, while the FIM provides the metric structure for efficient navigation through this space. The discrete nature of the IVM lattice creates natural quantization effects that can be exploited for computational efficiency.

4.7.3 Comprehensive Fisher Information Analysis

Two figures summarize the empirical Fisher analysis of the regression example introduced below: the matrix structure and its eigenspectrum. Both are generated by `quadmath/scripts/information_demo.py` and interpreted through the three 4D frameworks.

Note on the estimator: the matrix in the figure above is computed from per-sample **squared-loss** gradients (`2*x_i*r_i`) at a misspecified `w_est` (`information_demo.py`), i.e. a Gauss-Newton/Gram surrogate. It coincides with the empirical FIM of Eq. (9) only for true score functions — exactly at w_{true} in expectation under Gaussian noise — so treat the displayed matrix as a curvature-scale illustration rather than a model FIM estimate.

4.7.4 Natural Gradient Descent: Geodesic Motion on Information Manifold

The Fisher Information Matrix enables natural gradient descent, which implements geodesic motion on the information manifold. Unlike standard gradient descent that follows straight lines in parameter space, natural gradient descent follows curved paths that respect the intrinsic geometry defined by the FIM.

The natural gradient update used throughout this manuscript is Eq. (10) in the equations appendix, $\theta \leftarrow \theta - \eta F(\theta)^{-1}\nabla_{\theta}L(\theta)$, where η is the step size, F the empirical FIM of Eq. (9), and $\nabla_{\theta}L$ the gradient of the loss. Because F is the metric tensor $g_{ij} = F_{ij}$ of the statistical manifold, this update is steepest descent measured in the Fisher metric — geodesic motion rather than straight-line motion in parameter space.

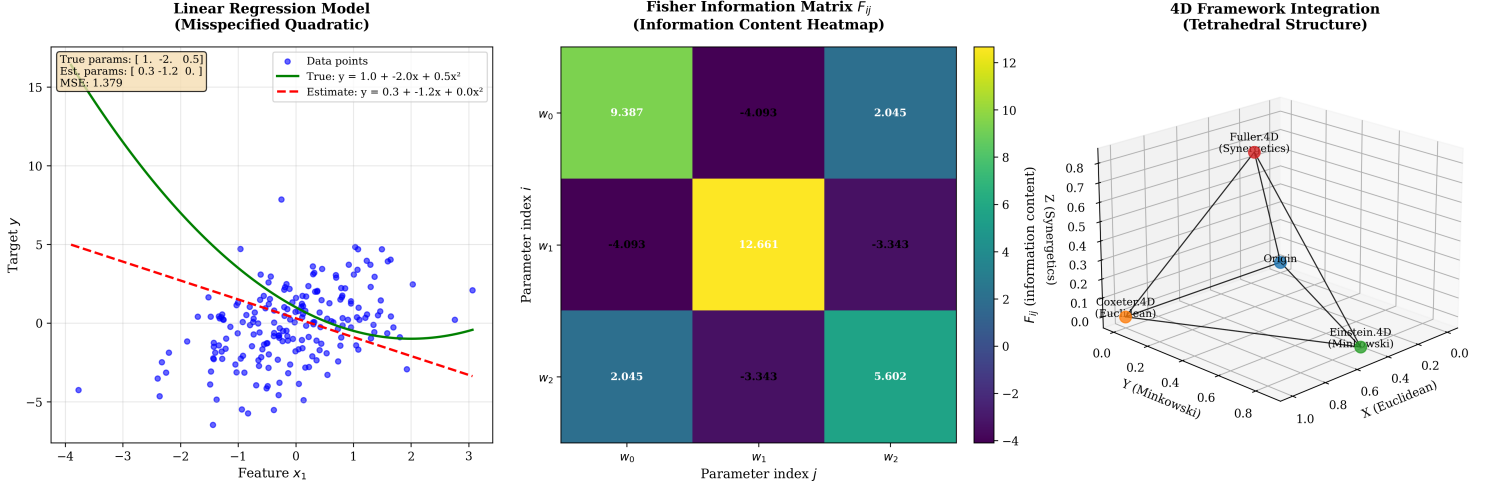


Figure 11: **Empirical Fisher Information Matrix with 4D framework context** (three panels; generated by quadmath/scripts/information_demo.py with seed default_rng(0)). The model is a 3-parameter linear regression $y = \mathbf{x}^\top \mathbf{w} + \varepsilon$ on $N = 200$ samples with standard-Gaussian features and noise $\varepsilon \sim \mathcal{N}(0, 0.1^2)$, generated at $\mathbf{w}_{\text{true}} = (1.0, -2.0, 0.5)$ and evaluated at the deliberately misspecified estimate $\mathbf{w}_{\text{est}} = (0.3, -1.2, 0.0)$. **Left panel:** the data points with the 1-D slice $y(x) = w_0 + w_1x + w_2x^2$ of both parameter vectors (green solid: true; red dashed: estimate) and the MSE annotation. **Center panel:** the 3×3 matrix of per-sample squared-loss gradients $2x_{ni}r_n$ (a Gauss-Newton/Gram surrogate of the FIM; see the estimator note below) as an annotated heatmap, with diagonal entries $\approx (9.39, 12.66, 5.60)$ and off-diagonal entries $\approx (-4.09, 2.05, -3.34)$; units are squared-loss gradient products. **Right panel:** schematic tetrahedron relating Coxeter.4D (Euclidean parameter space, metric δ_{ij}), Einstein.4D (Fisher metric replacing the flat metric), and Fuller.4D (tetrahedral/IVM structure).

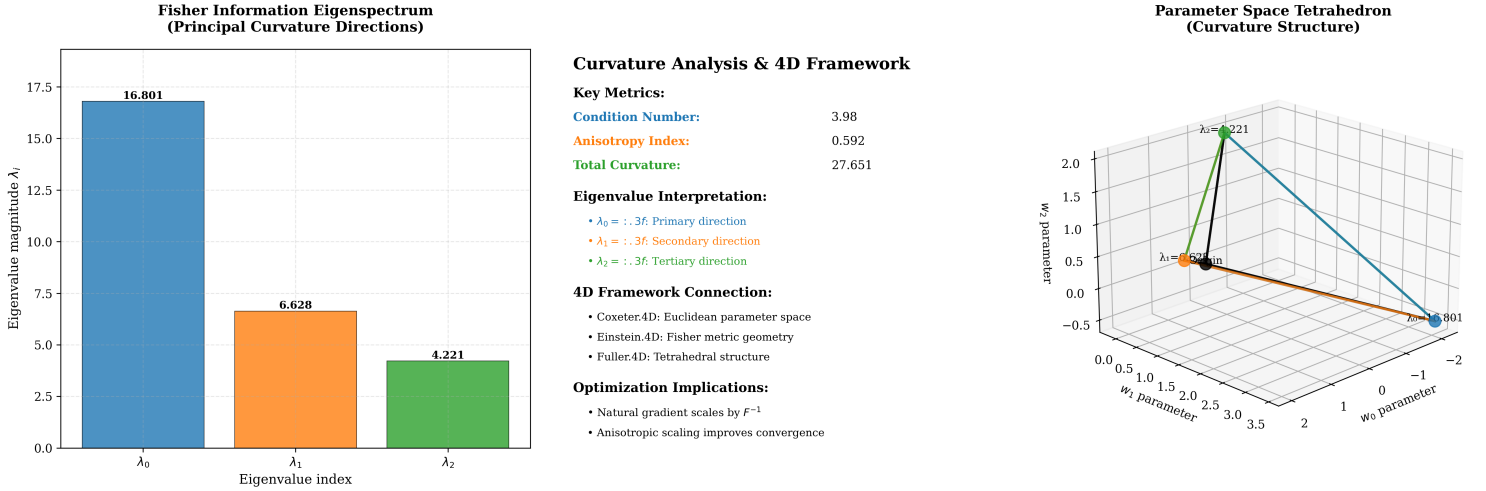


Figure 12: **Fisher Information eigenspectrum and parameter-space curvature** (three panels; generated by quadmath/scripts/information_demo.py from the same seeded run as the matrix figure above). **Left panel:** bar chart of the eigenvalues of the empirical FIM, sorted descending and annotated: $\lambda_0 \approx 16.80$, $\lambda_1 \approx 6.63$, $\lambda_2 \approx 4.22$ (units: squared-loss gradient products) — the principal curvature scales of the loss surface. **Center panel:** text summary of the curvature metrics: condition number $\lambda_{\text{max}}/\lambda_{\text{min}} \approx 3.98$ (anisotropy), anisotropy index ≈ 0.59 , and total curvature (trace of F) ≈ 27.65 , with per-direction and 4D-framework interpretation. **Right panel:** a parameter-space tetrahedron with one vertex at the origin and three vertices along the eigenvector directions scaled by $\sqrt{\lambda_i}$, so the tetrahedron's shape encodes the anisotropy; edge colors mark eigenvalue rank, and vertices are labeled with their eigenvalues. Large eigenvalues mark directions of rapid objective change (where the natural gradient takes small steps); small eigenvalues mark flat directions (larger steps).

The theoretical foundation of natural gradient descent was established by [Amari \(1998\)](#) in the context of information geometry. The key insight is that the natural gradient $F^{-1}\nabla L$ is the steepest descent direction when distances are measured using the Fisher metric rather than the Euclidean metric. This makes natural gradient descent invariant to smooth, invertible parameter transformations, a property that standard gradient descent lacks.

In the context of the 4D frameworks, natural gradient descent provides a unified approach to optimization that respects the intrinsic geometry of each framework:

- **Coxeter.4D:** The natural gradient respects the actual curvature structure of the objective function rather than imposing artificial Euclidean geometry.
- **Einstein.4D:** The Fisher metric replaces the spacetime metric, creating geodesic flows that follow the intrinsic geometry of the parameter space.
- **Fuller.4D:** The tetrahedral structure provides natural coordinate systems where the FIM can exhibit beneficial structural properties.

The efficiency of natural gradient descent comes from its ability to automatically adapt step sizes to local curvature. In directions of high curvature (large eigenvalues of F), the natural gradient takes smaller steps, while in directions of low curvature (small eigenvalues), it takes larger steps. This anisotropic scaling leads to faster convergence and better numerical stability compared to standard gradient descent.

4.7.5 Quadray-Specific Considerations

Under Quadray parameterizations, the FIM often exhibits block-structured and symmetric patterns that simplify matrix inversion for natural-gradient steps. This structural regularity arises from the tetrahedral symmetry of the IVM lattice and can be exploited for computational efficiency.

The discrete nature of the IVM lattice also influences the FIM structure, as parameter updates are constrained to integer coordinate positions. This creates a natural regularization effect that can improve optimization stability and convergence.

4.7.6 Variational Free Energy and Active Inference Integration

The Fisher Information framework naturally extends to variational inference and active inference, where the free energy principle guides both perception and action through information-geometric optimization.

For the full Active Inference treatment — expected free energy, perception and action updates, and further 4D natural-gradient visualizations — see [Section 9: Free Energy and Active Inference](#).

4.8 Multi-Objective and Higher-Dimensional Notes

The extension of these ideas — simplex faces as Pareto trade-off surfaces, integer volume as a solution-diversity measure, and higher-simplex volume decompositions — is developed in [Section 5 \(05_extensions.md\)](#).

4.9 External Validation and Computational Context

The methods above complement the computational framework in Kirby Uerner’s [4dsolutions ecosystem](#); implementation and educational context are collected in the [Resources](#) section.

4.10 Results

On the penalized quadratic of `quadmath/scripts/simplex_animation.py`, the discrete Nelder-Mead reaches the lattice point `Quadray(2,0,0,0)` (embedded $(2,2,2)$, best objective 0.6) in 12 recorded iterations, with the best value descending through the plateaus $11.1 \rightarrow 4.8 \rightarrow 3.5 \rightarrow 0.6$ and one degenerate-simplex restart mid-run; see the simplex figures above and the artifacts in `quadmath/output/`.

Natural Gradient Trajectory (Geodesic Motion on Information Manifold)

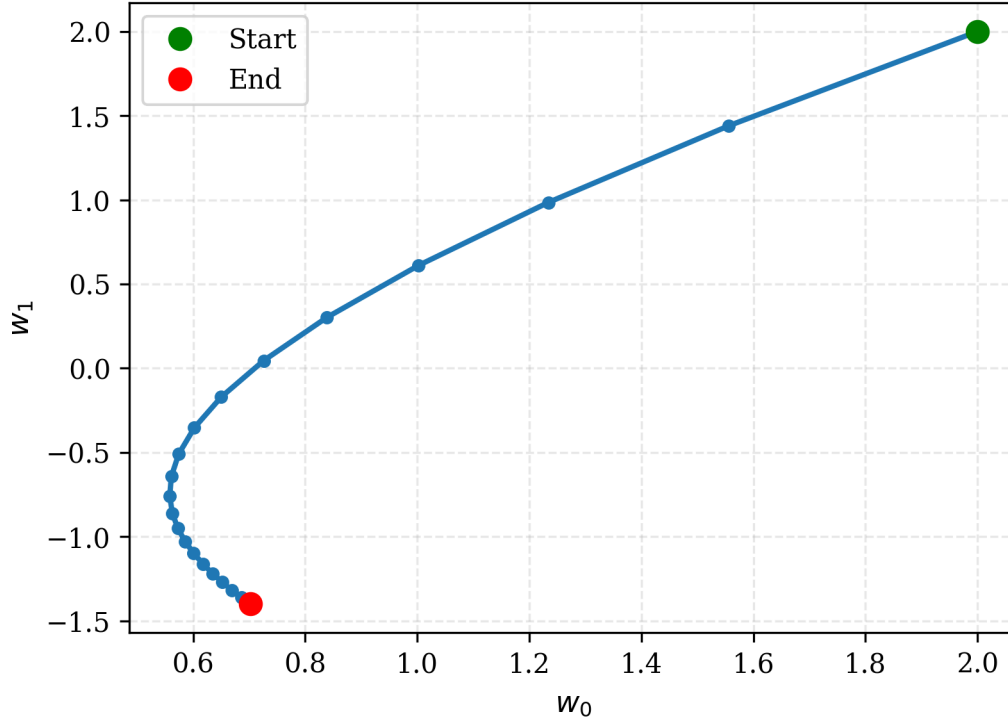


Figure 13: **Natural gradient trajectory on a quadratic bowl** (generated by `quadmath/scripts/information_demo.py`). The objective is the quadratic $L(\mathbf{w}) = \frac{1}{2}(\mathbf{w} - \mathbf{w}_{\text{true}})^\top A(\mathbf{w} - \mathbf{w}_{\text{true}})$ with $\mathbf{w}_{\text{true}} = (1, -2, 0.5)$ and A the positive-definite matrix with diagonal $(3, 2, 1)$ and off-diagonal $A_{12} = 0.5$; the metric used for the steps is the empirical FIM F of the regression example above (plus a 10^{-3} ridge for invertibility), not A . Starting from $(2, 2, 2)$, the blue line with markers shows 20 updates of the form $\Delta \mathbf{w} = -0.5 F^{-1} A(\mathbf{w} - \mathbf{w}_{\text{true}})$ — the 3-parameter trajectory (w_0, w_1, w_2) projected onto the (w_0, w_1) plane (parameter units). Green circle: start; red circle: the final iterate $(0.70, -1.40)$ after 20 steps — a fixed-metric preconditioned run stopped at its step budget, not yet at $\mathbf{w}_{\text{true}} = (1, -2, 0.5)$. The anisotropic F^{-1} preconditioning is visible as unequal progress along the two plotted coordinates.

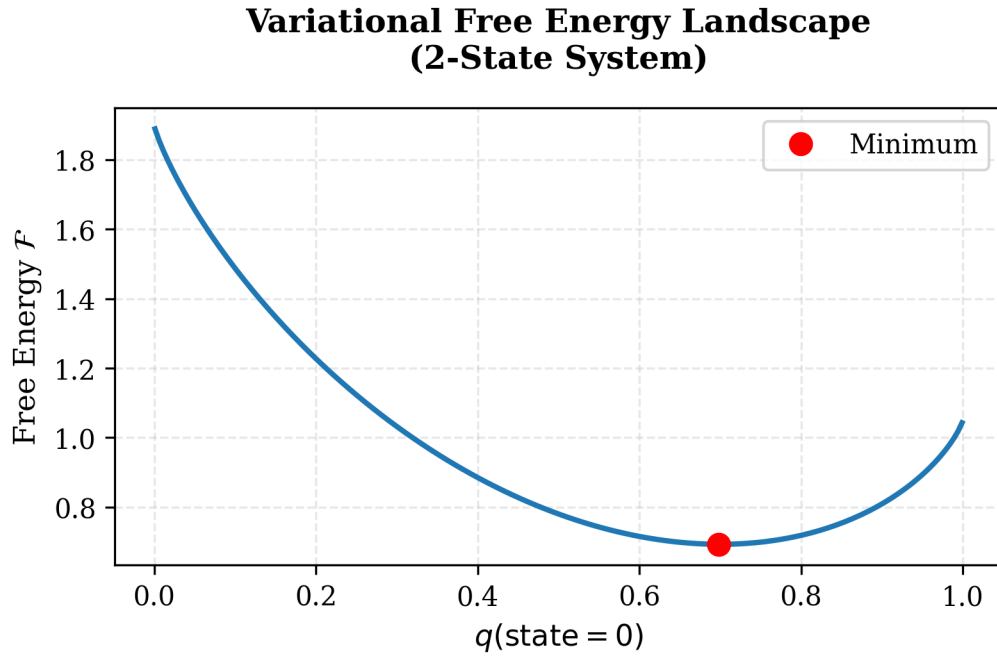


Figure 14: **Variational free energy for a 2-state toy model** (generated by `quadmath/scripts/information_demo.py`). The curve shows the free energy of Eq. (11), $\mathcal{F} = -\log P(o | s) + \text{KL}[Q(s) \| P(s)]$, as a function of the variational parameter $q = Q(\text{state} = 0)$ over the grid $q \in (0, 1)$ (200 points), for likelihood $P(o | s) = (0.7, 0.3)$, uniform prior $P(s) = (0.5, 0.5)$, and variational family $Q(s) = (q, 1 - q)$. **X-axis:** q (dimensionless probability). **Y-axis:** $\mathcal{F}$ in nats. The minimum (red marker) is $\mathcal{F} \approx \ln 2 \approx 0.693$ at $q = 0.7$, where Q coincides with the likelihood — the standard variational result that the optimal Q in the unconstrained family is the posterior itself. In the 4D reading, minimizing $\mathcal{F}$ is geodesic motion under the Fisher metric on the variational manifold (Einstein.4D analogy).

5 Extensions of 4D and Quadrays

Here we review some extensions of the Quadray 4D framework, including multi-objective optimization, machine learning, computer graphics and GPU acceleration, active inference, complex systems, pedagogy, and implementations, with an emphasis on cognitive security.

5.1 Multi-Objective Optimization

- A candidate $\mathbf{q}$ Pareto-dominates $\mathbf{q}'$ when $f_k(\mathbf{q}) \leq f_k(\mathbf{q}')$ for every objective k , with strict inequality for at least one; the Pareto front is the set of undominated candidates. Simplex faces of a candidate vertex set encode these trade-offs, each face selecting one non-dominated combination.
- Pareto front exploration via tetrahedral traversal: walking edge-adjacent simplices over the candidate set enumerates neighboring non-dominated combinations without a continuous relaxation.
- Integer tetravolume of the solution simplex serves as a discrete diversity measure: larger volume corresponds to more mutually separated trade-off points.

5.2 Machine Learning and Robustness

- **Geometric regularization:** Quadray-constrained weights/topologies yield structural priors and improved stability.
- **Adversarial robustness:** Discrete lattice projection reduces vulnerability to gradient-based adversarial perturbations by limiting directions.
- **Ensembles:** Tetrahedral vertex voting and consensus improve robustness.

References: see [Fisher information](#), [Natural gradient](#), and quadray conversion notes by Urner for embedding choices.

5.3 Computer Graphics and GPU Acceleration

- **Quadray visualization acceleration:** GPU-accelerated rendering of tetrahedral coordinate systems enables real-time exploration of 4D geometric structures. The parallel nature of GPU architectures naturally maps to the four-basis vector representation of quadrays, allowing simultaneous computation of vertex positions, edge connections, and face tessellations across thousands of tetrahedra.
- **Integer arithmetic optimization:** GPU compute shaders excel at integer-based volume calculations and determinant computations using the Bareiss algorithm. The discrete lattice structure of quadray coordinates benefits from parallel integer arithmetic units, achieving significant speedups over CPU implementations for large-scale geometric computations.
- **Dynamic programming acceleration:** GPU-accelerated dynamic programming algorithms leverage CUDA Dynamic Parallelism for adaptive parallel computation of recursive geometric algorithms. This approach enables efficient handling of varying computational workloads in tetrahedral decomposition and optimization problems, as demonstrated in applications like the Mandelbrot set computation where dynamic parallelism manages computational complexity effectively.
- **Parallel geometric algorithms:** Implementation of GPU-optimized versions of algorithms like QuickHull for convex hull computation in quadray space achieves substantial performance improvements. The tetrahedral lattice structure naturally supports parallel prefix sum operations and efficient neighbor queries, enabling real-time visualization of complex 4D geometric transformations.
- **Memory bandwidth optimization:** The structured memory access patterns of quadray coordinates align well with GPU memory hierarchies, enabling efficient coalesced memory access for large-scale geometric datasets. This optimization is particularly beneficial for applications requiring real-time rendering of complex polyhedral structures and dynamic tessellations.

References: GPU-accelerated geometry processing techniques ([arxiv.org](#)), CUDA Dynamic Parallelism for adaptive computation ([developer.nvidia.com](#)), and parallel scan algorithms for optimization ([developer.nvidia.com](#)).

5.4 Active Inference and Free Energy

- Free energy $\mathcal{F} = -\log P(o | s) + \text{KL}[Q(s) \| P(s)]$ (see Eq. (11) in the equations appendix); background: [Free energy principle](#) and overviews connecting to predictive coding and control.
- Belief updates follow steepest descent in Fisher geometry using the natural gradient (see Eq. (10) in the equations appendix); quadray constraints improve stability/interpretability.
- Links to metabolic efficiency and biologically plausible computation.
- For the full treatment, see 09_free_energy_active_inference.md (Appendix: The Free Energy Principle and Active Inference).

5.5 Complex Systems and Collective Intelligence

- Tetrahedral interaction patterns support distributed consensus and emergent behavior.
- Resource allocation and network flows benefit from geometric constraints.
- **Cognitive security**: Applying cognitive security can safeguard distributed consensus mechanisms from manipulation, preserving the reliability of emergent behaviors in complex systems. Incorporating cognitive security measures can protect the integrity of belief updates and decision-making processes, ensuring that actions are based on accurate and unmanipulated information.

5.6 Geospatial Intelligence and the World Game

- **Spatial data integration**: Quadray tetrahedral frameworks provide natural tessellations for geospatial data analysis, where the Dymaxion projection's minimal distortion aligns with Fuller's World Game objectives of holistic global perspective. The tetrahedral lattice supports efficient spatial indexing and neighbor queries for distributed geospatial intelligence operations.
- **Resource allocation optimization**: The World Game's goal of "making the world work for 100% of humanity" translates to multi-objective optimization problems where tetrahedral simplex faces encode trade-offs between population centers, resource distribution, and ecological constraints. Integer volume quantization ensures discrete, interpretable solutions for global resource allocation.
- **Cognitive security in distributed sensing**: Geospatial intelligence networks benefit from tetrahedral consensus mechanisms that resist manipulation of spatial data streams. The geometric constraints of Fuller.4D provide natural validation frameworks for detecting anomalous spatial patterns and maintaining data integrity across distributed sensor networks.
- **Tetrahedral tessellations for global modeling**: The World Game's emphasis on interconnected global systems maps naturally to tetrahedral decompositions of the Dymaxion projection, where each tetrahedron represents a coherent region for local optimization while maintaining global connectivity through shared faces and edges.

5.7 Quadrays, Synergetics (Fuller.4D), and William Blake

- Quadrays (tetrahedral coordinates) instantiate Fuller's Synergetics emphasis on the tetrahedron as a structural primitive; in this manuscript's terminology this corresponds to Fuller.4D. Tetrahedral frames support part-whole reasoning and efficient decompositions used throughout.
- William Blake's "fourfold vision" (single, twofold, threefold, fourfold) provides a historical metaphor for multiscale perception and inference. Read through Fisher geometry and natural gradient dynamics, it parallels multilayer predictive processing and counterfactual simulation. For background, see a concise overview of Blake's visionary psycho-topographies in British Art Studies ([visionary art analysis](#)) and the Active Inference Institute's MathArt Stream #8 ([Active Inference & Blake](#)).
- Juxtaposing Blake and Fuller foregrounds "comprehensivity": holistic design and sensemaking via geometric primitives. Context: ([Fuller & Blake: Lives in Juxtaposition](#)) and pedagogical antecedents in experimental design education at Black Mountain College ([Diaz, Chance and Design at Black Mountain College - PDF](#)).
- Implications for Quadray practice: four-facet summaries of models/trajectories, tetrahedral consensus in ensembles, and stigmergic annotation patterns for cognitive security and distributed sensemaking.

5.8 Pedagogy and Implementations

Kirby Urner’s comprehensive [4dsolutions ecosystem](#) provides extensive educational resources and cross-platform implementations for Quadray computation and visualization. For comprehensive details on educational frameworks, cross-language implementations, historical context, and community development, see the [Resources](#) section.

5.9 Higher Dimensions and Decompositions

- Decompose an n -simplex ($n > 3$) into tetrahedra and sum their signed integer volumes; the sum extends the quantized tetravolume of Section 3 to higher-dimensional content.
- Tessellations support parallel and distributed implementations (cells processed independently).

5.10 Limitations and Future Work

- Benchmark breadth: extend beyond convex/quadratic toys to real tasks (registration, robust regression, control) with ablations.
- Distance sensitivity: compare embeddings and their effect on optimizer trajectories; document recommended defaults.
- Hybrid schemes: study schedules that interleave continuous proposals with lattice projection.

6 Discussion

Quadray geometry (Fuller.4D) offers an interpretable, quantized view of geometry, topology, information, and optimization. The claim this discussion defends is that two mechanisms carry most of the weight. First, integer volumes enforce discrete dynamics, acting as a structural prior that regularizes optimization, prevents numerical fragility, and enables exact integer-based accelerated methods (Sections 3–5 present the evidence). Second, information geometry supplies the right optimization language for the synergetic tradition: updates proceed not through arbitrary parameter-space moves in continuous space, but along geodesics defined by information content (see Eq. (8) and Eq. (10) in the equations appendix; overview: [Natural gradient](#)). The subsections below state what follows from combining the two mechanisms, what remains unestablished, and where external validation stands.

6.1 Scope and Limitations

- **Embeddings and distances:** distance calculations are embedding-dependent — the quadray-to-Euclidean map must be selected and reported deliberately (Section 5 catalogs the comparison agenda); no single embedding is privileged.
- **Hybrid strategies:** some problems need both worlds — continuous steps interleaved with periodic lattice projection balance curvature-aware efficiency against discrete robustness.
- **Benchmarking:** the structural-prior benefits above remain hypotheses until benchmarked per domain; Section 5 lists the breadth and ablation gaps.

In practical analysis and simulation, numerical precision matters. Integer-volume reasoning is exact in theory, but empirical evaluation (e.g., determinants, Fisher Information, geodesics) can benefit from high-precision arithmetic when double precision is insufficient; the High-Precision Arithmetic Note in the equations appendix covers quad precision (`libquadmath`, `__float128`) and symbolic (SymPy) evaluation routes.

6.2 Fisher Information and Curvature

Claim: the Fisher Information Matrix (FIM) defines a Riemannian metric on parameter space whose eigenspectrum is a readable map of the loss surface — large eigenvalues of F mark sensitive (high-curvature) directions, small eigenvalues mark sloppy ones. Evidence: the eigenspectrum and curvature panels of Section 4 (the “Fisher Information eigenspectrum and parameter-space curvature” figure) display these scales for the empirical estimate of Eq. (9) in the equations appendix. Implication: curvature-aware steps using Eq. (10) in the equations appendix adaptively scale updates by the inverse metric, improving conditioning relative to vanilla gradient descent. Background: [Fisher information](#).

A curious connection unites geodesics in information geometry, the physical principle of least action, and Buckminster Fuller’s tensegrity geodesic domes (Fuller.4D). On statistical manifolds, geodesics are shortest paths under the Fisher metric, and natural-gradient flows trace paths that motivate gradient-weighted path-length heuristics (the `information_length` proxy in `metrics.py`) constrained by curvature (Eqs. (8), (10) in the equations appendix). In tensegrity domes, geodesic lines on triangulated spherical shells distribute stress nearly uniformly while the network balances continuous tension with discontinuous compression, attaining maximal stiffness with minimal material. Both systems exemplify constraint-balanced minimalism: an extremal path emerges by trading off cost (action or information length) against structure (metric curvature or tensegrity compatibility). The shared economy—optimal routing through low-cost directions—links geodesic shells in architecture to geodesic flows in parameter spaces; see background on tensegrity/geodesic domes online.

6.3 Quadray Coordinates and 4D Structure (Fuller.4D vs Coxeter.4D vs Einstein.4D)

Quadray coordinates provide a tetrahedral basis with projective normalization, aligning with close-packed sphere centers (IVM); overview: [Quadray coordinates](#) and synergetics background. The optimization-relevant consequence: symmetries common in quadray parameterizations often yield near block-diagonal structure in F , simplifying inversion and preconditioning (Section 4 develops this observation; the appendix’s namespaces summary fixes the definitions). We stress the boundaries because category errors here are the framework’s main failure mode: (i) Fuller.4D for lattice and integer volumes, (ii) Coxeter.4D for Euclidean embeddings,

lengths, and simplex families, (iii) Einstein.4D for metric analogies only — not for interpreting synergetic tetravolumes.

6.4 Integrating FIM with Quadray Models

Applying the FIM within quadray-parameterized models ties statistical curvature to tetrahedral structure. Practical takeaways:

- Use `fisher_information_matrix` to estimate $\mathbb{F}$ from per-sample gradients; inspect principal directions via `fim_eigenspectrum`.
- Exploit block patterns induced by quadray symmetries to stabilize metric inverses and reduce compute.
- Combine integer-lattice projection with natural-gradient steps to balance discrete robustness and curvature-aware efficiency.
- Purely discrete alternatives (e.g., `discrete_ivm_descent`) provide monotone integer-valued descent when gradients are unreliable; hybrid schemes can interleave discrete steps with curvature-aware continuous proposals.

6.5 Implications for Optimization and Estimation

6.5.1 Clarifications on “frequency/time” dimensions

- Fuller’s discussions often treat frequency/energy as an additional organizing dimension distinct from Euclidean coordinates. In our manuscript, we keep the shape/angle relations (Fuller.4D) separate from time/energy bookkeeping; when temporal evolution is needed, we use explicit trajectories and metric analogies (Einstein.4D) without conflating with Euclidean 4D objects (Coxeter.4D). This separation avoids category errors while preserving the intended interpretability.

6.5.2 On distance-based tetravolume formulas (clarification)

- The bridging-vs-native dichotomy of Section 3 reduces to a practical rule: when inputs are edge lengths, compute with PdF or Cayley–Menger in Euclidean length space and convert via the S3 factor (Eqs. (6), (5) in the equations appendix); when inputs are integer quadrays, use the native determinants of Tom Ace or Gerald de Jong (Eqs. (2), (7)), which return IVM tetravolumes directly with no XYZ intermediates. All routes agree numerically on shared cases, so the choice is driven by input type and desired exactness, not by the answer.

6.5.3 Symbolic analysis (bridging vs native) (Results linkage)

- Evidence: exact (SymPy) comparisons confirm that CM+S3 and Ace 5×5 produce identical IVM tetravolumes on canonical small integer-quadray examples — see the bridging vs native validation figure in Section 3 and the manifest `sympy_symbolics.txt` alongside `bridging_vs_native.csv` in `quadmath/output/`. Implication: the two computational cultures are interchangeable on the lattice, so exactness can be chosen per call site.
- Curvature-aware optimizers: Kronecker-factored approximations (K-FAC) leverage structure in $\mathbb{F}$ to accelerate training and improve stability; see [K-FAC \(arXiv:1503.05671\)](#). Similar ideas apply when quadray structure induces separable blocks.
- Model selection: eigenvalue spread of $\mathbb{F}$ provides a lens on parameter identifiability; near-zero modes suggest redundancies or over-parameterization.
- Robust computation: lattice normalization in quadray space yields discrete plateaus that complement FIM-based scaling for numerically stable trajectories.

6.6 Community Ecosystem and Validation

The computational ecosystem around Quadrays and synergetic geometry — chiefly the 4dsolutions organization (Section 7) — is treated here as an external validation surface: its cross-language implementations in-

independently verify algorithmic correctness against this manuscript's codebase, and its educational materials document applications across environments this paper does not attempt to survey. The claim worth stating explicitly: agreement between independent implementations is evidence for the mathematics, not merely for the code.

7 Resources

This section provides comprehensive resources for learning about and working with Quadrays, synergetics, and the computational methods discussed in this manuscript.

7.1 Core Concepts and Background

7.1.1 Information Geometry and Optimization

- **Fisher information:** [Fisher information \(reference\)](#) — see also Eq. (8) in the equations appendix
- **Natural gradient:** [Natural gradient \(reference\)](#) — see also Eq. (10) in the equations appendix

7.1.2 Active Inference and Free Energy

- **Active Inference Institute:** [Welcome to Active Inference Institute](#)
- **Comprehensive review:** [Active Inference — recent review \(UCL Discovery, 2023\)](#)

7.1.3 Mathematical Foundations

- **Tetrahedron volume formulas:** length-based [Cayley-Menger determinant](#) and determinant-based expressions on vertex coordinates (see [Tetrahedron - volume](#))
- **Exact determinants:** [Bareiss algorithm](#), used in our integer tetravolume implementations
- **Optimization baseline:** the [Nelder-Mead method](#), adapted here to the Quadray lattice

7.2 Quadrays and Synergetics (Core Starting Points)

7.2.1 Introductory Materials

- **Quadray coordinates (intro and conversions):** [Urner - Quadray intro](#), [Urner - Quadrays and XYZ](#)
- **Quadrays and the Philosophy of Mathematics:** [Urner - Quadrays and the Philosophy of Mathematics](#)
- **Synergetics background and IVM:** [Synergetics \(Fuller, overview\)](#)
- **Quadray coordinates overview:** [Quadray coordinates \(reference\)](#)

7.2.2 Historical and Background Materials

- **RW Gray projects — Synergetics text:** [rwgrayprojects.com \(synergetics\)](#)
- **Fuller FAQ:** [C. J. Fearnley's Fuller FAQ](#)
- **Synergetics resource list:** [C. J. Fearnley's resource page](#)
- **Wikieducator:** [Synergetics hub](#)
- **Quadray animation:** [Quadray.gif \(Wikimedia Commons\)](#)
- **Fuller Institute:** [BFI — Big Ideas: Synergetics](#)

7.3 4dsolutions Ecosystem: Comprehensive Computational Framework

The [4dsolutions organization](#) provides the most extensive computational framework for Quadrays and synergetic geometry, spanning 29+ repositories with implementations across multiple programming languages.

7.3.1 Core Computational Modules

Primary Python Libraries

- **Math for Wisdom (m4w):** [m4w \(repo\)](#)
 - **Quadray vectors and conversions:** `qrays.py` ([Qvector](#), [SymPy-aware](#))
 - **Synergetic tetravolumes and modules:** `tetravolume.py` [with PdF-CM vs native IVM and BEAST algorithms](#)

Cross-Language Validation

- **Rust implementation:** [rusty_rays](#) (performance-oriented)
 - Sources: [Rust library implementation](#), [Rust command-line interface](#)
- **Clojure implementation:** [synmods](#) (functional paradigm)
 - Sources: [qrays.clj](#), [ramping_up.clj](#)

7.3.2 Primary Hub: School_of_Tomorrow (Python + Notebooks)

Repository: [School_of_Tomorrow](#)

Core Modules

- [qrays.py](#): Quadray implementation with normalization, conversions, and vector ops ([source](#))
- [quadcraft.py](#): POV-Ray scenes for CCP/IVM arrangements, animations, and tutorials ([source](#))
- [flextegrity.py](#): Polyhedron framework, concentric hierarchy, POV-Ray export ([source](#))
- **Additional modules:** [polyhedra.py](#), [identities.py](#), [smold_play.py](#) (synergetic modules)

Key Notebooks

- [Qvolume.ipynb](#): Tom Ace 5×5 determinant with random-walk demonstrations ([source](#))
- [VolumeTalk.ipynb](#): Comparative analysis of bridging vs native tetravolume formulations ([source](#))
- [QuadCraft_Project.ipynb](#): 1,255 lines of interactive CCP navigation and visualization tutorials ([source](#))
- **Additional notebooks:** [TetraBook.ipynb](#), [CascadianSynergetics.ipynb](#), [Rendering_IVM.ipynb](#), [SphereVolumes.ipynb](#) (visual and curricular materials)

7.3.3 Additional Repositories

Tetravolumes (Algorithms and Pedagogy)

- **Repository:** [tetravolumes](#)
- **Code:** [tetravolume.py](#)
- **Notebooks:** [Atoms R Us.ipynb](#), [Computing Volumes.ipynb](#)

Visualization and Rendering

- **BookCovers:** VPython for interactive educational animations ([repo](#))
 - Examples: [bookdemo.py](#), [stickworks.py](#), [tetravolumes.py](#)

7.3.4 Educational Framework and Curricula

Oregon Curriculum Network (OCN)

- **OCN portal:** [OCN portal](#)
- **Python for Everyone:** [pymath page](#)

Historical Documentation

- **Python5 notebooks:** [Polyhedrons 101.ipynb](#)
- **Historical variants:** [qrays.py](#) also appears in [Python5 \(archive\)](#)
- **Python edu-sig archives:** [Python edu-sig archives](#) tracing 25+ years of development

7.3.5 Media and Publications

- **YouTube demonstrations:** [Synergetics talk 1](#), [Synergetics talk 2](#), [Additional](#)
- **Academia profile:** [Kirby Urner at Academia.edu](#)

7.4 Community Discussions and Collaborative Platforms

7.4.1 Active Platforms

- **Math4Wisdom Knowledge Engineering:** [Collaborative platform](#) with various art, resources, and cross-reference materials
- **synergeo discussion archive:** [Groups.io platform](#) with ongoing community discussions and technical exchanges

7.4.2 Historical Archives

- **GeodesicHelp threads:** [GeodesicHelp computations archive \(Google Groups\)](#) documenting computational approaches and problem-solving techniques

7.5 Related Projects and Applications

7.5.1 Tetrahedral Voxel Engines

- **QuadCraft:** [Tetrahedral voxel engine using Quadrays](#)

7.5.2 Academic Publications

- **Flextegrity:** [Generating the Flextegrity Lattice \(academia.edu\)](#)

7.5.3 Context and Integration

These materials popularize the IVM/CCP/FCC framing of space, integer tetravolumes, and projective Quadray normalization. They inform the methods in this paper and complement the `src/` implementations (see `quadray.py`, `cayley_menger.py`, `linalg_utils.py`).

The ecosystem provides extensive validation, pedagogical context, and practical implementations that complement and extend the methods developed in this manuscript. Cross-language implementations serve as independent verification of algorithmic correctness while educational materials demonstrate practical applications across diverse computational environments.

8 Equations and Math Supplement (Appendix)

Conventions for this appendix: vertices are $P_0, \dots, P_3$ with Quadray components (a_i, b_i, c_i, d_i) ; volumes are V_{xyz} (Euclidean, cubic length units) and V_{ivm} (synergetics/IVM tetravolume units, unit regular tetrahedron $V_{ivm} = 1$), related by $V_{ivm} = S3 V_{xyz}$ with $S3 = \sqrt{9/8}$ (Section 3); M denotes the Quadray-to-XYZ embedding matrix (Sections 3 and 14), $\|\cdot\|_2$ the Euclidean norm, and $\text{KL}[Q \| P]$ the Kullback-Leibler divergence of Q relative to P (Section 10). Three symbol overloads are scoped by context and flagged where they occur: c is a Quadray component (and a PdF edge length) in the volume formulas but the speed of light in the Minkowski line element; d is the fourth Quadray component in the coordinate formulas but an edge length in the length-based formulas and the distance function $d(\cdot, \cdot)$ of the embedding section; and P is a vertex label P_i in the volume sections but a probability distribution in the free-energy sections.

8.1 Volume of a Tetrahedron (Lattice)

$$V_{xyz} = \frac{1}{6} |\det [P_1 - P_0, P_2 - P_0, P_3 - P_0]| \quad (1)$$

Notes.

- $P_0, \dots, P_3$ are Cartesian (XYZ) vertex coordinates, in length units; each bracketed column is an edge vector, the determinant is the volume of the parallelepiped they span, and the $1/6$ factor converts it to the tetrahedron volume V_{xyz} in cubic length units. This is the coordinate (difference) form of the homogeneous-row determinant of Eq. (4); for IVM units directly from Quadray coordinates, see the native formula of Eq. (7).

Tom Ace 5×5 tetravolume (IVM units):

$$V_{ivm} = \frac{1}{4} \left| \det \begin{pmatrix} a_0 & b_0 & c_0 & d_0 & 1 \\ a_1 & b_1 & c_1 & d_1 & 1 \\ a_2 & b_2 & c_2 & d_2 & 1 \\ a_3 & b_3 & c_3 & d_3 & 1 \\ 1 & 1 & 1 & 1 & 0 \end{pmatrix} \right| \quad (2)$$

Notes.

- Row i lists the four Quadray components (a_i, b_i, c_i, d_i) of vertex P_i , augmented with an affine 1; the last row $(1, 1, 1, 1, 0)$ encodes the projective normalization constraint, so the determinant is invariant to adding (t, t, t, t) to every vertex (Section 3). Division by 4 returns the IVM tetravolume V_{ivm} ; for integer quadrays the determinant is an exact integer (Bareiss algorithm), computed as a `fractions.Fraction`. This matrix is identical, row for row, to the named matrix Q of Eq. (3).

8.2 Expanded Ace 5×5 Matrix

The Ace 5×5 matrix of Eq. (2) written out explicitly and named Q :

$$Q(P_0, P_1, P_2, P_3) = \begin{bmatrix} a_0 & b_0 & c_0 & d_0 & 1 \\ a_1 & b_1 & c_1 & d_1 & 1 \\ a_2 & b_2 & c_2 & d_2 & 1 \\ a_3 & b_3 & c_3 & d_3 & 1 \\ 1 & 1 & 1 & 1 & 0 \end{bmatrix}, \quad V_{ivm} = \frac{1}{4} |\det Q(P_0, \dots, P_3)| \quad (3)$$

Notes.

- **Matrix structure:** row i holds the four Quadray components (a_i, b_i, c_i, d_i) of vertex P_i , plus the affine coordinate 1 — literally the same rows as Eq. (2).
- **Last row:** $(1, 1, 1, 1, 0)$ enforces the projective normalization constraint (Section 3).

- **Volume computation:** $V_{ivm} = \frac{1}{4} |\det Q|$ in IVM units, exact for integer quadrays.
- **Notation:** the Ace matrix is written Q ; the symbol M is reserved for the Quadray-to-XYZ embedding matrix (Sections 3 and 14).

XYZ determinant volume and S3 conversion:

$$V_{xyz} = \frac{1}{6} \left| \det \begin{pmatrix} x_0 & y_0 & z_0 & 1 \\ x_1 & y_1 & z_1 & 1 \\ x_2 & y_2 & z_2 & 1 \\ x_3 & y_3 & z_3 & 1 \end{pmatrix} \right|, \quad V_{ivm} = S3 V_{xyz}, \quad S3 = \sqrt{\frac{9}{8}} \quad (4)$$

Notes.

- **Homogeneous-row determinant in Cartesian coordinates:** each row is $(x_i, y_i, z_i, 1)$ for vertex P_i , with coordinates in length units; the absolute determinant divided by 6 is the Euclidean volume V_{xyz} in cubic length units. This is the homogeneous form of Eq. (1); conversion to IVM units uses $V_{ivm} = S3 V_{xyz}$ with $S3 = \sqrt{9/8}$ as used throughout.

8.3 Cayley-Menger Determinant (Coxeter.4D)

For tetrahedron volume from edge lengths (Coxeter.4D approach):

$$288 V_{xyz}^2 = \det \begin{pmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & d_{01}^2 & d_{02}^2 & d_{03}^2 \\ 1 & d_{10}^2 & 0 & d_{12}^2 & d_{13}^2 \\ 1 & d_{20}^2 & d_{21}^2 & 0 & d_{23}^2 \\ 1 & d_{30}^2 & d_{31}^2 & d_{32}^2 & 0 \end{pmatrix} \quad (5)$$

Notes.

- **Pairwise distances:** d_{ij} is the Euclidean distance between vertices P_i and P_j in length units; the matrix stores the squared distances d_{ij}^2 , so the formula is length-only — no coordinates enter (Coxeter.4D).
- **Length-only formulation:** Cayley-Menger provides a length-only formula for simplex volumes, here specialized to tetrahedra.
- **Conversion to IVM:** V_{xyz} is the Euclidean volume in cubic length units; use $V_{ivm} = S3 V_{xyz}$ with $S3 = \sqrt{9/8}$ (Eq. (4)); the PdF formula of Eq. (6) consumes the same lengths in closed algebraic form.

8.4 Piero della Francesca Formula (PdF)

For tetrahedron volume from edge lengths meeting at a vertex:

$$144 V_{xyz}^2 = 4a^2 b^2 c^2 - a^2 (b^2 + c^2 - f^2)^2 - b^2 (c^2 + a^2 - e^2)^2 - c^2 (a^2 + b^2 - d^2)^2 + (b^2 + c^2 - f^2)(c^2 + a^2 - e^2)(a^2 + b^2 - d^2) \quad (6)$$

Notes.

- **Edge lengths:** a, b, c are the three edges meeting at the apex vertex P_0 — $a = d_{01}$, $b = d_{02}$, $c = d_{03}$ — and d, e, f are the respective opposite edges, $d = d_{23}$, $e = d_{13}$, $f = d_{12}$, in length units (the same d_{ij} as Eq. (5)).
- **Conversion to IVM:** V_{xyz} is the Euclidean volume in cubic length units (the prefactor 144 is the standard Heron-like PdF normalization); use $V_{ivm} = S3 V_{xyz}$ with $S3 = \sqrt{9/8}$ (Eq. (4)).

8.5 Gerald de Jong Formula (GdJ)

Native Quadray formula for tetrahedron volume:

$$V_{ivm} = \frac{1}{4} |\det [\pi(P_1) - \pi(P_0), \pi(P_2) - \pi(P_0), \pi(P_3) - \pi(P_0)]|, \quad \pi(P_i) = (a_i - d_i, b_i - d_i, c_i - d_i) \quad (7)$$

Notes.

- **Projection:** π maps a Quadray vertex P_i with components (a_i, b_i, c_i, d_i) to $\mathbb{R}^3$ by subtracting the fourth component from the first three; each bracketed column is a projected edge vector. Because π kills the direction $(1, 1, 1, 1)$, it is invariant under $q \rightarrow q + t(1, 1, 1, 1)$ — the projective fiber of Eq. (33) (Section 14) — so the determinant is unchanged by projective normalization of the vertices. When all four vertices share the same fourth component (in particular $d_i = 0$), $\pi(P_i)$ reduces to (a_i, b_i, c_i) .
- **Native IVM:** no S3 conversion; the factor $1/4$ returns IVM tetravolume directly — for the unit tetrahedron (origin plus three IVM neighbor moves) the determinant is exactly 4, giving $V_{ivm} = 1$ (Section 3).
- **Exact arithmetic:** integer Quadray coordinates give an integer determinant, evaluated exactly by the Bareiss algorithm as a `fractions.Fraction`; the code implements Eq. (7) as `integer_tetra_volume`, and the magnitude agrees exactly with the Ace determinant of Eq. (2) on every integer-quadray input.

See code: `tetra_volume_cayley_menger`. For tetrahedron volume background, see [Tetrahedron - volume](#). Exact integer determinants in code use the [Bareiss algorithm](#). External validation: these formulas align with implementations in the 4dsolutions ecosystem. See the [Resources](#) section for comprehensive details.

8.6 Fisher Information Matrix (FIM)

Background: [Fisher information](#).

$$F_{i,j} = \mathbb{E} \left[\frac{\partial \log p(x; \theta)}{\partial \theta_i} \frac{\partial \log p(x; \theta)}{\partial \theta_j} \right] \quad (8)$$

Notes.

- **Symbols:** x is an observation, $\theta = (\theta_1, \dots, \theta_m)$ the parameter vector, and $p(x; \theta)$ the likelihood; the expectation is over $x \sim p(\cdot; \theta)$, so F is a function of θ . Each score $\partial \log p(x; \theta) / \partial \theta_i$ carries inverse parameter units, so $F_{i,j}$ carries inverse squared parameter units per observation.
- F is symmetric positive semi-definite: large eigenvalues mark sensitive (high-curvature) directions, small eigenvalues sloppy ones (Section 4 and the Discussion). The empirical estimate of Eq. (9) is the per-observation counterpart.
- See code: `fisher_information_matrix` in `src/quadmath/inference/information.py` — empirical outer-product estimator. Figure: the empirical estimate is shown in the FIM heatmap figure of Section 4.

8.7 Empirical Fisher Information Matrix

For empirical estimation from data, the Fisher Information Matrix is computed as:

$$F_{i,j} = \frac{1}{N} \sum_{n=1}^N \frac{\partial \log p(x_n; \theta)}{\partial \theta_i} \frac{\partial \log p(x_n; \theta)}{\partial \theta_j} \quad (9)$$

Notes.

- **Symbols:** $x_1, \dots, x_N$ are N i.i.d. observations and $g_n = \nabla_{\theta} \log p(x_n; \theta)$ the per-sample score vector (same units as Eq. (8)); the estimator is $F = \frac{1}{N} \sum_n g_n g_n^T$, with a `normalize` flag controlling the $1/N$ division.

- Converges to Eq. (8) as $N \rightarrow \infty$; used by natural-gradient descent (Eq. (10)) and the information-geometry applications of Section 4.

8.8 Natural Gradient

Background: [Natural gradient](#) (Amari).

$$\theta \leftarrow \theta - \eta F(\theta)^{-1} \nabla_{\theta} L(\theta) \quad (10)$$

Explanation.

- **Symbols:** θ the parameter vector, $L(\theta)$ a differentiable loss, $\nabla_{\theta} L$ its gradient, $F(\theta)$ the Fisher matrix of Eq. (8) — or its empirical estimate, Eq. (9) — evaluated at θ , and $\eta > 0$ the step size (learning rate).
- **Update:** right-preconditioning by the inverse Fisher metric gives the steepest-descent direction under the Fisher metric (Amari), invariant to smooth invertible reparameterizations of θ .
- **Damping:** the implementation `natural_gradient_step` solves the damped system $(F + \lambda I) \delta = \nabla_{\theta} L$ and applies $\theta \leftarrow \theta - \eta \delta$, with a small Tikhonov ridge λ (default 10^{-9}) keeping the solve stable when F is near-singular.

See code: `natural_gradient_step` in `src/quadmath/inference/information.py` — damped inverse-Fisher step.

8.9 Free Energy (Active Inference)

$$\mathcal{F} = -\log P(o \mid s) + \text{KL}[Q(s) \parallel P(s)] \quad (11)$$

Explanation.

- **Symbols:** o the observation, s the latent state, $Q(s)$ the approximate posterior (recognition distribution), and P the generative model, so $P(o \mid s)$ is the likelihood and $P(s)$ the prior; $\mathcal{F}$ is the variational free energy in nats (natural logarithm, Section 10). **Notation flag:** capital P here is a probability distribution, unrelated to the vertex labels P_i of the volume sections.
- **Partition:** minimizing $\mathcal{F}$ over Q trades the expected negative log-likelihood $\mathbb{E}_Q[-\log P(o \mid s)]$ (the displayed $-\log P(o \mid s)$ is read under this Q -expectation, as implemented) against the KL divergence $\text{KL}[Q \parallel P]$ that ties the posterior to the prior; see [Free energy principle](#).

See code: `free_energy` in `src/quadmath/inference/information.py` — discrete-state variational free energy (inputs are unnormalized distributions, normalized internally).

Note: The main figures demonstrating natural gradient trajectories and free energy landscapes are shown in [Section 4: Optimization in 4D](#). The appendix focuses on unique figures specific to mathematical formulations and validation.

8.10 Expected Free Energy (Active Inference)

Background: [Active Inference](#) (Parr, Pezzulo & Friston, MIT Press, 2022).

$$G = \text{KL}[Q(s) \parallel P(s)] - \mathbb{E}_q[\log P(o \mid s)] - \log P(o) \quad (12)$$

Explanation.

- **Symbols:** $Q(s)$ the approximate posterior and $P(s)$ the prior over states (as in Eq. (11)), $P(o \mid s)$ the likelihood, $P(o)$ the prior preference over outcomes in nats (uniform when omitted), $H[Q] = -\sum_s Q(s) \log Q(s)$ the Shannon entropy in nats, and $\mathbb{E}_q$ expectation under Q ; G is the expected free energy minimized during action selection.
- **Epistemic term:** the KL divergence between variational posterior and prior over states.

- **Entropy:** the posterior entropy is already inside the KL term, since $\text{KL}[Q\|P] = \mathbb{E}_q[\log Q(s)] - \mathbb{E}_q[\log P(s)] = -H[Q(s)] - \mathbb{E}_q[\log P(s)]$; subtracting H again would count it twice.
- **Ambiguity:** the negative expected log-likelihood of outcomes penalizes noisy observations.
- **Pragmatic term:** prior preferences $P(o)$ enter negatively ($-\log P(o)$), so preferred outcomes lower G ; agents minimize G during action selection.

See code: `expected_free_energy` in `src/quadmath/inference/information.py` — all four terms of Eq. (12) as implemented.

8.11 Quadray Normalization (Fuller.4D)

Given a Quadray $q = (a, b, c, d)$, choose $k = \min(a, b, c, d)$ and set $q' = q - (k, k, k, k)$, enforcing non-negative entries with at least one zero. The shift is the projective normalization of Section 3 — q and q' represent the same direction, the fiber formalized in Eq. (33) (Section 14). Here k is the normalization offset of the glossary (Section 10); it is unrelated to the shell-frequency k of the lattice sections (Sections 11 and 13).

8.12 Distance (Embedding Sketch; Coxeter.4D slice)

Choose a linear map M from Quadray space to $\mathbb{R}^3$ (or $\mathbb{R}^4$) consistent with the tetrahedral axes — the embedding matrix of Sections 3 and 14 (canonical Urner family, Eq. (32)). Then for Quadray points p, q , the distance is $d(p, q) = \|M(p - q)\|_2$ in length units; under the integer Urner family the squared distance is the exact integer identity of Eq. (36), which the lattice-search layer ranks with (Section 13).

8.13 Minkowski Line Element (Einstein.4D analogy)

$$ds^2 = -c^2 dt^2 + dx^2 + dy^2 + dz^2 \quad (13)$$

Background: [Minkowski space](#).

Signature convention $(-, +, +, +)$ (mostly-plus, Section 2): c is the speed of light, (t, x, y, z) are spacetime coordinates in time and length units respectively, and ds^2 has units of length squared. Here c is the physical constant, not the Quadray component or PDF edge length used in the volume sections.

8.14 High-Precision Arithmetic Note

When evaluating determinants, FIMs, or geodesic distances for sensitive problems, use quad precision (binary128) via GCC's `libquadmath` (`__float128`, functions like `expq`, `sqrtq`, and `quadmath_snprintf`). See [GCC libquadmath](#). Where possible, it is useful to use symbolic math libraries like SymPy to compute exact values.

8.14.1 Reproducibility artifacts and external validation

- **This manuscript's artifacts:** Raw data in `quadmath/output/` for reproducibility and downstream analysis:
 - `fisher_information_matrix.csv` / `.npz`: empirical Fisher matrix and inputs
 - `fisher_information_eigenvalues.csv` / `fisher_information_eigensystem.npz`: eigenspectrum and eigenvectors
 - `natural_gradient_path.png` with `natural_gradient_path.csv` / `.npz`: projected trajectory and raw coordinates
 - `ivm_neighbors_data.csv` / `ivm_neighbors_edges_data.npz`: neighbor coordinates (Quadray and XYZ)
 - `polyhedra_quadray_constructions.png`: synergetics volume relationships schematic
- **External validation resources:** The [4dsolutions ecosystem](#) provides extensive cross-validation. See the [Resources](#) section for comprehensive details on computational implementations and validation.

8.15 Namespaces summary (notation)

- Coxeter.4D: Euclidean E^4 ; regular polytopes; not spacetime (cf. Coxeter, Regular Polytopes, Dover ed., p. 119). Connections to higher-dimensional lattices and packings as in Conway & Sloane.
- Einstein.4D: Minkowski spacetime; indefinite metric; used here only as a metric analogy when discussing geodesics and information geometry.

- Fuller.4D: Quadrays/IVM; tetrahedral lattice with integer tetravolume; unit regular tetrahedron has volume 1; synergetics scale relations (e.g., S3).

9 Appendix: The Free Energy Principle and Active Inference

9.1 Overview

The Free Energy Principle (FEP) posits that biological systems maintain their states by minimizing variational free energy, thereby reducing surprise via prediction and model updating. Active Inference extends this by casting action selection as inference under prior preferences. Background: see the concise overview on the [Free energy principle](#) and the monograph [Active Inference \(MIT Press\)](#).

This appendix emphasizes relationships among: (i) the four-fold partition of Active Inference, (ii) Quadrays (Fuller.4D) as a geometric scaffold for mapping this partition, and (iii) information-geometric flows (Einstein.4D analogy) that underpin perception-action updates. For the naming of 4D namespaces used throughout—Coxeter.4D (Euclidean E4), Einstein.4D (Minkowski spacetime analogy), Fuller.4D (Synergetics/Quadrays)—see `02_4d_namespaces.md`.

9.2 Mathematical Formulation and Equation Callouts (Equations linkage)

- Variational free energy (discrete states) — see Eq. (11) in the equations appendix, implemented by `free_energy`: $\mathcal{F} = -\log P(o | s) + \text{KL}[Q(s) \| P(s)]$, where o are observations, s latent states, $Q(s)$ the variational posterior, and $P(s)$ the prior; minimizing $\mathcal{F}$ drives perception.
- Expected free energy (action selection) — see Eq. (12) in the equations appendix, implemented by `expected_free_energy`: $G = \text{KL}[Q(s) \| P(s)] - \mathbb{E}_q[\log P(o | s)] - \log P(o)$, where the epistemic term is the KL divergence between posterior $Q(s)$ and prior $P(s)$ over states (its entropy part, $\mathbb{E}_q[\log Q(s)] = -H[Q(s)]$, is already included, so H is not subtracted again), the ambiguity term $-\mathbb{E}_q[\log P(o | s)]$ penalizes expected surprise about outcomes o , and the pragmatic preference $-\log P(o)$ lowers G for preferred outcomes; agents minimize G when selecting actions.
- Fisher Information Matrix (FIM) as metric — see Eq. (8) in the equations appendix and `fisher_information_matrix`: $F_{ij} = \mathbb{E}[\partial_{\theta_i} \log p(x; \theta) \partial_{\theta_j} \log p(x; \theta)]$, the expected outer product of score functions $\partial_{\theta_i} \log p(x; \theta)$, serving as the Riemannian metric on parameter space θ .
- Natural gradient descent under information geometry — see Eq. (10) in the equations appendix and `natural_gradient_step`: the update $\theta \leftarrow \theta - \eta F^{-1} \nabla_{\theta} L$ preconditions the loss gradient $\nabla_{\theta} L$ by the inverse Fisher metric F^{-1} at step size η ; the implementation solves the damped system $(F + \varepsilon I) \delta = \nabla_{\theta} L$ and moves by $-\eta \delta$. Overview: [Natural gradient](#).

Figures: three Active Inference figures follow — the partition tetrahedron, the 4D natural-gradient trajectory (`figure_13_4d_trajectory.png`), and the free-energy landscape (`figure_14_free_energy_landscape.png`).

Discrete variational optimization on the quadray lattice: `discrete_ivm_descent` greedily descends a free-energy-like objective over IVM moves, yielding integer-valued trajectories. See the path animation artifact `discrete_path.mp4` in `quadmath/output/`.

9.3 Four-Fold Partition and Tetrahedral Mapping (Quadrays; Fuller.4D)

Active Inference partitions the agent-environment system into four coupled states:

- Internal (μ) — agent’s internal states
- Sensory (s) — observations
- Active (a) — actions
- External (ψ) — latent environmental causes

See, for an overview of this partition and generative process formulations, the [Active Inference review](#) and the general entry on [Active inference](#).

Tetrahedral mapping via Quadrays (Fuller.4D): assign each state to a vertex of a tetrahedron, using Quadray coordinates (A, B, C, D) with non-negative components and at least one zero after normalization. One canonical

Four-fold partition mapped to Quadray tetrahedron

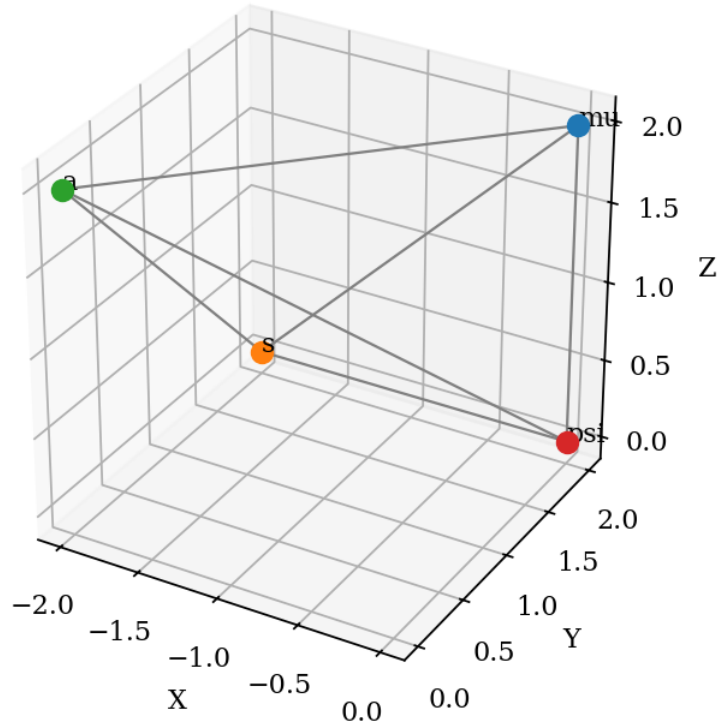


Figure 15: **Active Inference four-fold partition mapped to a Quadray tetrahedron in Fuller.4D.** The four vertices of a regular tetrahedron, embedded in XYZ via the quadray embedding t_{0_xyz} (axes range -2 to 2), carry the four partition components: μ internal states (blue), s sensory observations (orange), a actions (green), and ψ external causes (red). The gray edges are the six pairwise couplings of the partition — e.g., μ - s perceptual inference and a - ψ control (see the partition section below). Generated by `quadmath/scripts/active_inference_figures.py` (MPLBACKEND=Agg, fixed seed 42).

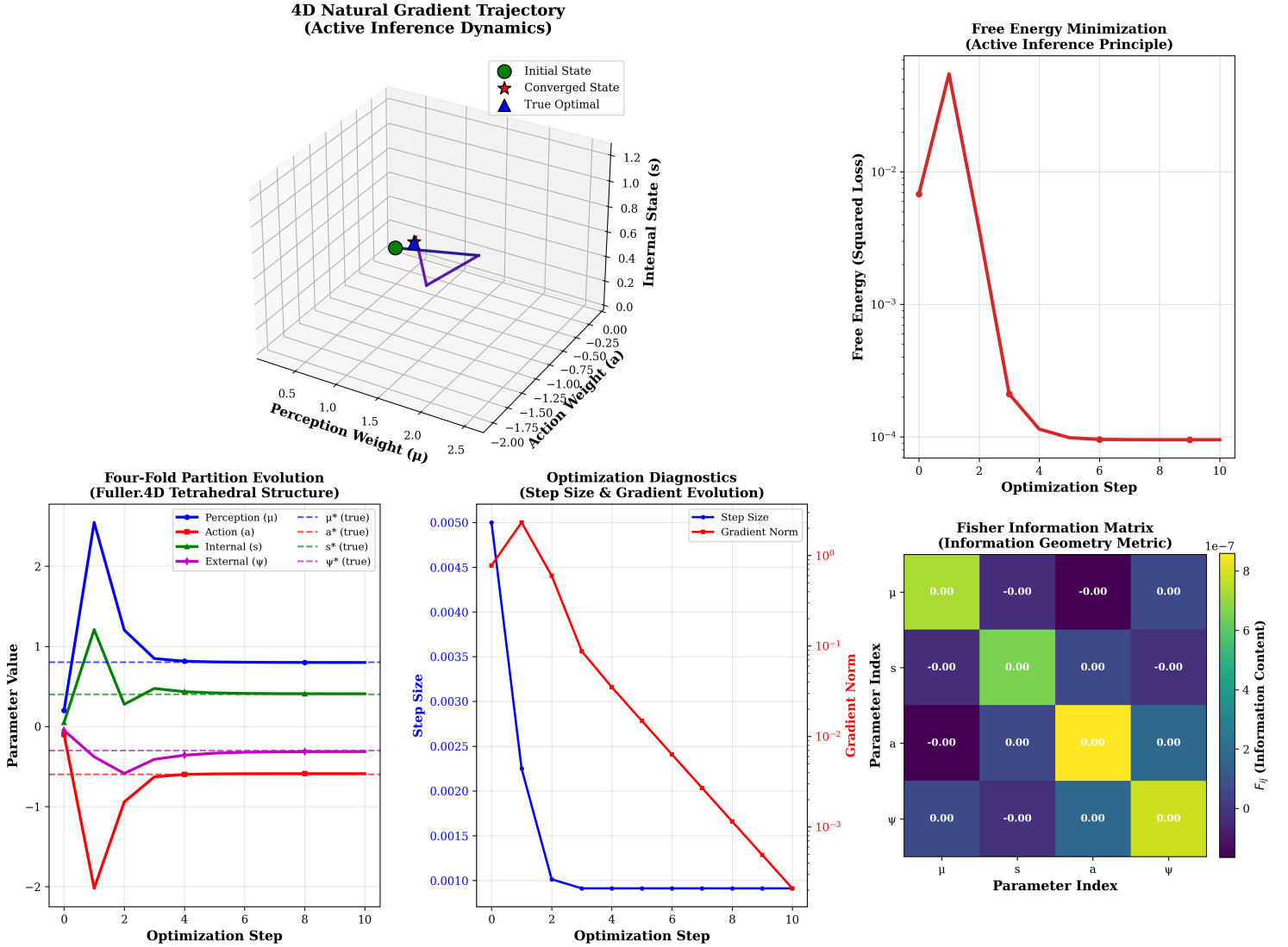


Figure 16: **4D natural-gradient trajectory with Active Inference context** (figure_13_4d_trajectory.png). Top left: the trajectory of natural gradient descent in (μ, a, s) coordinates — perception weight μ , action weight a , internal-state coordinate s — from the initial state (green circle) through the converged state (red star) toward the true optimum (blue triangle), converging in 11 steps with final parameter errors below 0.015. Top right: free energy (squared loss, log scale) falling from about 10^{-1} to 10^{-4} over the 11 optimization steps. Bottom left: evolution of the four partition components μ, a, s, ψ against their true values (dashed lines). Bottom middle: step size (linear scale) and gradient norm (log scale) per step. Bottom right: the 4×4 Fisher information matrix F_{ij} (entries of order 10^{-7}) that preconditions the updates via Eq. (10). Generated by quadmath/scripts/active_inference_figures.py (MPLBACKEND=Agg, fixed seed 42); raw data in figure_13_data.npz.

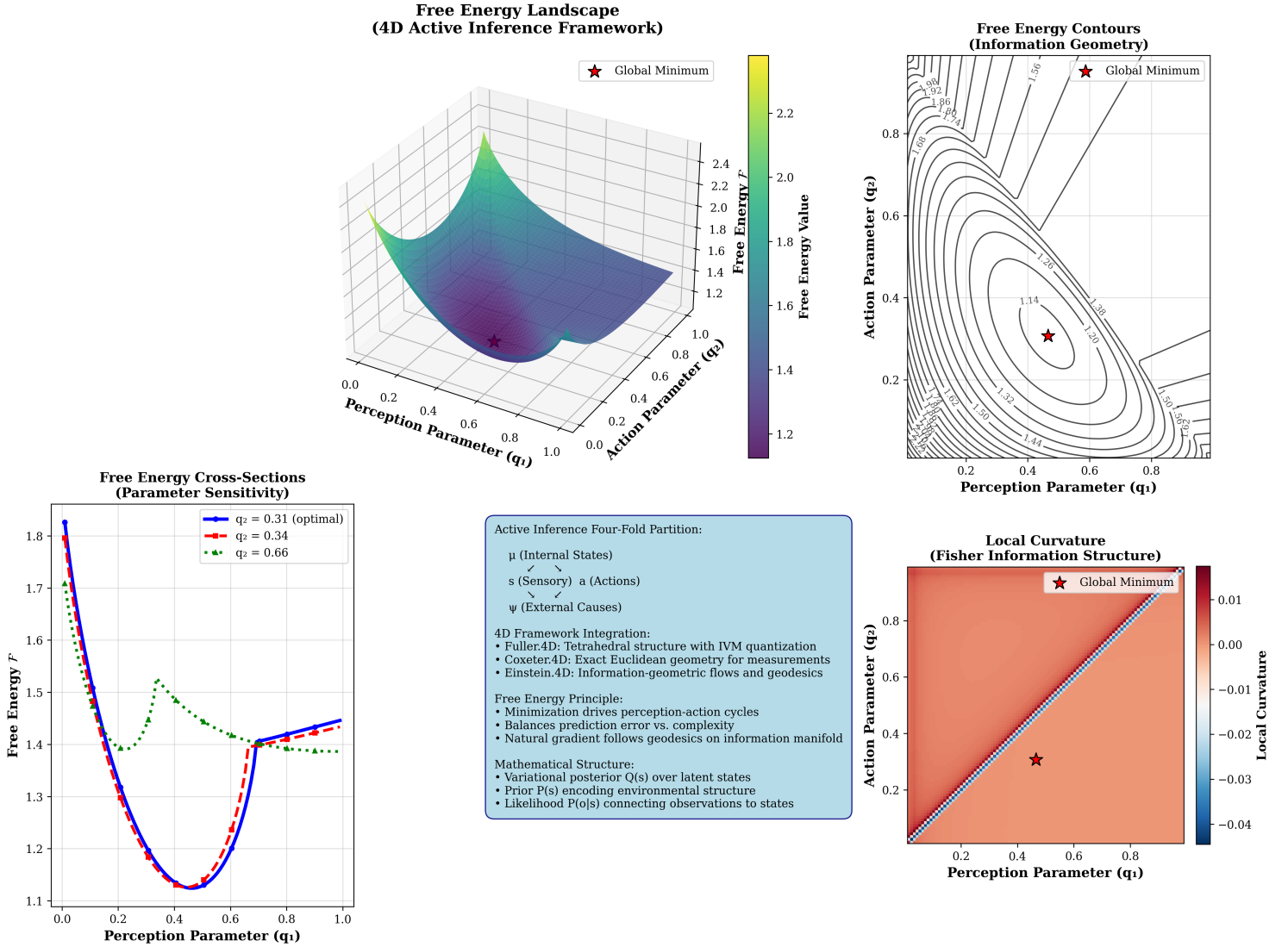


Figure 17: **Free-energy landscape over perception and action parameters** (figure_14_free_energy_landscape.png). Top left: the variational free energy $\mathcal{F}$ of Eq. (11) as a surface over the perception parameter q_1 and action parameter q_2 (values in nats, about 1.2 to 2.4; color bar at right), with the global minimum starred. Top right: level contours of the same surface. Bottom left: one-dimensional cross-sections $\mathcal{F}(q_1)$ at three fixed values of q_2 , showing sensitivity to each parameter. Bottom right: local curvature over (q_1, q_2) (Fisher information structure). The text panel summarizes the four-fold partition (μ, s, a, ψ) , the roles of Fuller.4D/Coxeter.4D/Einstein.4D, and the generative-model objects $Q(s)$, $P(s)$, $P(o | s)$. Generated by quadmath/scripts/active_inference_figures.py (MPLBACKEND=Agg, fixed seed 42); raw data in figure_14_data.npz.

mapping is $A \leftrightarrow \mu$ (internal), $B \leftrightarrow s$ (sensory), $C \leftrightarrow a$ (active), $D \leftrightarrow \psi$ (external). The edges capture the pairwise couplings (e.g., μ - s for perceptual inference; a - ψ for control). Integer tetravolume then quantifies the “coupled capacity” region spanned by jointly feasible states in a time slice; see `Quadray` and `tetravolume` methods in `03_quadray_methods.md`.

Interpretation note: this Quadray-based mapping is a didactic geometric scaffold. It is not standard in the Active Inference literature, which typically develops the four-state partition in probabilistic graphical terms. Our use highlights structural symmetries and discrete volumetric quantities available in Fuller.4D, building on the computational foundations developed in the [4dsolutions ecosystem](#) for tetrahedral modeling and volume calculations. See the [Resources](#) section for comprehensive details on the computational implementations.

Code linkage (no snippet): see `example_partition_tetra_volume` in `src/quadmath/core/examples.py` and the partition tetrahedron figure above.

9.4 How the 4D namespaces relate here

- Fuller.4D (Quadrays): geometric embedding of the four-state partition on a tetrahedron; integer tetravolumes and IVM moves provide discrete combinatorial structure.
- Coxeter.4D (Euclidean E4): exact Euclidean measurements (e.g., Cayley–Menger determinants) for tetrahedra underlying volumetric comparisons and scale relations.
- Einstein.4D (Minkowski analogy): information-geometric flows (natural gradient, metric-aware updates) supply a continuum picture for perception–action dynamics.

The three roles are complementary: Fuller.4D encodes partition structure, Coxeter.4D provides exact metric geometry for static comparisons, and Einstein.4D guides dynamical descent.

9.5 Joint Optimization in the Tetrahedral Framework (Methods linkage)

- Perception: update μ to minimize prediction error on s under the generative model; `perception_update` integrates the flow $\dot{\mu} = D\mu - \partial_{\mu}F$, where D is the model’s derivative operator and F the free energy of Eq. (11).
- Action: select a that steers ψ toward preferred outcomes; `action_update` descends the action gradient, $a \leftarrow a - \eta \partial_a F$ at step size η .

Continuous-time flows (Einstein.4D analogy for metric/geodesic intuition): see `perception_update` and `action_update` in `src/quadmath/inference/information.py`; their discrete Quadray counterpart is `discrete_ivm_descent` in `src/quadmath/optimize/discrete_variational.py`, described above.

9.6 Implications for AI and Robust Computation

FEP/Active Inference provide algorithms that unify perception and action under uncertainty, offering biologically plausible alternatives to standard RL with adaptive exploration and robust decision-making. See [applications in AI \(arXiv:1907.03876\)](#).

9.7 Code, Reproducibility, and Cross-References

– Equation references: [Eq. \(Free Energy\)](#), [Eq. \(FIM\)](#), [Eq. \(Natural Gradient\)](#) in `08_equations_appendix.md`. – Code anchors (for readers who want to run experiments): `free_energy`, `fisher_information_matrix`, `natural_gradient_step`, `perception_update`, `action_update`, and `discrete_ivm_descent` in `src/quadmath/inference/information.py` and `src/quadmath/optimize/discrete_variational.py`.

Demo and figures generated by `quadmath/scripts/information_demo.py` and `quadmath/scripts/active_inference_figures.py` output to `quadmath/output/`:

- **Active Inference Visualizations (embedded above):** `figure_13_4d_trajectory.png`, `figure_14_free_energy_landscape.png`, `partition_tetrahedron.png`
- **Information Geometry Visualizations** (rendered in the Optimization in 4D section): `fisher_information_matrix.png`, `fisher_information_eigenspectrum.png`, `natural_gradient_path.png`, `free_energy_curve.png`

- **Raw data:** figure_13_data.npz, figure_14_data.npz, fisher_information_matrix.csv, fisher_information_matrix.npz (F, grads, X, y, w_true, w_est), fisher_information_eigenvalues.csv, fisher_information_eigensystem.npz
- **External validation:** Cross-reference with volume calculations and tetrahedral modeling tools from the [4dsolutions ecosystem](#). See the [Resources](#) section for comprehensive details.

10 Appendix: Symbols and Glossary

This appendix consolidates the symbols, variables, and constants used throughout the manuscript.

10.1 Sets and Spaces

Symbol	Name
$\mathbb{R}^n$	Euclidean space
IVM	Isotropic Vector Matrix
Coxeter.4D	Euclidean 4D (E^4)
Einstein.4D	Minkowski spacetime (3+1)
Fuller.4D	Synergetics/Quadray tetrahedral space

Descriptions:

- $\mathbb{R}^n$: n -dimensional real vector space.
- IVM: Quadray integer lattice (CCP sphere centers).
- Coxeter.4D: Four-dimensional Euclidean geometry (not spacetime); see Coxeter, Regular Polytopes (Dover ed., p. 119); related lattice/packing background in Conway & Sloane.
- Einstein.4D: Relativistic spacetime with Minkowski metric.
- Fuller.4D: Quadrays with projective normalization and IVM unit conventions.

10.2 Quadray Coordinates and Geometry

Symbol	Name	Description
$q = (a, b, c, d)$	Quadray point	Non-negative coordinates with at least one zero after normalization
A, B, C, D	Quadray axes	Canonical tetrahedral axes mapped by the embedding
k	Normalization offset	$k = \min(a, b, c, d)$ used to set $q' = q - (k, k, k, k)$
q'	Normalized Quadray	Canonical representative with at least one zero and non-negative entries
$P_0, \dots, P_3$	Tetrahedron vertices	Vertices used in volume formulas
d_{ij}	Pairwise distances	Distance between vertices P_i and P_j (squared in CM matrix)
$\det(\cdot)$	Determinant	Determinant of a matrix
$ \cdot $	Magnitude	Absolute value (determinant magnitude)
V_{ivm}	Tetrvolume (IVM)	Tetrahedron volume in synergetics/IVM units; unit regular tetra has $V_{ivm} = 1$
V_{xyz}	Tetrvolume (XYZ)	Euclidean tetrahedron volume
$S3$	Scale factor	$S3 = \sqrt{9/8}$ with $V_{ivm} = S3 V_{xyz}$ (synergetics unit convention)
Coxeter.4D	Namespace	Euclidean E^4 ; regular polytopes
Einstein.4D	Namespace	Minkowski spacetime (metric analogy only here)
Fuller.4D	Namespace	Quadrays/IVM; integer tetrvolume

Symbol	Name	Description
Eq. (lattice_det)	Lattice determinant	Integer-lattice volume via 3x3 determinant
Eq. (ace5x5)	Tom Ace 5x5	Direct IVM tetravolume from Quadrays
Eq. (cayley_menger)	Cayley-Menger	Length-based formula: $288 V^2 = \det(\cdot)$

10.3 Optimization and Algorithms

Symbol	Name
α	Reflection coefficient
γ	Expansion coefficient
ρ	Contraction coefficient
σ	Shrink coefficient
V_{ivm}	Integer volume monitor

Descriptions:

- $\alpha, \gamma, \rho, \sigma$: Nelder-Mead parameters (typical values 1, 2, 0.5, 0.5).
- V_{ivm} : Tracks simplex volume across iterations.

10.4 Information Theory and Geometry

Symbol	Name	Description
$\log$	Natural logarithm	Logarithm base e
$\mathbb{E}[\cdot]$	Expectation	Mean with respect to a distribution
F_{ij}	Fisher Information Matrix	$\mathbb{E}[\partial_{\theta_i} \log p \cdot \partial_{\theta_j} \log p]$; Eq. (8) in the equations appendix
$\mathcal{F}$	Variational free energy	$-\log P(o s) + \text{KL}[Q(s) \ P(s)]$; Eq. (11) in the equations appendix
$\text{KL}[Q \ P]$	Kullback-Leibler divergence	$\sum Q \log(Q/P)$; information distance
$\nabla_{\theta} L$	Natural gradient	$F(\theta)^{-1} \nabla_{\theta} L(\theta)$; Eq. (10) in the equations appendix
η	Step size	Learning-rate scalar used in updates
θ	Parameters	Model parameter vector; indices θ_i
ds^2	Minkowski line element	$-c^2 dt^2 + dx^2 + dy^2 + dz^2$; Eq. (13) in the equations appendix
c	Speed of light	Physical constant appearing in Minkowski metric

10.5 Embeddings and Distances

Symbol	Name	Description
M	Embedding matrix	Linear map from Quadray to $\mathbb{R}^3$ (Urner-style unless noted)
$\ \cdot\ _2$	Euclidean norm	$\sqrt{x_1^2 + \dots + x_n^2}$
R, D	Edge scales	Cube edge R and Quadray edge D with $D = 2R$ (common convention)

10.6 Greek Letters (usage)

Symbol	Name	Description
$\alpha, \gamma, \rho, \sigma$	NM coefficients	Nelder-Mead parameters (reflection, expansion, contraction, shrink)
θ	Theta	Parameter vector in models and metrics
μ	Mu	Internal states (Active Inference)
ψ	Psi	External states (Active Inference)
η	Eta	Step size / learning rate

10.7 Notes (usage and cross-references)

- **Figures referenced:** In-text references use LaTeX’s automatic figure numbering for consistent cross-referencing.
- **Equation references:** Use labels defined in the text (e.g., Eq. (1) in the equations appendix).
- **Namespaces:** We use Coxeter.4D, Einstein.4D, Fuller.4D consistently to designate Euclidean E^4 , Minkowski spacetime, and Quadray/IVM synergetics, respectively. This avoids conflation of Euclidean 4D objects (e.g., tesseracts) with spacetime constructs and synergetic tetravolume conventions.
- **External validation:** Cross-reference implementations from the [4dsolutions ecosystem](#) for algorithmic verification and performance comparison baselines. See the [Resources](#) section for comprehensive details.

10.8 Polyhedra and Synergetic Shapes

Symbol	Name	Description
Tetrahedron	Regular tetrahedron	Fundamental unit with $V=1$ in IVM units
Cube	Regular hexahedron	$V=3$ in IVM units; orthogonal space-filling
Octahedron	Regular octahedron	$V=4$ in IVM units; edge-midpoint construction
Rhombic Dodecahedron	12-faced solid	$V=6$ in IVM units; Voronoi cell of FCC packing
Cuboctahedron	Vector equilibrium	$V=20$ in IVM units; shell of 12 IVM neighbors
Truncated Octahedron	Archimedean solid	$V=20$ in IVM units; space-filling tiling

10.9 Acronyms and abbreviations

Acronym	Meaning
CM	Cayley-Menger (determinant-based tetrahedron volume)
PdF	Piero della Francesca (Heron-like tetrahedron volume)
GdJ	Gerald de Jong (Quadray-native tetravolume expression)
K-FAC	Kronecker-Factored Approximate Curvature (optimizer using structured Fisher)
CCP	Cubic Close Packing (same centers as FCC)
FCC	Face-Centered Cubic (same centers as CCP)
E ⁴	Four-dimensional Euclidean space (Coxeter.4D)
NM	Nelder-Mead (simplex optimization algorithm)
4dsolutions	Kirby Urner's GitHub organization with extensive Quadray implementations
BEAST	Synergetic modules (B, E, A, S, T) in Fuller's hierarchical system
OCN	Oregon Curriculum Network (educational framework integrating Quadrays)
POV-Ray	Persistence of Vision Raytracer (used in quadcraft.py visualizations)

10.10 API Index (auto-generated; Methods linkage)

The table below enumerates public symbols from `src/` modules.

Module	Symbol	Kind	Signature	Summary
<code>quadmath.core.cayley_menger</code>	<code>ivm_tetra_volume_cayley_menger</code>	function	(d2)	Compute IVM tetravolume from squared distances via Cayley-Menger.
<code>quadmath.core.cayley_menger</code>	<code>squared_distances_from_quadray</code>	function	(p0, p1, p2, p3, embedding)	Build the 4x4 squared-distance matrix from four quadray vertices.
<code>quadmath.core.cayley_menger</code>	<code>tetra_circumradius</code>	function	(d2)	Circumscribed sphere radius of a tetrahedron from squared distances.
<code>quadmath.core.cayley_menger</code>	<code>tetra_inradius</code>	function	(d2)	Inscribed sphere radius of a tetrahedron from squared distances.
<code>quadmath.core.cayley_menger</code>	<code>tetra_volume_cayley_menger</code>	function	(d2)	Compute Euclidean tetrahedron volume from squared distances (Coxeter.4D).
<code>quadmath.core.examples</code>	<code>example_cuboctahedron_neighbors</code>	function	()	Return twelve-around-one IVM neighbors (vector equilibrium shell).

Module	Symbol	Kind	Signature	Summary
quadmath.core.examples	example_cuboctahedron_vertices	function	()	Return XYZ coordinates for the twelve-around-one neighbors.
quadmath.core.examples	example_ivm_neighbors	function	()	Return the 12 nearest IVM neighbors as permutations of {2,1,1,0} (Fuller.4D).
quadmath.core.examples	example_optimize	function	()	Run Nelder–Mead over integer quadrays for a simple convex objective (Fuller.4D).
quadmath.core.examples	example_partition_tetra_volume	function	(mu, s, a, psi)	Construct a tetrahedron from the four-fold partition and return tetravolume (Fuller.4D).
quadmath.core.examples	example_volume	function	()	Return the exact IVM tetravolume of the primitive lattice tetrahedron.
quadmath.core.geometry	lorentz_factor	function	(v, c)	Lorentz factor $\gamma = 1 / \sqrt{1 - v^2/c^2}$ (Einstein.4D).
quadmath.core.geometry	minkowski_interval	function	(dt, dx, dy, dz, c)	Return the Minkowski interval squared ds^2 (Einstein.4D).
quadmath.core.geometry	proper_time	function	(dt, dx, dy, dz, c)	Proper time elapsed for a timelike interval (Einstein.4D).
quadmath.core.geometry	spacetime_classify	function	(ds2, tol)	Classify a Minkowski interval squared as timelike, spacelike, or lightlike.
quadmath.core.linalg_utils	bareiss_determinant_int	function	(matrix)	Compute an exact integer determinant using the Bareiss algorithm.
quadmath.core.linalg_utils	bareiss_rank	function	(matrix)	Compute the exact integer rank of a matrix via Bareiss elimination.

Module	Symbol	Kind	Signature	Summary
quadmath.core. linalg_utils	integer_adjugate	function	(matrix)	Compute the exact integer adjugate (classical adjoint) of a square matrix.
quadmath.core.metrics	angle_error	function	(q1, q2)	Geodesic rotation angle between two quaternions, in radians.
quadmath.core.metrics	fim_eigenspectrum	function	(F)	Eigen-decomposition of a Fisher information matrix.
quadmath.core.metrics	fisher_condition_number	function	(F)	Compute the condition number of the Fisher information matrix.
quadmath.core.metrics	fisher_curvature_analysis	function	(F)	Comprehensive analysis of Fisher information matrix curvature.
quadmath.core.metrics	fisher_quadray_comparison	function	(F_cartesian, F_quadray)	Compare Fisher information matrices between coordinate systems.
quadmath.core.metrics	fisher_rao_metric	function	(p, q, eps)	Fisher-Rao geodesic distance on the probability simplex.
quadmath.core.metrics	information_length	function	(path_gradients)	Gradient-weighted proxy for informational path length (NOT the Jensen-Shannon divergence between two discrete distributions).
quadmath.core.metrics	jensen_shannon_divergence	function	(p, q, eps)	Jensen-Shannon divergence between two discrete distributions.
quadmath.core.metrics	kl_divergence	function	(p, q, eps)	Kullback-Leibler divergence between two discrete distributions.
quadmath.core.metrics	quat_log_euclidean_dispersion	function	(quats)	Root-mean-square chordal dispersion of quaternions about their mean.
quadmath.core.metrics	shannon_entropy	function	(p, eps)	Shannon entropy $H(p)$ for a discrete distribution.
quadmath.core.quadray	DEFAULT_EMBEDDING	constant	`quadmath.core. quadray` `Quadray` class	Quadray vector with non-negative components and at least one zero (Fuller.4D).

Module	Symbol	Kind	Signature	Summary
quadmath.core.quadray	ace_tetravolume_5x5	function	(p0, p1, p2, p3)	Tom Ace 5x5 determinant as the exact IVM tetra-volume (Fuller.4D).
quadmath.core.quadray	angle	function	(q1, q2, q3, embedding)	Angle at vertex q2 formed by rays q2->q1 and q2->q3 (radians).
quadmath.core.quadray	centroid	function	(*quads)	Component-wise mean of quadray points, rounded to the nearest lattice point.
quadmath.core.quadray	distance	function	(q1, q2, embedding)	Euclidean distance between two quadray points under the given embedding.
quadmath.core.quadray	dot	function	(q1, q2, embedding)	Return Euclidean dot product <q1,q2> under the given embedding.
quadmath.core.quadray	integer_tetra_volume	function	(p0, p1, p2, p3)	Compute the exact IVM tetra-volume of a lattice tetrahedron (Fuller.4D).
quadmath.core.quadray	magnitude	function	(q, embedding)	Return the Euclidean magnitude of q under the given embedding (vector norm).
quadmath.core.quadray	qconjugate	function	(q)	Conjugate (w, -x, -y, -z) of a quaternion in (w, x, y, z) order.
quadmath.core.quadray	qmul	function	(a, b)	Hamilton product of two quaternions.
quadmath.core.quadray	qrotate	function	(q, v_xyz, angle)	Rotate a 3-vector by a unit quaternion via Rodrigues ($v' = q v q^*$).
quadmath.core.quadray	quadray_from_xyz	function	(x, y, z, embedding)	Map an R^3 point back to the quadray lattice via pseudoinverse rounding.
quadmath.core.quadray	rotate_about_axis	function	(v_xyz, axis_xyz, angle)	Rotate a 3-vector about an axis by an angle (axis-angle convenience).

Module	Symbol	Kind	Signature	Summary
quadmath.core.quadray	slerp	function	(qa, qb, t)	Shortest-arc spherical linear interpolation between unit quaternions.
quadmath.core.quadray	to_xyz	function	(q, embedding)	Map quadray to R^3 via a 3x4 embedding matrix (Fuller.4D -> Coxeter.4D slice).
quadmath.core.symbolic	cayley_menger_volume_symbolic	function	(d2)	Return symbolic Euclidean tetrahedron volume from squared distances.
quadmath.core.symbolic	convert_xyz_volume_to_ivm	function	(V_xyz)	Convert a symbolic Euclidean volume to IVM tetravolume via S3.
quadmath.inference.information	action_update	function	(action, free_energy_fn, step_size, epsilon)	Continuous-time action update: $da/dt = -dF/da$.
quadmath.inference.information	active_inference_step	function	(mu, action, free_energy_fn, derivative_operator, step_size, epsilon)	Joint perception-action update step in Active Inference.
quadmath.inference.information	expected_free_energy	function	(log_p_o_given_s, q, p, log_p_o)	Expected free energy for Active Inference with prior preferences.
quadmath.inference.information	finite_difference_gradient	function	(function, x, epsilon)	Compute numerical gradient of a scalar function via central differences.
quadmath.inference.information	fisher_information_matrix	function	(gradients, normalize)	Estimate the Fisher information matrix via sample gradients.
quadmath.inference.information	fisher_information_quadray	function	(gradients, embedding_matrix)	Compute Fisher information matrix in both Cartesian and Quadray coordinates.
quadmath.inference.information	free_energy	function	(log_p_o_given_s, q, p)	Variational free energy for discrete latent states.
quadmath.inference.information	information_gain	function	(prior, posterior, eps)	Information gain (Bayesian surprise) between prior and posterior.
quadmath.inference.information	information_geometric_distance	function	(F, x1, x2)	Compute information-geometric distance between two points.

Module	Symbol	Kind	Signature	Summary
quadmath.inference.information	mutual_information	function	(p_joint, eps)	Mutual information $I(X; Y)$ from a joint probability matrix.
quadmath.inference.information	natural_gradient_step	function	(gradient, fisher, step_size, ridge)	Compute a natural gradient step using a damped inverse Fisher.
quadmath.inference.information	perception_update	function	(mu, derivative_operator, free_energy_fn, step_size, epsilon)	Continuous-time perception update: $d\mu/dt = D \mu - dF/d\mu$.
quadmath.lattice.conversions	embedding_basis	function	(M)	Return the four embedding COLUMNS as a (4,3) basis matrix (Fuller.4D axes).
quadmath.lattice.conversions	quadray_roundtrip	function	(q, M)	Assert the exact round-trip identity $\text{recon} == q$ for $q \rightarrow \text{XYZ} \rightarrow \text{quadray}$.
quadmath.lattice.conversions	quadray_to_xyz	function	(q, M)	Map a Quadray to Cartesian XYZ via a 3x4 embedding matrix (Fuller.4D $\rightarrow$ Coxeter.4D slice).
quadmath.lattice.conversions	urner_embedding	function	(scale)	Return a 3x4 Urner-style symmetric embedding matrix (Fuller.4D $\rightarrow$ Coxeter.4D slice).
quadmath.lattice.conversions	xyz_to_quadray_canonical	function	(xyz, M)	Recover the canonical integer quadray for an XYZ point in the embedding image, EXACTLY.
quadmath.lattice.ivm_dynamics	DynamicsParams	class	Parameters of a discrete IVM lattice dynamics run. `quadmath.lattice.ivm_dynamics` `FitResult` class	Outcome of gradient-free coupling identification.
quadmath.lattice.ivm_dynamics	IVMLattice	class	Finite IVM lattice ball with neighbor adjacency and diffusion operators. `quadmath.lattice.ivm_dynamics` `Trajectory` class	Deterministic simulation record.

Module	Symbol	Kind	Signature	Summary
quadmath.lattice. ivm_dynamics	ball_sites	function	(radius)	Enumerate canonical quadray sites with squared IVM radius $\leq$ radius 2 .
quadmath.lattice. ivm_dynamics	fit_trajectory	function	(observed, grid, lattice, kind, refine_rounds)	Identify the coupling alpha from an observed trajectory (gradient-free).
quadmath.lattice. ivm_dynamics	heat_step	function	(u, lattice, alpha)	One heat-diffusion step $u \leftarrow (1 - \alpha) u + \alpha S u$.
quadmath.lattice. ivm_dynamics	is_nonincreasing	function	(values, tol)	True iff no consecutive pair of values increases by more than tol.
quadmath.lattice. ivm_dynamics	majority_step	function	(u, lattice, alpha)	One rounded-averaging (“majority”) step on integer states.
quadmath.lattice. ivm_dynamics	make_lattice	function	(radius)	Build the finite IVM lattice ball of the given radius.
quadmath.lattice. ivm_dynamics	neighbor_shifts	function	()	Return the 12 canonical IVM neighbor shifts as quadray deltas.
quadmath.lattice. ivm_dynamics	render_dynamics_demo	function	(output_path, radius=, seed=, alpha=, horizon=, fit_alpha=, grid_count=, refine_rounds=)	Render the multi-snapshot IVM dynamics demo figure; return its path.
quadmath.lattice. ivm_dynamics	simulate	function	(T, params, lattice, u0)	Simulate T updates; deterministic given params (fixed seed).
quadmath.lattice. ivm_dynamics	site_radius_sq	function	(q)	Integer squared IVM radius of a quadray site (squared embedding norm).
quadmath.lattice. ivm_dynamics	step	function	(u, lattice, params)	Apply one update of the dynamics in params to the field.
quadmath.lattice. ivm_dynamics	sum_of_squares	function	(u)	Sum of squares of a field — the observable of the heat lemma.

Module	Symbol	Kind	Signature	Summary
quadmath.lattice. ivm_field	IVMField	class	Scalar field over an IVM lattice ball, stored on a deterministic site index. `quadmath. lattice.ivm_field` ` IVM_NEIGHBOR_STEPS` constant	

Module	Symbol	Kind	Signature	Summary
quadmath.lattice. ivm_field	TetrahedronFit	class	Result of :func:` fit_geometry`. ` quadmath.lattice. ivm_field` ` fit_geometry` function `(points, labels, embedding)` Least-squares recovery of a tetrahedron's orientation+scale from noisy 3D points. `quadmath.lattice. ivm_field` ` is_ivm_site` function `(q)` Return True iff q (after normalization) is an IVM lattice site . `quadmath. lattice.ivm_field` ` quadray_shell_norm` function `(q)` Return the IVM shell norm of q : an even integer equal to 2k . `quadmath.lattice. ivm_field` ` shell_ball_sites` function `(radius)` Enumerate the IVM lattice ball of the given shell radius, deterministically. `quadmath.lattice. ivm_field` ` shell_cardinalities` function `(max_shell)` Cardinalities of shells 0 .. max_shell (cuboctahedral numbers) . `quadmath. lattice.ivm_field` ` shell_sites` function `(k)` Enumerate all IVM lattice sites with quadray shell norm 2k . `quadmath.lattice .lattice_search` ` nearest` function `(site, R, k)` Return the k nearest IVM lattice sites within radius R . ` quadmath.lattice. lattice_search` ` squared_distance`	

Module	Symbol	Kind	Signature	Summary
quadmath.lattice. omni_numbering	NEIGHBOR_MOVES	constant	<pre> `quadmath. lattice.omni_numbering ` `clear_shell_cache ` function `( )` Drop memoized shell enumerations so the next call recomputes them. `quadmath. lattice.omni_numbering ` `cumulative_count` function `(k)` Return the total number of IVM sites through shellk(inclusive). ` quadmath.lattice. omni_numbering` ` generate_shell` function `(k)` Generate the sites of shellkof the omnidirectional close packing. `quadmath .lattice. omni_numbering` ` shell_count` function `(k)` Return the number of IVM sites on shellkof the close packing. `quadmath.lattice. omni_numbering` ` site_at_index` function `(index, max_shell)` Return the IVM site at a canonical global index . `quadmath. lattice.omni_numbering ` `site_index` function `(site, max_shell)` Return the canonical global index of an IVM site, or -1 if absent. ` quadmath.lattice. omni_numbering` ` sites_through_shell` function `( max_shell)` Return all IVM sites through shellmax_shellin canonical order. ` quadmath.learn. learning_eval` ` CrossValidationResult` class </pre>	K-fold cross-validation table for the IVM field learner.

Module	Symbol	Kind	Signature	Summary
quadmath.learn. learning_eval	GradientDescentTrainer	class	Full-batch gradient-descent fit of a linear model on standardized features. `quadmath.learn.learning_eval` `LearningCurveResult` class	Data-coverage curve for the IVM field learner.
quadmath.learn. learning_eval	RidgeSiteFit	class	Closed-form ridge fit of a single linear site model. `quadmath.learn.learning_eval` `TrajectorySplitResult` class	Outcome of a temporal train/test evaluation of dynamics identification.
quadmath.learn. learning_eval	cross_validate_field	function	(field_values, sites, k, lam_grid, seed, radius=, kernel_width=)	Cross-validate the Laplacian-regularized field learner over k folds.
quadmath.learn. learning_eval	enclosing_radius	function	(sites)	Return the smallest IVM ball radius containing every given site.
quadmath.learn. learning_eval	kfold_site_splits	function	(observed_sites, k, seed)	Partition observed lattice sites into k seeded folds.
quadmath.learn. learning_eval	learning_curve	function	(values, sites, train_fracs, seed, radius=, lam=, kernel_width=)	Trace held-out MSE as a function of the fraction of observed sites.
quadmath.learn. learning_eval	ridge_site_fit	function	(features, values, lam)	Fit a closed-form ridge regression with intercept on tabular rows.
quadmath.learn. learning_eval	three_way_split	function	(n, train_frac, val_frac, seed)	Split range(n) into seeded, disjoint train/val/test index lists.
quadmath.learn. learning_eval	trajectory_train_test	function	(observed, split_frac, lattice, kind=, grid=, refine_rounds=)	Identify dynamics parameters on a training prefix, score the held-out suffix.

Module	Symbol	Kind	Signature	Summary
quadmath.optimize. discrete_variational	DiscretePath	class	Optimization trajectory on the integer quadray lattice. `quadmath .optimize. discrete_variational` `apply_move` function `(q, delta) ` Apply a lattice move and normalize to the canonical representative. ` quadmath.optimize. discrete_variational` ` discrete_ivm_descent` function `( objective, start, moves=, max_iter=, on_step=)` Greedy discrete descent over the quadray integer lattice. `quadmath .optimize. discrete_variational` `neighbor_moves_ivm` function `()`` Return the 12 canonical IVM neighbor moves as Quadray deltas. `quadmath. optimize. nelder_mead_quadray` `SimplexState` class (vertices, exclude_idx)	
quadmath.optimize. nelder_mead_quadray	centroid_excluding	function	(vertices, exclude_idx)	Integer centroid of three vertices, excluding the specified index.
quadmath.optimize. nelder_mead_quadray	compute_volume	function	(vertices)	Exact IVM tetra-volume (a Fraction, absolute determinant divided by 4) of the first four vertices.
quadmath.optimize. nelder_mead_quadray	nelder_mead_quadray	function	(f, initial_vertices, alpha, gamma, rho, sigma, max_iter, tol, on_step)	Nelder-Mead on the integer quadray lattice.
quadmath.optimize. nelder_mead_quadray	order_simplex	function	(vertices, f)	Sort vertices by objective value ascending and return paired lists.

Module	Symbol	Kind	Signature	Summary
quadmath.optimize. nelder_mead_quadray	project_to_lattice	function	(q)	Project a quadray to the canonical lattice representative via normalize.
quadmath.paths	get_data_dir	function	()	Return quadmath/output/data path and ensure it exists.
quadmath.paths	get_figure_dir	function	()	Return quadmath/output/figures path and ensure it exists.
quadmath.paths	get_output_dir	function	()	Return quadmath/output path at the repo root and ensure it exists.
quadmath.paths	get_repo_root	function	(start)	Heuristically find repository root by walking up from start.
quadmath.pipeline	FieldLearner	class	Laplacian-regularized field learner over an IVM ball; satisfies :class:`Fittable`. `quadmath.pipeline` `FieldModel` class	Anything that predicts a scalar field value at a lattice site.
quadmath.pipeline	Fittable	class	Anything that can fit a field model from data, then predict. `quadmath.pipeline` `LatticeBall` class	Immutable view of an IVM lattice ball; satisfies :class:LatticeSource.
quadmath.pipeline	LatticeSource	class	Anything that can enumerate IVM lattice sites (structural). `quadmath.pipeline` `Pipeline` class	Immutable sequence of :class:Step objects; monoid-style composition.

Module	Symbol	Kind	Signature	Summary
quadmath.pipeline	Step	class	A named, typed callable unit of a : class: `Pipeline`. `quadmath.pipeline` `dynamics_step` function `(params, T )` Step simulating the discrete IVM dynamics in params (ignores input). ` quadmath.pipeline` ` learn_step` function `(lam, seed, radius )` Step fitting an : class: `IVMField` by Laplacian-regularized learning. ` quadmath.pipeline` ` sites_step` function `(radius)` Step producing the IVM ball sites of shell radius (ignores input) . `quadmath.stats. benchmarks` ` BENCH_DEFAULTS` constant	

Module	Symbol	Kind	Signature	Summary
quadmath.stats. benchmarks	BenchRow	class	One timed benchmark result. `quadmath. stats.benchmarks` ` bench_conversions` ` function   `(n, trials)`   Benchmark quadray /XYZ conversions over Ndeterministic samples.     `quadmath .stats.benchmarks`   ` bench_field_fit`   ` function `(n_sites, trials)` Benchmark IVMField.learnon a synthetic field with a fixed seed. ` quadmath.stats. benchmarks` ` bench_lattice_search` function `(n_sites , queries, trials)` Benchmark nearest-site queries through the lattice_searchball index. `quadmath. stats.benchmarks` ` bench_shell_enumeration ` function `(k_max , trials)` Benchmark shell enumeration through shellk_max. `quadmath.stats. benchmarks` `run_all ` function `()`` Run every benchmark with the module-level :data:`BENCH_DEFAULTS `. `quadmath.stats .benchmarks` ` summary_table` function `(rows)` Render aligned fixed- width ASCII rows as a table. `quadmath. stats.benchmarks` ` time_callable` function `(fn, trials=, warmup=)` Timefnwith time.perf_counterand return per-trial wall seconds. ` quadmath.stats. statistics` ` benjamini_hochberg` function `(pvals)` Benjamini-Hochberg FDR-adjusted p-values (step-up procedure).	

Module	Symbol	Kind	Signature	Summary
quadmath.tools. glossary_gen	build_api_index	function	(src_dir)	
quadmath.tools. glossary_gen	generate_markdown_table	function	(entries)	
quadmath.tools. glossary_gen	inject_between_markers	function	(markdown_text, begin, end, payload)	
quadmath.validate. validate	DEFAULT_CHECKS	constant	`quadmath. validate.validate` ` DEFAULT_TOLERANCE` constant	
quadmath.validate. validate	NOTES	constant	`quadmath. validate.validate` ` SKIPPED_CHECKS` constant	

Module	Symbol	Kind	Signature	Summary
quadmath.validate. validate	ValidationReport	class	Immutable outcome of a single validation check. `quadmath. validate.validate` ` check_associativity` ` function   `(a, b, c, tol)`   Verify $(a*b)*$ $c == a*(b*c)$ within <b>tol</b> via the core Hamilton product.     `quadmath.validate. validate`   ` check_conjugate_inverse `   function   `(q, tol)`   Verify $q *$ $\text{conj}(q)$ equals the identity $(1, 0, 0, 0)$ within <b>tol</b> .     ` quadmath.validate. validate`   ` check_double_cover`   function   `(q1, q2, tol)`   Verify the $S^0$ $(3)$ homomorphism $R(q1*$ $q2) == R(q1) R(q2)$ within <b>tol</b> .     ` quadmath.validate. validate`   ` check_normalization`   function   `(q, tol)`   Verify the quaternion norm is within <b>tol</b> of 1 (norm of $(a, b, c, d)$ ).     ` quadmath.validate. validate`   ` check_slerp_midpoint`   function   `(q0, q1, tol)`   Verify the shortest-arc slerp midpoint lies on the geodesic of $(q0, q1)$ .     `quadmath.validate .validate`   ` run_validation`   function   ` quaternions, checks)`   Run checks over <b>quaternions</b> and collect deterministic reports.     `quadmath .viz._common`   ` atomic_target`   function   `(path)`   Yield a temporary sibling of <b>path</b> ; move it onto <b>path</b> only on success.     `quadmath .viz._common`   ` 	A single animation frame.

Module	Symbol	Kind	Signature	Summary
quadmath.viz. animations	GRID_SIZE	constant	`quadmath.viz. animations` ` diffusion_frames` function `(n_steps, seed)` Render explicit heat diffusion on the IVM radius-3 ball adjacency. ` quadmath.viz. animations` ` frames_strip` function `(frames, out_path, labels, save )` Render frames as a single-row matplotlib strip, one panel per frame. ` quadmath.viz. animations` ` frames_to_gif` function `(frames, out_path, fps, scale)` Assemble frames into an animated GIF at <u>out_path</u> . ` quadmath.viz. animations` ` lattice_frames` function `(shells, n )` Render a pulsing IVM lattice ball. ` `quadmath.viz. animations`   ` simplex_frames`   function   `(q0, q1, n)`   Render a quaternion-slerp rotation of the IVM radius-1 ball.     ` quadmath.viz.plots`   `plot_error_histogram`   function   `(errors , bins, save, out_path)`   Plot a histogram of error values with a dashed vertical mean line.     `quadmath. viz.plots`   ` plot_lattice_shell_3d`   function   `(k, save, out_path)`   Scatter the sites of one IVM lattice shell in 3D (equal-aspect axes).     `quadmath. viz.plots`   ` plot_loss_history`   function   `(losses,	

Module	Symbol	Kind	Signature	Summary
quadmath.viz. vis_lattice	GALLERY_FILES	constant	`quadmath.viz. vis_lattice` ` dynamics_strip` ` function   `(axs, trajectory, t_indices, embedding=, cmap=, titles=)`   Render evolution snapshots of a trajectory as a strip of 3D panels.     `quadmath.viz. vis_lattice`   ` field_slice`   ` function `(ax, field , sites, plane, q0=, cmap=, title=, colorbar=)` Heatmap of a scalar IVM field restricted to a lattice plane. ` quadmath.viz. vis_lattice` ` gallery` function `(paths_out_dir, seed) ` Compose the three lattice-gallery figures deterministically. `quadmath.viz. vis_lattice` ` shell_scatter` ` function   `(ax, sites , k, embedding=, color =, size=, axis_hints=, title=)`   Scatter one IVM frequency shell in 3D with tetrahedral axis hints .     `quadmath.viz. vis_stats`   ` GALLERY_FILES` constant (paths_out_dir, seed)	
quadmath.viz.vis_stats	gallery	function		Compose the four statistics-gallery figures deterministically.
quadmath.viz.vis_stats	plot_ciBars	function	(labels, means, lows, highs, ax, title=)	Point estimates with confidence intervals [lows, highs] as error bars.

Module	Symbol	Kind	Signature	Summary
quadmath.viz.vis_stats	plot_ecdf	function	(values, ax, title=)	Empirical cumulative distribution function as a sorted step plot.
quadmath.viz.vis_stats	plot_latency_hist	function	(times, ax, bins=, title=)	Histogram of a latency sample with a dashed vertical mean line.
quadmath.viz.vis_stats	plot_scaling_loglog	function	(sizes, times, ax, title=)	Log-log scatter of times versus sizes with the fitted power law.
quadmath.viz.visualize	animate_discrete_path	function	(path, embedding, save, out_path)	Animate a point moving along a discrete quadray path.
quadmath.viz.visualize	animate_simplex	function	(vertices_list, embedding, save, out_path)	Animate simplex evolution across iterations.
quadmath.viz.visualize	plot_ivm_neighbors	function	(embedding, save, out_path)	Scatter the 12 IVM neighbor points in 3D.
quadmath.viz.visualize	plot_partition_tetrahedron	function	(mu, s, a, psi, embedding, save, out_path)	Plot the four-fold partition as a labeled tetrahedron in 3D.
quadmath.viz.visualize	plot_simplex_trace	function	(state, save, out_path)	Plot per-iteration diagnostics for Nelder-Mead.

11 Static IVM Field Learning

11.1 Overview

This section develops machine learning on a static synergetic geometry: scalar fields defined over the Isotropic Vector Matrix (IVM) lattice, learned from sparse noisy observations by Laplacian-regularized kernel-weighted least squares on the lattice graph. All methods live in the `ivm_field.py` module (`quadray.IVMField`, `quadray_shell_norm`, `shell_sites`, `fit_geometry`) and reuse the quadray machinery of `quadray.py` described in [Quadray Methods](#). The learner uses `numpy` only — no machine-learning frameworks — and is fully deterministic for a fixed observation set.

11.2 The IVM Lattice in Quadray Coordinates

Quadray coordinates represent IVM close-packed sphere centers as non-negative integer quadrays normalized so the minimum component is zero (see [Quadray Methods](#)). Two facts organize the lattice. First, normalization adds or subtracts (k, k, k, k) , so the component sum is a projective invariant modulo 4. Second, the IVM sites are exactly one of the four residue classes:

$$q \text{ is an IVM site} \iff \sum_i q_i \equiv 0 \pmod{4}, \quad (14)$$

implemented as `is_ivm_site()` in `ivm_field.py`. The remaining three cosets are the octahedral ($\text{sum} \equiv 2$) and tetrahedral ($\text{sum} \equiv 1, 3$) voids of the packing — lattice points of the ambient grid, but not sphere centers.

The shell norm of a site is the L^1 magnitude of its sum-zero (Coxeter 4D hyperplane) representative. With $s = \sum_i q_i$:

$$N(q) = \sum_{i=1}^4 \left| q_i - \frac{s}{4} \right| = 2k, \quad k \in \mathbb{Z}_{\geq 0}, \quad (15)$$

computed by `quadray_shell_norm()`. Shell k of the lattice is the set of sites with $N(q) = 2k$; `shell_sites(k)` enumerates it by scanning the bounding box $[0, 2k]^4$, filtering on the membership condition (14) and the norm (15), and sorting lexicographically — a deterministic site order. The shell cardinalities are the cuboctahedral numbers:

$$|\{q : N(q) = 2k\}| = 10k^2 + 2, \quad k \geq 1, \quad (16)$$

giving the sequence 1, 12, 42, 92, 162, ... for $k = 0, 1, 2, 3, 4$ — the center plus the cuboctahedral numbers, with shell 1 the twelve-around-one vector equilibrium of [Quadray Methods](#). `shell_cardinalities()` counts shells of an enumerated ball, and the test suite pins the sequence exactly.

11.3 The Field Model

`IVMField.lattice_ball(radius)` stores a scalar field over the lattice ball $B_R = \{q : N(q) \leq 2R\}$ as a one-dimensional `numpy` array keyed by the deterministic site index of `shell_ball_sites()` (shell-major, lexicographic within shell), with a dictionary mapping each normalized site to its array position. Two lattice-graph ingredients drive learning. The adjacency structure uses the twelve IVM neighbor moves — the permutations of $(2, 1, 1, 0)$ collected in `IVM_NEIGHBOR_STEPS` — so two sites in the ball are graph-adjacent exactly when one step of close packing separates them. The graph Laplacian L (method `IVMField._laplacian()`) is the symmetric difference operator on that adjacency:

$$(Lf)_i = \deg(i) f_i - \sum_{j \sim i} f_j. \quad (17)$$

11.4 Learning: Laplacian-Regularized Kernel-Weighted Least Squares

Observations y_j arrive at a sampled subset Ω of sites. Graph distances $d(\cdot, \cdot)$ are hop counts from a multi-source BFS (`_multi_source_distances()` in `ivm_field.py`), and the Gaussian kernel over hops with width τ (`kernel_width`) is:

$$K(d) = \exp(-(d/\tau)^2), \quad d(i, j) = \text{hop distance.} \quad (18)$$

`IVMField.learn()` minimizes a two-regime objective over the ball, with Laplacian regularization weight $\lambda \geq 0$:

$$\min_f \sum_{i \in \Omega} (f_i - y_i)^2 + \sum_{i \notin \Omega} c_i (f_i - t_i)^2 + \lambda \sum_{i \sim j} (f_i - f_j)^2, \quad (19)$$

with three data-fidelity terms: observed sites are pinned to their data with unit weight (no self-smoothing of real observations); unobserved sites carry a kernel confidence weight and a Nadaraya-Watson kernel target,

$$c_i = \max(K(\min_{j \in \Omega} d(i, j)), \varepsilon), \quad t_i = \frac{\sum_{j \in \Omega} K(d(i, j)) y_j}{\sum_{j \in \Omega} K(d(i, j))}, \quad (20)$$

where ε — the `_WEIGHT_FLOOR` constant, 10^{-12} , in `ivm_field.py` — is a numerical floor keeping the normal-equation matrix of (19) positive definite on the connected lattice ball. The normal equations of (19) are

$$(C + \lambda L)f = Ct, \quad (21)$$

with $C = \text{diag}(c_i)$ and t the target vector; `IVMField.learn()` solves them with a dense `numpy.linalg.solve`. The Laplacian term propagates field structure from observed sites across the lattice graph, and the confidence weighting degrades gracefully to pure Laplacian propagation as observation density falls. Because observed rows carry unit weight, the estimator interpolates exact data as $\lambda \rightarrow 0$ and denoises noisy data for moderate λ — both behaviors are verified numerically in `tests/test_ivm_field.py` with fixed seeds.

Two structural facts justify the estimator. A linear field in the embedded XYZ coordinates (the `to_xyz()` image of `quadray.py`) is harmonic on the IVM graph — the twelve neighbor moves sum to zero — so the harmonic extension implied by (21) reproduces it exactly from boundary data (asserted to 10^{-5} in the test suite). And for noisy smooth fields, the learned field tracks the kernel-weighted local mean, which averages observation noise across graph neighborhoods:

$$\text{MSE}(f) = \frac{1}{|B_R|} \sum_{i \in B_R} (f_i - f_i^{\text{truth}})^2, \quad (22)$$

the quantity returned by `IVMField.score()`.

11.5 Recovering Tetrahedral Geometry from Noisy Points

`fit_geometry()` recovers the orientation and scale of a tetrahedron from noisy 3D point clouds, using the quadray basis of `quadray.py`. The four canonical vertex images are $v_i = \text{to_xyz}(e_i)$, where e_i are the unit quadray axes mapped through the 3×4 embedding matrix of `quadray.py`. Given labeled observations $p_m^{(i)} \approx R v_i + o$ (noisy samples of vertex i under rotation R and offset o), vertex centroids c_i are formed and the linear map G is fit in closed form:

$$G = \arg \min_{M \in \mathbb{R}^{3 \times 3}} \sum_{i=1}^4 \|M v_i - c_i\|^2, \quad \text{scale} = \text{sign}(\det G) |\det G|^{1/3}. \quad (23)$$

The least-squares solve uses `numpy.linalg.lstsq` on the transpose system, so noisy multi-sample vertices average out; the residual reported by `TetrahedronFit.residual` is the RMS vertex-centroid mismatch. The test suite recovers a rotated, scaled tetrahedron to 10^{-6} from $\sigma = 10^{-7}$ noise, verifies the signed scale for reflected (negative determinant) fits, and checks that doubling the embedding halves the recovered matrix — orientation and scale separate cleanly.

11.6 Demonstration

The script `quadmath/scripts/ivm_field_demo.py` (run with `MPLBACKEND=Agg`, seed 12) renders the pipeline on the radius-3 ball (147 lattice sites): the synthetic field (a harmonic linear part plus a weak quadratic bowl), the noisy observations at 74 of the 147 sites, and the learned field. The learned reconstruction beats the raw observation noise (reconstruction MSE 0.0558 against observation-noise MSE 0.0720, printed by the demo):

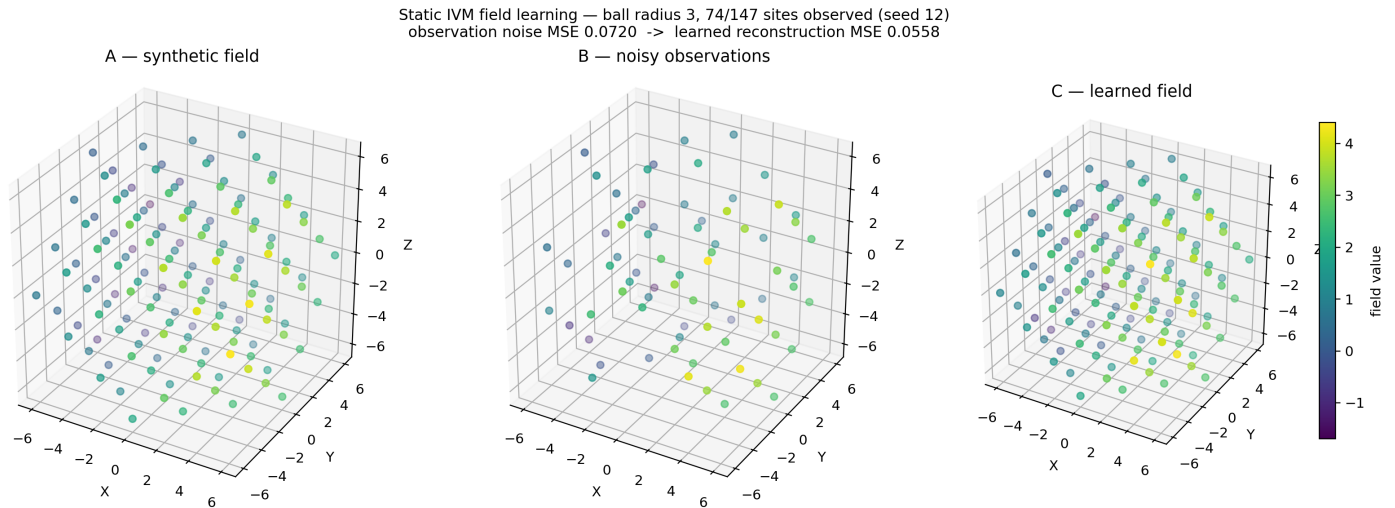


Figure 18: Static IVM field learning on the radius-3 IVM lattice ball (147 sites, shown at their XYZ embedding from the quadray coordinates; axes in embedding units). Panel A: the synthetic ground-truth field — a harmonic linear part plus a weak quadratic bowl — with dot color giving field value on the shared color bar (right). Panel B: the noisy observations at 74 of the 147 sites (seed 12). Panel C: the learned field from Laplacian-regularized kernel-weighted least squares (Eqs. (19)–(21)); reconstruction MSE 0.0558 against observation-noise MSE 0.0720. Reproduce with `quadmath/scripts/ivm_field_demo.py` (`MPLBACKEND=Agg`, seed 12).

11.7 Cross-References

- Coordinate foundations and the twelve-around-one shell: [Quadray Methods](#)
- Optimization methods on tetrahedral lattices: [Optimization in 4D](#)
- Applications and generalizations: [Extensions](#)

12 Dynamics and Learning on the IVM Lattice

12.1 Overview

Where [Section 4](#) descends a *single point* along the IVM lattice, this section evolves a *field* — one scalar per lattice site — and then asks the inverse question: given an observed field trajectory, what coupling produced it? Everything here is implemented in `ivm_dynamics.py` (`ball_sites`, `make_lattice`, `heat_step`, `majority_step`, `step`, `simulate`, `fit_trajectory`, `sum_of_squares`, `is_nonincreasing`) on top of the `Quadray` class and `to_xyz` embedding in `quadray.py`; the demo figure is produced by `quadmath/scripts/ivm_dynamics_demo.py` (which delegates to `render_dynamics_demo` in `src/quadmath/lattice/ivm_dynamics.py`, per the thin-orchestrator contract in `quadmath/scripts/AGENTS.md`).

12.2 Lattice sites, shells, and the 12-around-one move graph

A *site* is a canonical quadray representative — non-negative integer components with at least one zero, selected by `Quadray.normalize` (see [Quadray Methods](#)). The radial observable is the squared embedding norm `site_radius_sq(q)`; because the shift vector $(1, 1, 1, 1)$ lies in the kernel of the `DEFAULT_EMBEDDING` matrix, the radius is invariant under quadray normalization. `ball_sites(R)` enumerates the canonical sites with squared radius at most R^2 : for a canonical site the squared radius is at least twice the square of its largest component, so components in $[0, R]$ cover the ball.

`make_lattice(R)` joins two sites when one of the 12 canonical IVM neighbor shifts (`neighbor_shifts`, all permutations of $(2, 1, 1, 0)$, each at squared radius 8 — the close-packing distance) maps one site onto the other after renormalization. Normalization never moves the embedded point, so adjacency is exactly “embedding difference is one of the 12 shifts”. Lattice sites are the quadrays whose coordinate sum is divisible by 4; the remaining residue classes are IVM voids and are excluded. On the $R = 3$ ball (13 sites) the graph is a single connected component: the close-packed **12-around-one cluster**, i.e. the origin (degree 12) and its twelve cuboctahedron neighbors (degree 5). The update laws below act independently on each connected component of the ball.

12.3 Heat update and the monotonicity lemma

The heat (diffusion) update blends a field $u_t \in \mathbb{R}^N$ — one value per lattice site, N the site count — with its symmetric-normalized adjacency average $S = D^{-1/2}AD^{-1/2}$, where A is the adjacency matrix of the move graph, D the diagonal degree matrix, and rows of S are zero at isolated sites:

$$u_{t+1} = (1 - \alpha)u_t + \alpha S u_t, \quad \alpha \in [0, 1]. \quad (24)$$

Lemma (heat averaging is non-increasing in sum of squares). For every $\alpha \in [0, 1]$, every lattice built by `make_lattice`, and every state,

$$\|u_{t+1}\|_2^2 \leq \|u_t\|_2^2. \quad (25)$$

Proof sketch. S is symmetric with spectrum in $[-1, 1]$ (it is similar to the random-walk matrix $D^{-1}A$). The update operator $(1 - \alpha)I + \alpha S$ is therefore symmetric with eigenvalues in $[1 - 2\alpha, 1] \subseteq [-1, 1]$, hence non-expansive in the 2-norm. $\square$

This is exactly the claim the test suite asserts — *per step*, across seeds and the full coupling range (`test_heat_update_l2_nonincreasing_per_step` in `tests/unit/lattice/test_ivm_dynamics.py`). Two scope restrictions are deliberate and tested:

1. The guarantee is specific to the symmetric normalization S . Plain row-average diffusion $P = D^{-1}A$ on a truncated ball is **not** covered (boundary rows make P non-doubly-stochastic); the module neither uses nor claims it.
2. In the demo run ($\alpha = 0.35$, $T = 40$, seed 7) the sum of squares decreases monotonically from 37.04 to 3.93 while never increasing at any single step.

12.4 Majority update on integer states

For integer-valued fields the rounded-averaging (“majority”) update is

$$u_{t+1}(s) = \text{rint}\left((1 - \alpha) u_t(s) + \alpha \frac{1}{\deg(s)} \sum_{n \in N(s)} u_t(n)\right), \quad (26)$$

with $N(s)$ the neighbor set of site s and $\deg(s) = |N(s)|$ its degree, identity rows at isolated sites, and round-half-to-even ties (`numpy rint`), computed by `majority_step`. Every new value is a rounded convex combination of the site and its neighbors, so the extremal structure is preserved:

Proposition (range containment). Under (26) the field maximum is non-increasing and the minimum non-decreasing, at every step, for every $\alpha \in [0, 1]$.

We deliberately make **no** sum-of-squares claim for this update: rounding can increase it, and the test suite pins a demonstrating case (origin at 0 with its twelve packers at 1 maps to thirteen sites at value 1, raising the sum of squares from 12 to 13 — `test_majority_step_sum_of_squares_can_increase`). The honest summary: heat is L_2 -monotone, majority is range-preserving, and neither claim is stretched to cover the other update.

12.5 Learning the coupling from an observed trajectory

Given observed snapshots $u_{\text{obs}}(0..T)$ on a known lattice, the coupling α is identified gradient-free: `fit_trajectory` re-simulates the field from the observed initial condition for each candidate coupling and minimizes the trajectory mean squared error over the T post-initial snapshots and the N sites, where $u_\alpha(t)$ is the re-simulated field under candidate coupling α :

$$\mathcal{L}(\alpha) = \frac{1}{TN} \sum_{t=1}^T \|u_\alpha(t) - u_{\text{obs}}(t)\|_2^2, \quad (27)$$

scanning the supplied grid, then re-gridding the interval between the best candidate’s grid neighbors `refine_rounds` times (grid/coordinate search; ties resolve to the smallest α ; fully deterministic). The result is a global optimum *over the evaluated candidates only* — no convergence beyond the refinement resolution is claimed. In the demo configuration (synthetic trajectory generated at $\alpha_{\text{true}} = 0.3$ on the $R = 3$ lattice, $T = 40$), a 21-point grid plus refinement (33 MSE evaluations) recovers $\alpha \approx 0.300$ with $\mathcal{L} \approx 4.5 \times 10^{-33}$ — machine precision for a bitwise-deterministic re-simulation. When the true coupling lies off-grid, refinement converges to within a grid-spacing fraction of it (`test_fit_refinement_finds_offgrid_coupling`: 0.37 recovered to ≤ 0.0125 from a coarse $\{0, 0.25, 0.5, 0.75, 1\}$ grid).

12.6 Demo figure

Generated by `uv run python quadmath/scripts/ivm_dynamics_demo.py (MPLBACKEND=Agg)`; all randomness is seeded (`DynamicsParams.seed`), so the figure is reproducible byte-for-byte up to PNG encoding.

12.7 Reproducibility and test contract

- **Tests:** `PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 uv run coverage run -m pytest tests/unit/lattice/test_ivm_dynamics.py -q` then `uv run coverage report` — 42 tests, 100% statement and branch coverage of `src/quadmath/lattice/ivm_dynamics.py`, no mocks, all examples real numerics with fixed seeds.
- **Markdown:** `uv run python quadmath/scripts/validate_markdown.py`.
- The two provable claims (lemma (25), the range proposition under (26)) are asserted per step; the non-claims (row-average diffusion, majority L_2 behavior) are pinned by the honesty tests described above rather than left implicit.

IVM lattice dynamics: heat diffusion, majority rounding, coupling learning

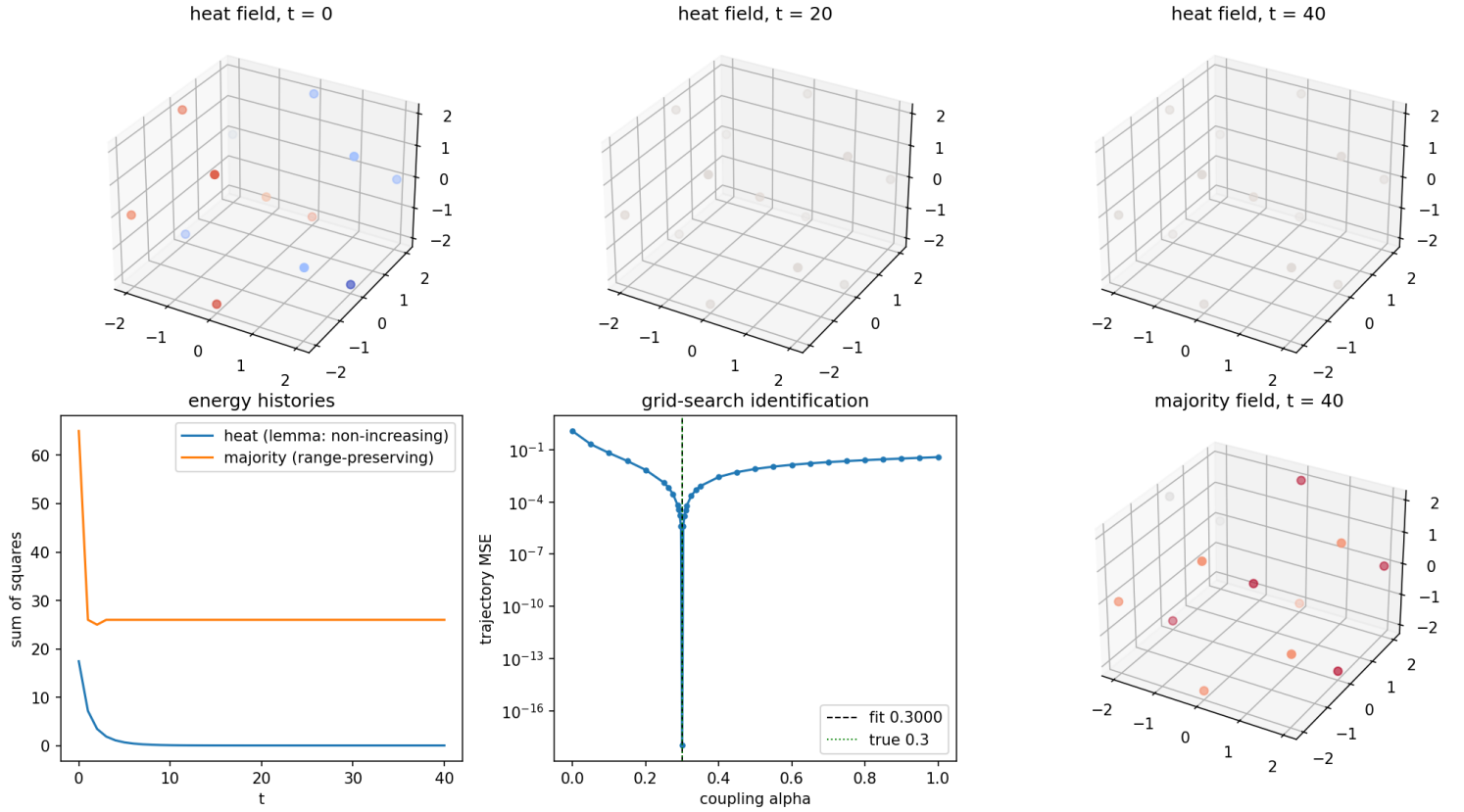


Figure 19: **Dynamics and learning on the IVM lattice ($R = 3$ ball, 13 sites).** Top row: heat-field snapshots at $t = 0, 20, 40$ ($\alpha = 0.35$, seed 7) over the embedded quadray sites (XYZ axes in embedding units); dot color encodes field value on a shared symmetric scale, and diffusion homogenizes the field by $t = 40$. Bottom left: sum-of-squares histories over $t = 0 \dots 40$ — the heat curve decreases monotonically (lemma (25)); the majority curve falls steeply and plateaus, but unlike heat it carries no monotonicity guarantee, since rounding can increase the sum of squares (test_majority_step_sum_of_squares_can_increase). Bottom middle: trajectory MSE versus coupling α on a logarithmic axis over the search grid, with the identified optimum (fit 0.3000) coinciding with the true generating $\alpha = 0.3$ (dotted). Bottom right: the final integer majority field at $t = 40$. Generated by `uv run python quadmath/scripts/ivm_dynamics_demo.py (MPLBACKEND=Agg, seeded via DynamicsParams.seed)`.

12.8 Cross-references

- Site geometry and normalization: [Quadray Methods](#).
- Discrete point descent on the same move set: [Optimization in 4D](#).
- Field/energy vocabulary (free energy, Fisher information): [Appendix B](#); equation index: [Appendix A](#).

13 Lattice Tooling: Omnidirectional Numbering and Nearest-Site Search

13.1 Overview

Sections 2 and 3 develop the Quadray/IVM framework analytically; this section documents the computational tooling that operates directly on the close-packed lattice. Two modules extend the analytical core without touching it:

- `omni_numbering` — omnidirectional close-packing numbering: vectorized enumeration of the IVM shell sequence, cumulative counts, and bidirectional site/index mappings.
- `lattice_search` — fast nearest-lattice-point queries: `nearest(site, R, k)` and `within_radius(site, R)` built on a precomputed ball index, entirely in NumPy (no scipy).

Both modules operate on canonical quadray integer 4-tuples (non-negative components, at least one zero — the projective normalization of Section 3), and both rely on the same verified shell invariants.

13.2 Omnidirectional Close Packing and Frequency Shells

In the isotropic vector matrix (IVM), equal spheres pack omnidirectionally — closest packing. Starting from a central sphere, the packing builds up in consecutive **frequency shells**: the shell of frequency $k \in \mathbb{Z}_{\geq 0}$ sits k radius-length increments outward of the center, the same four-dimensional buildup language synergetics uses for the growing vector equilibrium. Writing N_k for the population of shell k , the lone center is the whole of shell 0 ($N_0 = 1$), and every shell with $k \geq 1$ carries exactly

$$N_k = 10k^2 + 2 \quad (28)$$

sphere centers. The first shell — the 12 permutations of $(2, 1, 1, 0)$ — is the cuboctahedron (vector equilibrium) of the “twelve around one” motif; each subsequent shell is the next layer of the omnidirectional buildup. The cumulative count through frequency k , written C_k , is the centered-cuboctahedral closed form

$$C_k = 1 + \sum_{j=1}^k (10j^2 + 2) = 1 + 2k + \frac{10k(k+1)(2k+1)}{6}. \quad (29)$$

The functions `shell_count(k)` and `cumulative_count(k)` evaluate Eqs. (28) and (29) (scalar or array input), and the test suite cross-validates them against direct summation.

13.2.1 Reachability is enumerative, not congruence-based

A normalized integer quadray q with $\sum_i q_i \equiv 0 \pmod{4}$ is a candidate IVM point, but not every candidate is reachable from the origin by the 12 neighbor moves. The module therefore builds the site set **layer by layer**: each new shell is produced at once by adding all 12 moves to the previous frontier, re-normalizing rows, and removing sites already seen (packed-key membership tests). The result is cached per depth: `sites_through_shell(k)` returns the $(C_k, 4)$ integer array in canonical order — the center first, then each shell in lexicographic (a, b, c, d) order — and `generate_shell(k)` returns one shell. An independent breadth-first reference over `itertools.permutations` confirms exact agreement through frequency 8, including the counts of Eq. (28) on every shell.

13.2.2 Site/index mapping

`site_index(site, max_shell)` returns the position of a site in that canonical enumeration (or -1 if absent within the depth), and `site_at_index(index, max_shell)` is its inverse. Lookups accept any projective representative — the input is translated by $-(k, k, k, k)$ before matching — so $(2, 1, 1, 0)$ and $(3, 2, 2, 1)$ map to the same index, while $(0, 1, 1, 2)$, a different permutation and hence a different sphere, maps elsewhere. Component overflow

is guarded: inputs whose normalized components reach the 16-bit packed-key width (2^{16} , the per-component field of the int64 keys) return -1 rather than wrapping.

Executable check of the bookends:

```

1 from quadmath.lattice.omni_numbering import (
2     shell_count, cumulative_count, sites_through_shell,
3     site_index, site_at_index,
4 )
5
6 assert shell_count(1) == 12 and shell_count(2) == 42      # 10k^2 + 2
7 assert cumulative_count(2) == 55                          # 1 + 12 + 42
8 assert site_index(site_at_index(12, 3), 3) == 12          # roundtrip
9 assert site_index((2, 0, 0, 0), 6) == -1                  # unreachable site

```

13.3 Fast Nearest-Site Queries

13.3.1 Exact lattice distances

Under `quadrax.DEFAULT_EMBEDDING` the four embedding columns C_j satisfy $C_i \cdot C_j = 4\delta_{ij} - 1$ at unit scale, so the Euclidean squared distance between integer 4-vectors p, q is the exact integer

$$d^2(p, q) = 4 \sum_j \delta_j^2 - \left(\sum_j \delta_j \right)^2, \quad \delta = p - q, \quad (30)$$

implemented in `squared_distance` and cross-validated in the tests against direct embedding coordinates. Because the identity is exact on integers, ranking and filtering never suffer floating-point boundary errors: a site is inside the query radius $R > 0$ iff $d^2 \leq R^2$ evaluated exactly (in float64 the same algebra is exact for the magnitudes involved here).

13.3.2 Truncation bound

Every site s on frequency shell g satisfies $d^2(\mathbf{0}, s) \geq 8g$ — verified by exhaustive enumeration through frequency 8 in the test suite — and the shell maximum is exactly $8g^2$. Combining this invariant with the triangle inequality gives the depth at which a query can stop:

$$g \leq \frac{(\|c\| + R)^2}{8}, \quad (31)$$

for a query center c and radius $R > 0$, where $\|c\|$ is the Euclidean norm of the embedded center: any site within R of c must live on a shell at most this deep. `within_radius(site, R)` therefore enumerates exactly through $\lfloor (\|c\| + R)^2 / 8 \rfloor + 1$ shells, filters with Eq. (30), and returns all hits sorted by squared distance with lexicographic (a, b, c, d) tie-breaking. Requests whose required depth exceeds the precomputed index depth (`MAX_SHELL = 32`, about 1.1×10^5 sites — $C_{32} = 114465$ by Eq. (29)) raise `ValueError` rather than silently truncating.

13.3.3 Shell-sweep nearest

`nearest(site, R, k)` sweeps shells outward, ranking each shell's squared distances with `numpy.argsort`; once k candidates are in hand, the invariant behind Eq. (31) supplies an early-stop test — no unvisited shell can contain a closer site: a site at depth $g+1$ satisfies $d^2(\mathbf{0}, s) \geq 8(g+1)$, so the sweep stops once $8(g+1) > (\sqrt{d_{(k)}^2} + \|c\|)^2$, with $d_{(k)}^2$ the current k -th best squared distance. The sweep never under-enumerates (the bound is conservative; ties at the boundary are kept), and answers always match the brute-force reference over the same region, as the tests assert on fixed-seed random queries.

```

1 import numpy as np
2 from quadmath.lattice.lattice_search import nearest, within_radius
3
4 sites, d2 = nearest((0.4, -0.3, 0.9, 0.1), R=2.0, k=5) # 5 nearest centers
5 ball, ball_d2 = within_radius((2, 1, 1, 0), R=4.0) # everything within 4
6 assert np.all(np.diff(ball_d2) >= 0) # ascending distances

```

13.4 API Summary

Function	Purpose
<code>omni_numbering.shell_count(k)</code>	Shell population, Eq. (28)
<code>omni_numbering.cumulative_count(k)</code>	Cumulative count through shell k , Eq. (29)
<code>omni_numbering.generate_shell(k)</code>	Sites of one frequency shell (lexicographic)
<code>omni_numbering.sites_through_shell(k)</code>	Whole enumeration ($C_k, 4$) in canonical order
<code>omni_numbering.site_index / site_at_index</code>	Bidirectional site/index mapping with projective normalization
<code>lattice_search.squared_distance</code>	Exact squared distances via Eq. (30)
<code>lattice_search.within_radius(site, R)</code>	All sites within radius, exact and sorted
<code>lattice_search.nearest(site, R, k)</code>	k nearest sites within radius, shell-sweep with early stop

13.5 Verification

Both modules ship with 100% test coverage (branch coverage included): the enumeration is checked against an independent breadth-first reference through frequency 8, distance identities against direct embedding computations, and query results against brute-force filtering on fixed-seed random centers. Run:

```

1 PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 uv run coverage run -m pytest -q
2 uv run coverage report

```

14 Conversions & Specification

14.1 Overview

Sections 2, 3 and 13 establish the Quadray/IVM framework and its computational tooling; this section documents the conversion layer that joins its two coordinate namespaces — `conversions.py`, the module that maps between Fuller.4D quadray coordinates and Coxeter.4D Cartesian coordinates. Three concerns live here: the embedding itself and its projective fiber, the Gram identities that make lattice distances exact, and an **exact rational inversion** that answers “which lattice point is this point?” with `fractions.Fraction` arithmetic instead of floating-point rounding.

The full mathematical specification of this layer is the repository-root file `SPEC.md`; the `tests/test_spec_examples.py` pins its worked examples numerically. This section is the manuscript projection of that specification: same definitions, same error taxonomy, same guarantees, prose instead of source. The analytical foundations it draws on are [Quadray Methods](#) (normalization, the `Quadray` class), the shell machinery of [IVM Field Learning](#), and the nearest-site search of [Lattice Tooling](#), whose exact-distance identity this section re-derives from the embedding’s Gram matrix.

14.2 The embedding (Fuller.4D to Coxeter.4D)

The default embedding maps the four quadray axes (A, B, C, D) to the vertices of a regular tetrahedron in $\mathbb{R}^3$. With scale factor s (uniform; scales all resulting coordinates):

$$M = s \times \begin{pmatrix} 1 & -1 & -1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \end{pmatrix}, \quad \text{xyz}(q) = M q. \quad (32)$$

`urner_embedding(scale=1.0)` builds exactly this matrix (each row scaled by `scale`), and `quadray.DEFAULT_EMBEDDING` is the same unscaled matrix stored as a tuple-of-rows; the two agree entry for entry at scale 1. Every row of M sums to zero:

$$\sum_j M_{ij} = 0 \quad \forall i, \quad \text{hence} \quad q \sim q + t \mathbf{1}, \quad t \in \mathbb{Z}, \quad (33)$$

where $\mathbf{1} = (1, 1, 1, 1)$. The vector $\mathbf{1}$ spans the kernel of M , so the map is projective: `to_xyz(q) == to_xyz(q + t * (1, 1, 1, 1))` for every integer t , and quadray normalization (see [Quadray Methods](#)) simply selects the min-0 representative of that equivalence class. The forward map `quadray_to_xyz(q, M=None)` delegates to `quadray.to_xyz`; passing `M=None` selects `quadray.DEFAULT_EMBEDDING`, and an explicit `M` is shape-validated against the shared internal helper `_embedding_rows` (a $(3, 4)$ array is required, `ValueError` otherwise) before the product is taken. Existing callers that pass an explicit embedding — for example `quadmath/scripts/quadray_clouds.py` — are unaffected.

14.3 Gram identities and exact distances

The embedding matrix satisfies two exact Gram identities. Row-wise, at scale s :

$$M M^T = 4 s^2 I_3, \quad (34)$$

and column-wise, with C_j the four columns of M and J_4 the all-ones matrix:

$$M^T M = 4 I_4 - J_4, \quad \text{i.e.} \quad C_i \cdot C_j = 4 \delta_{ij} - 1 \quad (s = 1). \quad (35)$$

`embedding_basis(M=None)` returns the columns of the validated embedding as a $(4, 3)$ array B whose rows are the C_j ; at scale 1 the identity (35) reads $B B^T = 4 I_4 - J_4$ exactly (diagonal 3, off-diagonal -1), and it scales by

s^2 for `urner_embedding(scale)`. This is the identity that `lattice_search.squared_distance` relies on: for integer quadrays p, q with $\delta = p - q$, the Euclidean squared distance in embedded coordinates is the exact integer

$$d^2(p, q) = \sum_i (M\delta)_i^2 = 4 \sum_j \delta_j^2 - \left(\sum_j \delta_j \right)^2, \quad (36)$$

the same identity documented as Eq. (30) in **Lattice Tooling**, where it powers ranking and filtering; the test suite cross-validates it against float embeddings and `quadray.distance`. Because the identity is exact on integers, nearest-site decisions never suffer floating-point boundary errors.

Executable check of the two identities:

```

1 import numpy as np
2 from quadmath.lattice.conversions import urner_embedding, embedding_basis
3 from quadmath.lattice.lattice_search import squared_distance
4
5 M = urner_embedding()
6 B = embedding_basis() # (4, 3): rows are the columns of M
7 assert np.allclose(M @ M.T, 4.0 * np.eye(3)) # Eq. (eq:conv-gram-row)
8 assert np.allclose(B @ B.T, 4.0 * np.eye(4) - np.ones((4, 4)))
9 delta = np.array([2, 1, 1, 0]) # shell-1 displacement
10 assert squared_distance(delta, np.zeros((1, 4), dtype=np.int64)) == 8 # Eq. (eq:conv-distance)

```

14.4 Exact canonical inversion

The forward map is projective but not injective: by (33) the fiber of an embedded point is the full coset $\{q + t \mathbf{1} : t \in \mathbb{Z}\}$, and an inverse must select one representative from it.

$$y = (x_1, x_2, x_3, 0), \quad (C_0 \ C_1 \ C_2) \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \text{xyz}, \quad q^* = \text{normalize}(y). \quad (37)$$

`xyz_to_quadray_canonical(xyz, M=None)` resolves that ambiguity deterministically by exact rational arithmetic (`Fraction`; a Python float enters as its exact binary rational — no rounding, no tolerance). The algorithm, in prose: (1) convert `xyz` and `M` to exact `Fraction` entries and require every row of `M` to sum to exactly zero, so that normalization preserves the image point (33); (2) because $C_3 = -(C_0 + C_1 + C_2)$, rank $M = 3$ is equivalent to $\det[C_0 \ C_1 \ C_2] \neq 0$ (a `ValueError` otherwise); (3) solve the 3×3 system on columns 0–2 by Cramer’s rule for the particular preimage $y = (x_1, x_2, x_3, 0)$ of (37); (4) the embedded point lies in the image of the integer lattice $\{Mq : q \in \mathbb{Z}^4\}$ if and only if x_1, x_2, x_3 are all integers — then every integer preimage is $y + t \mathbf{1}$, and the min-0 representative `Quadray(x1, x2, x3, 0).normalize()` is the unique deterministic tie-break on the fiber (33) (the same rule as `Quadray.normalize`); a non-integral preimage raises `ValueError` (the point is not in the image).

The taxonomy is fail-closed. `ValueError` covers: `xyz` of length other than 3; `M` of the wrong shape; any row sum different from zero; $\det = 0$ (rank below 3); a non-integral preimage. `TypeError` covers entries that are not `int`/`float`/`Fraction`/`numbers.Integral` — note that `numpy.float64` (a float subclass) and `numpy.int64` (integral) are accepted, while `numpy.float32` is rejected. There is no floating-point fuzz anywhere: an off-lattice point raises rather than snapping to a neighbor.

The float-valued inverse `quadray.quadray_from_xyz` is a different, deliberately fuzzier contract: it applies the pseudoinverse $M^+ = M^T(MM^T)^{-1}$ and rounds each component half-up (`floor(v + 0.5)`, not banker’s rounding — its docstring argument shows why half-up provably stays in the $\mathbf{1}$ -coset of the exact preimage, where per-component round-half-even can leave it). For points on the lattice it round-trips exactly; for general $\mathbb{R}^3$ points it returns the nearest lattice point in quadray coordinates (component-wise — not always nearest in embedded XYZ distance), which is a snapping operation, not an inverse. The component-wise rule is the chosen contract;

an XYZ-nearest variant would be a separate function that searches neighboring classes. `quadray_roundtrip(q, M=None)` pins the round-trip contract

$$\text{from_xyz}(\text{to_xyz}(q)) = q, \quad (38)$$

and raises `AssertionError` explicitly (not a bare `assert`, so the check survives `python -O`) when it fails. The contract is exact for normalized integer quadrays and, because `quadray_roundtrip` threads the *same* embedding through both legs, it is **scale-robust**: for $M = c M_0$ with any $c \neq 0$ the pseudoinverse projection is scale-independent,

$$\text{pinv}(cM) (cM q) = q - \frac{\sum_i q_i}{4} \mathbf{1}, \quad (39)$$

since $\text{pinv}(cM) = (1/c) \text{pinv}(M)$ and the row space of cM_0 is that of M_0 (the $(1/c) \cdot c$ cancels). A shell site embedded at scale $1/2$ — the half-integer FCC picture of the same sites — round-trips through the same scaled embedding. The single documented `AssertionError` case is an **unnormalized** input: the inverse canonicalizes to the min-0 representative, e.g. $(2, 2, 2, 1)$ comes back as $(1, 1, 1, 0)$. Embeddings whose rows do not sum to zero lie outside the Urner family of (32) and outside this guarantee.

Executable check of exact recovery and round-trips:

```

1 from fractions import Fraction
2 from quadmath.lattice.conversions import (
3     quadray_to_xyz, xyz_to_quadray_canonical, quadray_roundtrip, urner_embedding,
4 )
5 from quadmath.core.quadray import Quadray
6 from quadmath.lattice.omni_numbering import sites_through_shell
7
8 q0 = Quadray(2, 1, 1, 0) # cuboctahedron vertex, shell 1
9 assert xyz_to_quadray_canonical(quadray_to_xyz(q0)) == q0
10 assert xyz_to_quadray_canonical(
11     (Fraction(-1), Fraction(1), Fraction(1))
12 ) == Quadray(1, 1, 1, 0) # exact entries, no float fuzz
13 for site in sites_through_shell(2)[:8]: # round-trips over enumerated sites
14     assert quadray_roundtrip(Quadray(*site)) == Quadray(*site)
15 assert quadray_roundtrip(q0, urner_embedding(0.5)) == q0 # scale-robust (same rows, both legs)

```

14.5 Lattice context

The canonical representatives produced by (37) sit in the IVM site structure of **IVM Field Learning**. A normalized quadray whose component sum is $\equiv 0 \pmod{4}$ is an IVM sphere center (`is_ivm_site`); the other three residue classes are the octahedral and tetrahedral voids of the packing. The radial observable is the shell norm

$$N(q) = \sum_i |q_i - \sigma/4| = 2k, \quad \sigma = \sum_i q_i, \quad (40)$$

computed by `quadray_shell_norm`: shell k is the set of sites with $N(q) = 2k$, populated by $10k^2 + 2$ centers (Eq. (28) in **Lattice Tooling**), and `ivm_field.shell_sites(k)` enumerates shell k in lexicographic order (1, 12, 42, 92, 162 sites for $k = 0..4$). Conversions, shells, and nearest-site search share one geometry; Sections 11 and 13 detail the field learner and the query machinery respectively.

14.6 API Summary

Function	Purpose
<code>conversions.urner_embedding(scale=1.0)</code> <code>conversions.quadray_to_xyz(q, M=None)</code>	The (3, 4) embedding matrix of Eq. (32) Forward map via <code>quadray.to_xyz</code> ; $M=None$ = <code>quadray.DEFAULT_EMBEDDING</code>
<code>conversions.xyz_to_quadray_canonical(xyz, M=None)</code> <code>conversions.quadray_roundtrip(q, M=None)</code>	Exact rational canonical inverse, Eqs. (33) and (37) Asserts the round-trip identity (38), scale-robust per (39)
<code>conversions.embedding_basis(M=None)</code>	(4, 3) array of embedding columns; Gram identity (35)
<code>quadray.to_xyz(q, embedding)</code> <code>quadray.quadray_from_xyz(x, y, z, embedding)</code>	Underlying forward product $xyz = M q$ Float inverse: pseudoinverse + round-half-up (snapping, not exact inversion)
<code>lattice_search.squared_distance(p, sites)</code>	Exact squared distances via Eq. (36)

14.7 Verification

`tests/test_conversions.py` covers the layer end to end: round-trips of (38) over shells 0–4 (309 enumerated sites, center included) and at scaled embeddings per (39), exact canonical recovery of every site through (37), the Gram identities (34) and (35) with `embedding_basis`, the distance identity (36) cross-checked against float embeddings and `quadray.distance`, and the fail-closed error taxonomy (`ValueError` for off-image points, bad shapes, zero determinants and non-zero row sums; `TypeError` for inexact-foreign entry types such as `numpy.float32`). The repository-root `SPEC.md` (the full mathematical specification) and `tests/test_spec_examples.py` pin the worked examples of this section numerically as they land. Run the suite:

```

1 PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 uv run coverage run -m pytest -q
2 uv run coverage report

```

15 Learning and Evaluation on the IVM Lattice

15.1 Overview

Learning on a lattice is easy to fake. A learner that merely interpolates its observations through the graph can look excellent on the very data it was fit to, and a dynamics model identified on early snapshots can silently diverge on late ones. This section develops the training/testing methodology used to evaluate the two IVM learners honestly — the static field learner of **Static IVM Field Learning** and the dynamics identifier of **IVM Lattice Dynamics**. All utilities live in the `learning_eval.py` module (`kfold_site_splits()`, `cross_validate_field()`, `trajectory_train_test()`, `learning_curve()`, `enclosing_radius()`), use `numpy` only, and are fully deterministic for fixed seeds. Every behavioral claim below is pinned in `tests/test_learning_eval.py` with fixed RNG seeds and real numerics.

15.2 Why held-out evaluation is harder on a lattice

On independent data, random resampling is benign. On a lattice it is not: field values at neighboring sites are strongly correlated, and the learners of `ivm_field.py` exploit exactly that correlation — the kernel-weighted target of the normal equations (21) predicts an unobserved site as a weighted average of its graph neighbors. A random split that leaves graph neighbors on both sides therefore lets the learner “cheat”: the held-out site is predicted from observations one hop away, and the apparent skill measures the smoothness of the field under (19), not generalization to unseen territory. The autocorrelation that makes lattice learning work is the same autocorrelation that inflates its evaluation:

$$\text{Cov}(f_i, f_j) \approx \sigma_f^2 K(d(i, j)), \quad d(i, j) = \text{hop distance on the IVM graph}, \quad (41)$$

with K the Gaussian hop kernel of (18) and σ_f^2 the field variance. Because K decays with hop distance, the leakage is local — but the IVM ball is small, and a random split leaks everywhere.

The methodology below therefore always holds out *structure*, not just rows: whole folds of sites for the field learner (never shared between training and scoring), and a temporal suffix of snapshots for dynamics identification (never shuffled). The pinned numerical result worth internalizing: on the radius-2 ball (55 sites, 80% observed), the noise-free held-out error of the kernel-weighted interpolator is dominated not by observation noise but by *coverage geometry* — which sites happen to be observed — so per-fold scores vary substantially even at fixed sample size, and honest evaluation must average over folds.

15.3 K-fold site splits

`kfold_site_splits(observed_sites, k, seed)` partitions observed sites into k folds by a seeded `numpy.random.default_rng` permutation cut with `numpy.array_split` (the first $n \bmod k$ folds receive one extra site). Each split is a (train, test) pair; folds are pairwise disjoint and jointly exhaustive, and site order within each list follows the original normalized index order, so the splits are a pure function of $(\text{sites}, k, \text{seed})$. With s the seeded permutation and F_j fold j ’s index set, the held-out score of a fit $\hat{f}_\lambda^{(-F_j)}$ that never saw F_j is

$$\widehat{M}(\lambda, F_j) = \frac{1}{|F_j|} \sum_{i \in F_j} (\hat{f}_\lambda^{(-F_j)}(i) - y_i)^2, \quad (42)$$

the quantity `IVMField.score()` returns. Duplicate sites are rejected after normalization (two projective representatives of one lattice point would sit on both sides of a split), as is $k < 2$ or $k > n$.

15.4 Cross-validation and honest model selection

`cross_validate_field(field_values, sites, k, lam_grid, seed)` runs (42) for every fold and every regularization candidate of `lam_grid`, fitting each fold from scratch on the training sites only — a fresh `IVMField.lattice_ball` per fit,

via `IVMField.learn()` with the objective (19). The result is the full table $\widehat{M}(\lambda, F_j)$ with per-candidate mean and standard deviation over folds, and the selected strength

$$\lambda^* = \arg \min_{\lambda \in \mathcal{G}} \frac{1}{k} \sum_{j=1}^k \widehat{M}(\lambda, F_j), \quad (43)$$

resolved to the earliest candidate on ties. Selection uses validation folds only; the returned `refit_field` is then refit with λ^* on *all* observed sites, matching the deployment condition where every observation is available. Separating the two is what makes the selection honest: choosing λ on the same data the final model fits is how interpolation gets mistaken for skill.

Two regimes are pinned numerically on synthetic fields. With exact observations of a harmonic (linear in embedded XYZ) field at 80% coverage, cross-validation selects $\lambda^* = 0$: the kernel-weighted interpolator generalizes best, and Laplacian smoothing only biases the fit away from exact data — the mean held-out MSE is strictly increasing in λ across a grid spanning two decades. With noisy observations of a smooth non-harmonic field, the selection flips to $\lambda^* > 0$: interpolation carries observation noise into the fit, and moderate regularization wins on the held-out folds. The margin is honest about the learner’s limits — because observed rows are pinned to their data, the noise propagates through the graph regardless of λ , so the achievable gain is set by how much neighbor averaging the kernel already performs, not by the regularizer alone.

15.5 Temporal splits for dynamics identification

`trajectory_train_test(observed, split_frac, lattice)` evaluates `fit_trajectory()`-style identification of the coupling α (**IVM Lattice Dynamics**). The snapshot matrix ($T + 1$ rows, row 0 the initial condition) is split **temporally**: the first nearest-integer-rounded fraction of rows trains, the trailing rows are held out, and no shuffling is ever applied — a shuffled split interpolates between adjacent snapshots of a deterministic trajectory and hides exactly the forecast error being measured. The split point is clamped so training keeps at least two rows (initial condition plus one snapshot) and testing keeps at least one. Identification uses the training prefix only, with candidate grid and refinement passed through to `fit_trajectory()`. Two held-out errors are reported. The multi-step continuation error re-simulates from the observed initial condition with the identified $\hat{\alpha}$ and scores only the held-out rows:

$$M_{\text{test}}(\hat{\alpha}) = \frac{1}{(T - t_s) N} \sum_{t=t_s+1}^T \|u_{\hat{\alpha}}(t) - u_{\text{obs}}(t)\|^2, \quad (44)$$

where t_s is the last training row. This is the stringent metric: for any model that is not exactly right, error compounds with horizon, so the held-out tail punishes misspecification that early rows forgive. The one-step-ahead error is teacher-forced — each held-out transition is predicted a single step from the *true* previous state, the first transition starting from the last training row:

$$M_1(\hat{\alpha}) = \frac{1}{(T - t_s) N} \sum_{t=t_s}^{T-1} \|\text{step}(u_{\text{obs}}(t); \hat{\alpha}) - u_{\text{obs}}(t + 1)\|^2, \quad (45)$$

isolating local prediction error from error accumulation. The contrast between (44) and (45) is diagnostic: a correct model drives both to zero (pinned exactly: a heat trajectory with $\alpha = 0.3$ in the candidate grid yields train, continuation, and one-step errors of exactly 0), while a misspecified model class — a heat model identified on majority-dynamics data — shows a modest training error, a held-out continuation error more than twice as large (horizon compounding), and a one-step error five times smaller than the continuation error (local prediction is easier than long-horizon forecasting). Note the direction of the misspecification: `majority_step()` requires integer states, so the float-continuum heat model can be fit to integer data, but not the reverse.

15.6 Learning curves

`learning_curve(values, sites, train_fracs, seed)` traces the classic data-coverage curve on a lattice. One seeded permutation of the observed sites is drawn; for each requested fraction p the first $m(p) = \text{round}(p \cdot n)$ sites of that permutation (clamped to $[1, n - 1]$) form the training subset and the remainder the held-out complement:

$$M(p) = \frac{1}{n - m(p)} \sum_{i \notin P_{m(p)}} (\hat{f}_{P_{m(p)}}(i) - y_i)^2, \quad P_m = \text{first } m \text{ entries of the permutation}, \quad (46)$$

reported together with the in-sample score of the same fit. The subsets are *nested* by construction — growing p never removes an earlier observation — so the curve is a proper coverage curve rather than a family of independent splits. The pinned example (radius-2 ball, quadratic bowl observed with noise $\sigma = 0.4$ everywhere, $\lambda = 0.05$) shows held-out MSE falling by more than a factor of two from 10% to 85% coverage, with non-monotone wiggles in between: with few observations the held-out complement is large and its geometry varies between fractions, so the curve is monotone-ish, not monotone — the final point sits far below the first, and that is the asserted invariant. The curve also separates interpolation from generalization visibly: the in-sample MSE stays below 0.15 at every coverage level while the held-out MSE never drops below 0.7 — the pinned observations are fit closely at all sizes, and the difficulty is entirely on unseen sites.

15.7 Three-way splitting and trainable site fits

The lattice surfaces above all share one shape: a seeded split of sites or snapshots, a fit on the training side, and scores on the held-out side. The same shape is available for plain tabular work in the same module (`src/quadmath/learn/learning_eval.py`), under the same discipline — all split randomness confined to the seeded permutation of the split, and fits that are pure deterministic functions of their inputs.

`three_way_split(n, train_frac=0.6, val_frac=0.2, seed=0)` partitions `range(n)` into disjoint train/validation/test index lists. One seeded `numpy.random.default_rng` permutation of `range(n)` is drawn and cut into three contiguous blocks: the first $m_{\text{train}} = \text{round}(\text{train_frac} \cdot n)$ indices train, the next $m_{\text{val}} = \text{round}(\text{val_frac} \cdot n)$ validate, and the remainder test. Each returned list is sorted, the three are pairwise disjoint with union exactly `range(n)`, and the split is a deterministic function of $(n, \text{train_frac}, \text{val_frac}, \text{seed})$ — the same deterministic-permutation contract as the k-fold site splits above, applied to a plain index range rather than observed lattice sites. Rounding is unguarded at the tails: for small n a trailing block may come back empty ($n = 3$ under the default fractions leaves the test list empty), and a `ValueError` is raised when either fraction is not strictly positive or their sum reaches 1 — a split with no held-out rows at all cannot be honest.

`ridge_site_fit(features, values, lam=1e-3)` fits a single linear site model in closed form. With X_c the mean-centered design matrix, y_c the mean-centered target, and $\bar{x}$, $\bar{y}$ the feature-column means and target mean, the ridge normal equations are solved exactly for the slope vector w :

$$(X_c^\top X_c + \lambda I) w = X_c^\top y_c, \quad b = \bar{y} - \bar{x}^\top w, \quad (47)$$

with b the intercept recovered from the centered solve; $\lambda \geq 0$ penalizes the centered coefficient norm, shrinking w toward zero, and $\lambda = 0$ recovers the exact least-squares solve. The returned `RidgeSiteFit` carries coefficients, intercept, and the in-sample `train_mse`, and the fit is a deterministic function of $(\text{features}, \text{values}, \text{lam})$ — a `numpy.linalg.solve`, not an iterative optimizer, so repeated calls on the same inputs reproduce it bit for bit. Validation is explicit: a negative λ , a non-2-D design matrix, a non-1-D target, disagreeing sample counts, or zero rows each raise before any linear algebra runs.

`GradientDescentTrainer(lr=0.05, max_iters=300, tol=1e-9)` is the iterative counterpart, useful where the closed form is deliberately being compared against: a full-batch gradient-descent fit of the same linear model. `fit` standardizes the feature columns internally (per-column zero mean and unit standard deviation, with constant columns passing through at standard deviation 1), then runs at most `max_iters` full-batch updates on the mean-squared-error loss,

$$\theta \leftarrow \theta - \frac{2\eta}{m} \begin{bmatrix} X_s^\top e \\ \mathbf{1}^\top e \end{bmatrix}, \quad e = X_s w + b - y, \quad (48)$$

where X_s is the standardized design, m the row count, η the learning rate, w and b the coefficient vector and bias, and y the target. The MSE at the top of each executed iteration is appended to `loss_history` — one float per iteration, recorded before the update it describes — and training stops early with `converged_ = True` as soon as two consecutive losses differ by less than `tol`. The fit exposes `coef_` expressed against the standardized features and `intercept_` in original target units, together with the stored `mean_` and `std_` it used, so `predict` re-applies the recorded standardization before the linear map and thus scores consistently across the train, validation, and test blocks of a `three_way_split`. Like `ridge_site_fit` the trainer draws no randomness of its own — no shuffling, no stochastic gradient — so its entire loss history is deterministic for fixed inputs: the same rows and hyperparameters replay the same loss curve to the last float, and the per-iteration record doubles as the divergence/plateau diagnostic that a scalar final loss cannot provide.

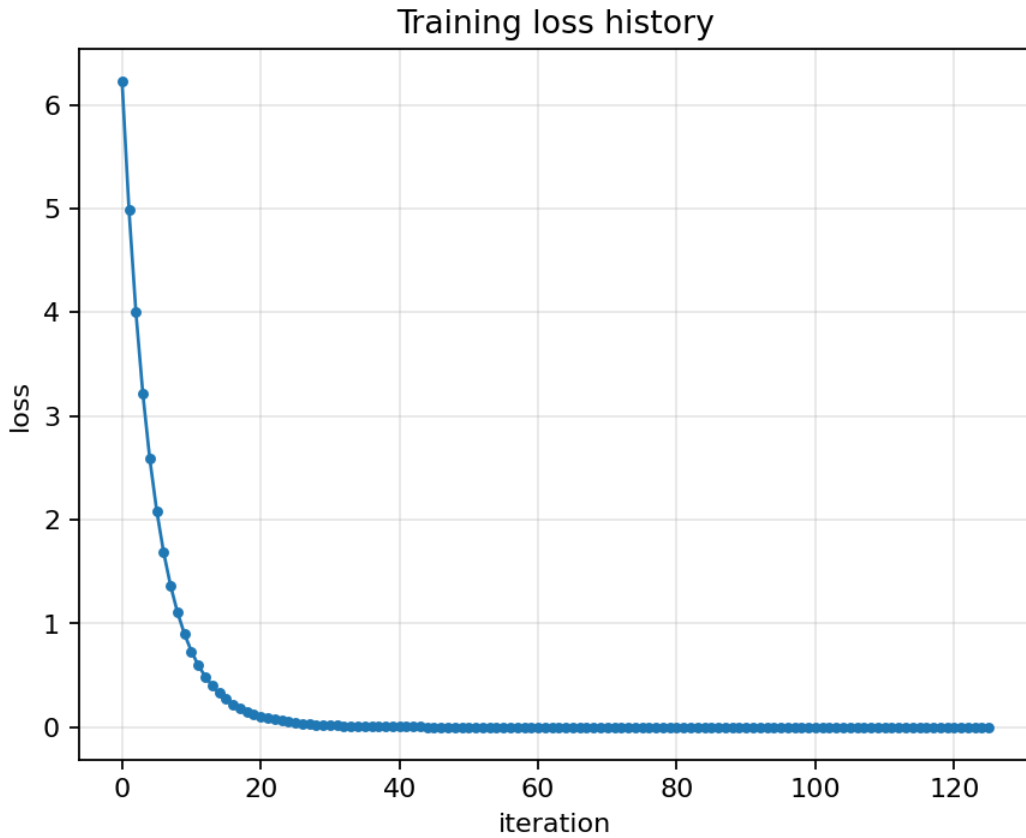


Figure 20: **Convergence of the full-batch gradient-descent trainer.** Per-iteration training loss (mean squared error, linear y -axis) of `GradientDescentTrainer` on a deterministic synthetic linear problem: a 40×3 standard-normal design matrix with true coefficients $(1.5, -2.0, 0.75)$, intercept 0.5, and Gaussian target noise $\sigma = 0.05$ (seed 0). Fitted with `lr = 0.05`, `max_iters = 300`, `tol = 1e-9`; the loss falls from 6.23 at iteration 0 to the noise-floor scale $\sigma^2 = 2.5 \times 10^{-3}$, flagged as converged after 126 iterations. Regenerate with `quadmath/scripts/learning_gallery.py`.

15.8 Verification

`tests/test_learning_eval.py` pins all of the above with fixed seeds and no mocks: disjoint-exhaustive folds with exact fold sizes, seed-determinism and seed-sensitivity of the splits, the $\lambda^* = 0$ and $\lambda^* > 0$ regimes with their table identities (mean and standard deviation recomputed from the fold table; the refit reproduced by hand),

exact-zero errors for the correctly specified dynamics against a $> 2\times$ continuation blow-up for the misspecified one, manual recomputation of the one-step-ahead error, fraction clamping at both extremes, and the learning curve's final-below-first invariant. The module carries 100% statement and branch coverage, and the validation rules of every entry point (distinct normalized sites, $k \in [2, n]$, fractions in the open unit interval, non-empty grids, minimum row counts) are each exercised.

15.9 Cross-References

- Coordinate foundations and the twelve-around-one shell: [Quadray Methods](#)
- The learner being evaluated: [Static IVM Field Learning](#)
- The dynamics being identified: [IVM Lattice Dynamics](#)
- Enumeration and search infrastructure: [Lattice Tooling](#)

16 Lattice Visualization Gallery

16.1 Overview

This section is the figure surface of the lattice layer: every rendering primitive used by the IVM chapters lives in `vis_lattice.py` (`shell_scatter`, `field_slice`, `dynamics_strip`, `gallery`). Each primitive receives its matplotlib axes explicitly, so panels compose inside caller-owned figures; only `gallery` creates figures (three, written as PNGs). Nothing renders at import time, and all randomness is drawn from a seeded `numpy.random.default_rng`, so the gallery is deterministic. The command-line entry is `quadmath/scripts/lattice_gallery.py` — a thin orchestrator that sets a headless backend and a fixed seed, delegates to `gallery` (thin-orchestrator contract of `quadmath/scripts/AGENTS.md`), and prints each written path on its own line: those stdout lines are the `make_all_figures` manifest contract.

16.2 Frequency shells

`shell_scatter(ax, sites, k)` embeds the sites of shell k (from `:func:omni_numbering.generate_shell`, which enumerates the $10k^2 + 2$ sites of shell k ; see [Lattice Tooling](#)) with `to_xyz` under `DEFAULT_EMBEDDING` and scatters them in 3D. With `axis_hints` it overlays four dashed rays from the origin toward the embedding matrix columns — the images of the quadray basis directions A, B, C, D , which are the vertices of a regular tetrahedron (pairwise dot product -1 , edge length $2\sqrt{2}$; see [Quadray Methods](#)). The hints make the tetrahedral axis frame of the lattice readable in print.

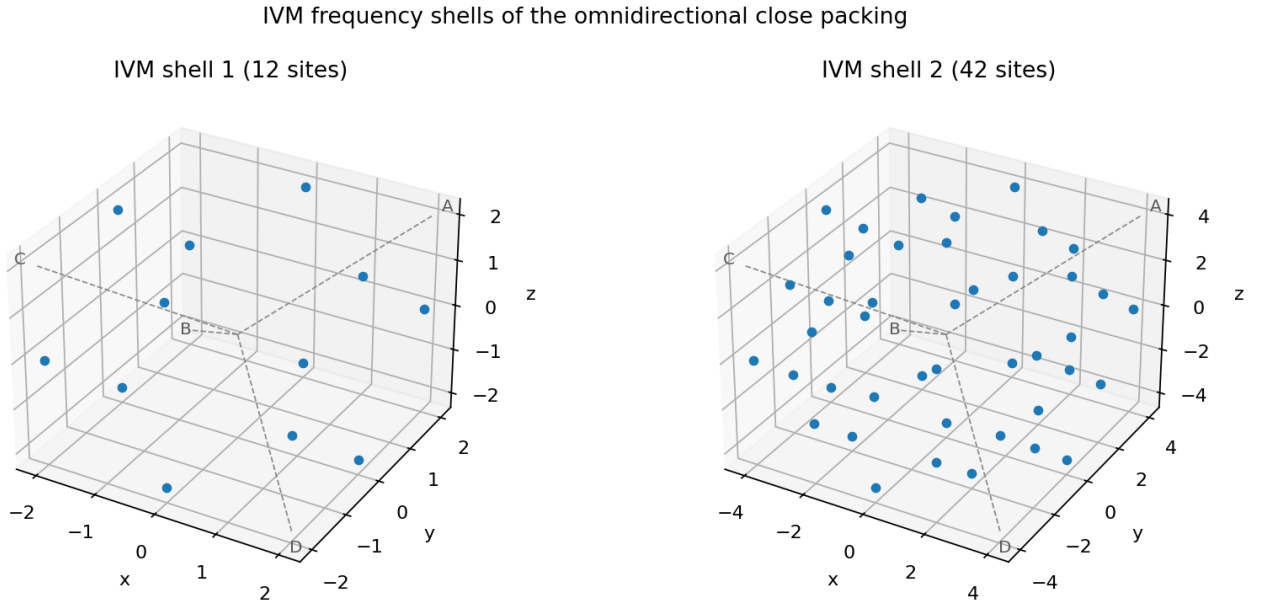


Figure 21: **Frequency shells of the omnidirectional close packing.** Shell 1 (12 sites) and shell 2 (42 sites) of the IVM lattice embedded via `DEFAULT_EMBEDDING` and scattered by `shell_scatter` as 3D point clouds in the x, y, z embedding coordinates; dashed gray rays mark the four tetrahedral quadray axes, labeled A, B, C, D , pointing toward the embedding-matrix columns. Reproduced with `uv run python quadmath/scripts/lattice_gallery.py` (fixed seed 12).

16.3 Lattice-plane slices

`field_slice(ax, field, sites, plane)` renders a scalar `ivm_field.IVMField` as a heatmap restricted to a lattice plane. A plane is parametrized by an origin q_0 and two independent integer translation vectors u, v — by default two of the twelve IVM neighbor moves, both permutations of $(2, 1, 1, 0)$ — and its points are

$$q(i, j) = \text{normalize}(q_0 + i u + j v), \quad (i, j) \in \mathbb{Z}^2, \quad (49)$$

with `normalize` the quadray canonical representative. Because each neighbor move has component sum 4, the sum stays divisible by 4 along the whole plane, so every $q(i, j)$ is again a lattice site. Two coordinate facts make the rendering exact: the embedding is linear, so the xyz image is the planar set $t(q_0) + i t(u) + j t(v)$; and the shift $(1, 1, 1, 1)$ lies in the embedding kernel, so plane membership is tested in integer quadray space — a site lies on the plane of (49) iff $q - q_0 = i u + j v + m(1, 1, 1, 1)$ for integers i, j, m . `field_slice` decides membership by an exact least-squares solve followed by an integer verification, so no site is ever misassigned by floating point. Sites off the plane are ignored, plane cells outside the field’s ball are masked, and the heatmap axes are the integer plane indices i (steps along u) and j (steps along v).

The gallery figure learns a field on the radius-3 ball (147 sites) from noisy observations of a linear synthetic truth (25% of sites observed, noise level 0.1, regularization `lam=0.01`, kernel width 0.5) and slices it through the default plane $((2, 1, 1, 0), (1, 2, 1, 0))$; the learned surface follows the planar lattice, with masked tiles where the ball ends. Field learning itself is the subject of [Static IVM Field Learning](#).

16.4 Dynamics strips

`dynamics_strip(axes, trajectory, t_indices)` renders selected snapshots of an `ivm_dynamics.simulate` trajectory (see [Dynamics and Learning on the IVM Lattice](#)) as a strip of 3D scatter panels over the embedded lattice. All panels share one symmetric color scale $[-v_{\max}, v_{\max}]$ with $v_{\max} = \max_t |u_t|$ over the selected snapshots, so evolution is directly comparable across panels. The gallery strip shows heat diffusion ($\alpha = 0.45$, horizon $T = 20$) on the radius-3 lattice (13 sites) at $t = 0, 10, 20$: the seeded random initial field relaxes toward its lattice average — the visually flat mid panel and right panel are the L_2 -monotone decay of the heat lemma in action.

16.5 Deterministic animations and gallery plots

Where the primitives above compose panels inside caller-owned figures, the module `src/quadmath/viz/animations.py` renders frame-based animations that own their pixels: each renderer returns a list of `Frame` objects — the frozen `dataclass Frame` in that file pairs a 2D `numpy` array with a human-readable `title`, and its `__post_init__` validation admits only `uint8` arrays (raw gray levels) or `float` arrays with every value in $[0, 1]$, rejecting anything that is not 2D, is of another `dtype`, or drifts outside the unit interval. No `matplotlib` is involved at frame level and no RNG or wall-clock input exists (except the explicit `seed` of `diffusion_frames`), so identical arguments yield byte-identical frames. All three renderers share one fixed-camera convention: sites are embedded in R^3 via `DEFAULT_EMBEDDING` and viewed by an orthographic camera looking down $+z$ — each point (x, y, z) projects to (x, y) on a `GRID_SIZE = 48` square grid whose extent is $[-8, 8]$ (module constant `_HALF_EXTENT`, sized to cover a radius-3 IVM ball of embedded norm 6 with margin even while pulsing), the vertical axis flipped so $+y$ points up, colliding points resolved by maximum brightness, and out-of-window points clamped to the border pixel.

`simplex_frames(q0, q1, n=16)` animates a quaternion-slerp rotation of the radius-1 ball: the two unit quaternions q_0, q_1 (validated to four components of unit norm) are interpolated on the shortest arc — sign-flipping q_1 when their dot product is negative, using the Shoemaker spherical formula $q(t) = \sin((1-t)\theta) q_0 / \sin \theta + \sin(t\theta) q_1 / \sin \theta$, and falling back to normalized linear interpolation when the inputs are nearly parallel — and at each $t = i/(n-1)$ the 13 lattice sites of `ball_sites` radius 1 are rotated by $q(t)$, projected onto the 48×48 grid, and shaded by radial shell, with origin brightness 1.0 and the outermost shell never dimmer than 0.25 so nothing renders invisible.

`lattice_frames(shells=3, n=12)` renders a pulsing IVM lattice ball: the radius-`shells` ball from `ball_sites` (147 sites at the default radius 3) has its embedded coordinates scaled per frame by the deterministic pulse $1 + 0.25 \sin(2\pi i/n)$ over one full growth-and-contraction cycle, while brightness per site stays the fixed radial-shell shading — brightest at the center, decaying linearly in shell index, outermost shell at 0.25 — so the pulse animates geometry, not shading.

`diffusion_frames(n_steps=12, seed=0)` seeds one-hot heat on the radius-3 ball and diffuses it over the IVM neighbor graph: the 147 ball sites become a graph in which two sites are adjacent iff their difference normalizes to one of the twelve `IVM_NEIGHBOR_STEPS`, a `numpy.random.default_rng(seed)` draw selects the source site deterministically, and

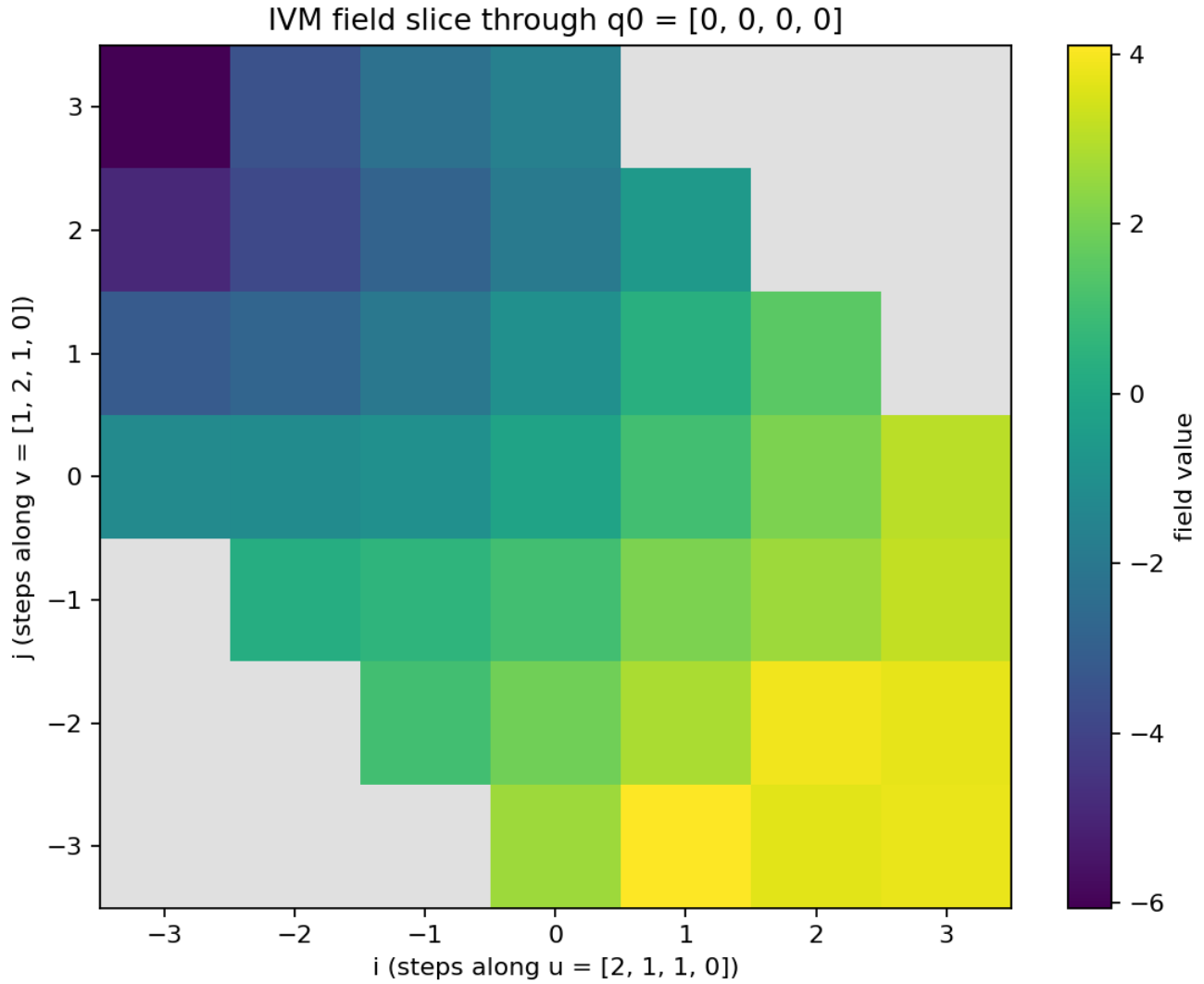


Figure 22: **Learned scalar field sliced along a lattice plane.** Heatmap of a Laplacian-regularized `IVMField` over the radius-3 ball, restricted by `field_slice` to the plane spanned by the neighbor moves $u = (2,1,1,0)$ and $v = (1,2,1,0)$: a viridis heatmap whose axes are the integer plane indices i (steps along u) and j (steps along v), with the colorbar reporting the field value and light-gray masked tiles marking plane cells outside the ball. Reproduced with `uv run python quadmath/scripts/lattice_gallery.py` (fixed seed 12).

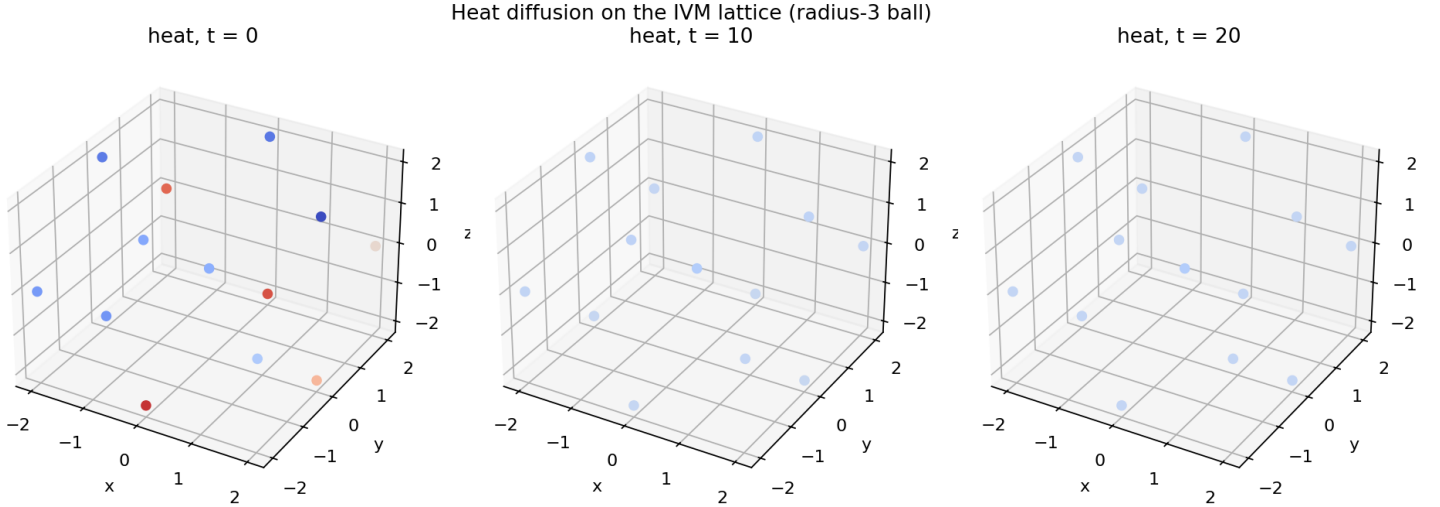


Figure 23: **Heat-diffusion evolution strip.** Snapshots of a seeded heat run at $t = 0, 10, 20$ on the radius-3 IVM lattice (13 sites), rendered by `dynamics_strip` as 3D scatter panels on one shared symmetric coolwarm scale $[-v_{\max}, v_{\max}]$ with $v_{\max} = \max_t |u_t|$ over the selected snapshots; the field visibly relaxes as the sum of squares decays monotonically. Reproduced with `uv run python quadmath/scripts/lattice_gallery.py` (fixed seed 12).

each explicit update applies $u \leftarrow u + \alpha$ (mean of graph neighbors $- u$) with $\alpha = 0.25$ (stable below 0.5) and values clipped at zero. Each frame is normalized by dividing by its (always positive) maximum heat, giving float values in $[0, 1]$; because heat only spreads to graph neighbors, the set of heated sites grows monotonically across frames.

The companion module `src/quadmath/viz/plots.py` supplies standalone figure builders for diagnostics that do not belong to any gallery: `plot_loss_history` draws a training loss sequence as a marker line over its iteration index, `plot_shell_growth` plots the IVM shell cardinalities — the cuboctahedral numbers $10k^2 + 2$ produced by `shell_cardinalities` in `src/quadmath/lattice/ivm_field.py` — versus shell index, `plot_error_histogram` histograms error values with a dashed vertical mean line, and `plot_lattice_shell_3d` scatters the sites of one shell (`shell_sites`, embedded via `DEFAULT_EMBEDDING`) in 3D with equal-aspect axes. Unlike the axes-composing primitives of `vis_lattice`, each owns a fixed-size figure (6.4×4.8 inches at 160 dpi), applies a fixed style, and on `save=True` writes its PNG into `quadmath/output/figures/` via `quadmath.paths.get_figure_dir`, returning the output path (empty string when the caller keeps the figure). All four are input-agnostic and seed-free: inputs are plain sequences or shell indices, styling is fixed, and no randomness or wall-clock time enters, so runs stay deterministic headless (MPLBACKEND=Agg).

`frames_to_gif(frames, out_path, fps=8, scale=8)` assembles a frame sequence into an animated GIF: float frames are quantized to `uint8` gray levels by rounding $255v$, each frame is upsampled by the integer factor `scale` with nearest-neighbor resampling, and the sequence is saved with per-frame duration $1000 / \text{fps}$ milliseconds and infinite looping. PIL is imported lazily inside the function (Pillow stays optional at import time), and because the PIL GIF encoder is deterministic and embeds no timestamps, identical `(frames, fps, scale)` inputs render byte-identical GIF files — the same byte-stability contract the PNG gallery asserts.

`frames_strip(frames, out_path=None, labels=None, save=True)` renders the print still-counterpart of `frames_to_gif`: a single-row matplotlib strip with one grayscale `imshow` panel per Frame, figure width scaling with the panel count while the height stays one panel plus the title band, each panel pinned to the unit brightness interval (`vmin = 0`, `vmax = 1`; `uint8` frames are first mapped by $v/255$, so both Frame dtypes share the GIF's gray-level convention), per-panel titles taken from `labels` or, by default, from each frame's own title, and axes switched off with zero inter-panel spacing so consecutive panels butt together. Matplotlib is imported lazily inside the function (the module stays matplotlib-free at import time), and saving follows the `plots.py` convention: a bare `out_path` name is written into `quadmath/output/figures/` via `quadmath.paths.get_figure_dir`, a path carrying a directory component is used verbatim, and the returned string is the written path ("" when `save` is off or no name is given). An empty frames sequence or a labels length mismatch raises `ValueError`; the render has no RNG and no wall-clock input, so identical inputs give byte-identical PNGs — the test suite asserts the byte stability, the full $[0, 1]$ grayscale

excursion, the uint8/float equivalence, and the panel-count scaling.

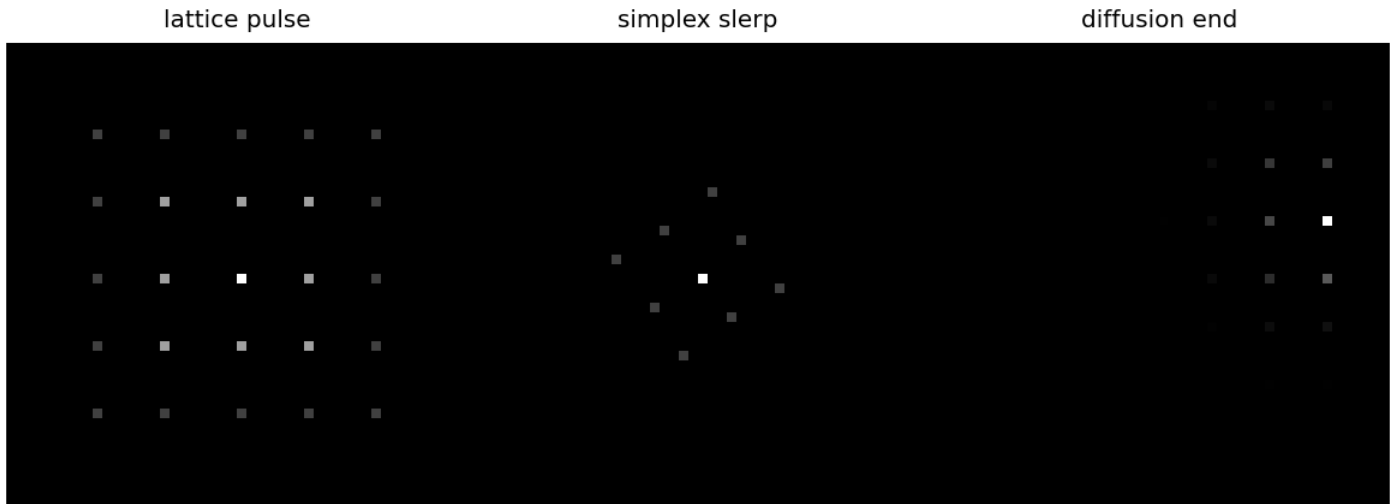


Figure 24: **Three animation stills in one strip.** Single-row `frames_strip` composition of one evenly spaced still from each 6-frame animation: left, the pulsing radius-2 IVM ball sampled at the quarter-pulse fraction (`lattice_frames`, radial-shell brightness, brightest at the center); center, the quaternion-slerp rotation of the radius-1 ball in the midpoint region of the arc ($t = 0.4$ on the way from the identity to a 90-degree rotation about z , `simplex_frames`); right, the final state of seeded heat diffusion on the radius-3 ball (`diffusion_frames`, brightness normalized by the frame's maximum heat). All panels are 48×48 orthographic projections under the fixed $+z$ camera with brightness in $[0, 1]$; reproduced with `uv run python quadmath/scripts/animation_stills.py`.

The command-line surface is `quadmath/scripts/animation_gallery.py` — a thin orchestrator in the `quadmath/scripts/AGENTS.md` contract that sets the headless Agg backend and renders three GIFs via `frames_to_gif`: `animation_lattice.gif` (pulsing radius-3 ball, 12 frames), `animation_simplex.gif` (slerp from the identity to a 90-degree rotation about the x axis, 16 frames), and `animation_diffusion.gif` (12 diffusion steps from seed 0), writing each into `quadmath/output/figures/` and printing each path on its own line. The script is registered in `quadmath/scripts/make_all_figures.py` alongside the other galleries, so the manifest contract of the Overview applies unchanged: only path lines on stdout, byte-identical re-renders for identical inputs. The companion `quadmath/scripts/animation_stills.py` follows the same thin-orchestrator contract for print stills: it renders the three 6-frame sequences of the module — `lattice_frames` at radius 2, `simplex_frames` from the identity to a 90-degree rotation about z with the endpoint quaternion derived from `rotate_about_axis`, and `diffusion_frames` at seed 0 — samples each at an evenly spaced fraction (quarter-pulse, slerp midpoint region, final diffusion step), composes the three stills with `frames_strip` into `animation_frames_strip.png`, and prints the written path on its own line.

16.6 Reproducibility and test contract

- `gallery(paths_out_dir, seed=12)` writes exactly the three files `vis_gallery_shell.png`, `vis_gallery_field.png`, `vis_gallery_dynamics.png` (module constant `GALLERY_FILES`, in that order) into `paths_out_dir`, creating it when missing, and returns their paths. Every panel is fully seeded, and the PNGs carry no timestamp metadata: the test suite asserts byte-identical re-renders for a fixed seed.
- Tests: `tests/test_vis_lattice.py` — 19 tests, no mocks, deterministic assertions without pixel diffs (artist placement on caller-provided axes, exact field values at known lattice cells, byte-level reproducibility). Scope with `PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 uv run coverage run -m pytest tests/test_vis_lattice.py -q` and `uv run coverage report` — 100% statement and branch coverage of `src/quadmath/viz/vis_lattice.py`.
- Tests: `tests/unit/viz/test_animations.py` covers the frame module (Frame validation, the three renderers, `frames_to_gif`) and the strip renderer — `frames_strip` label and emptiness validation, byte-identical re-renders in temporary directories, uint8/float equivalence, the full grayscale excursion, and panel-count scaling — headless via `MPLBACKEND=Agg` (set in `tests/conftest.py`).

- Figure regeneration: `uv run python quadmath/scripts/lattice_gallery.py` (the script sets `MPLBACKEND=Agg` itself); stdout is exactly the three written paths.
- Still-strip regeneration: `uv run python quadmath/scripts/animation_stills.py` (the script sets `MPLBACKEND=Agg` itself); stdout is exactly the one written strip path, `quadmath/output/figures/animation_frames_strip.png`.
- Markdown: `uv run python quadmath/scripts/validate_markdown.py`.

16.7 Cross-references

- Site geometry, normalization, and the embedding: [Quadray Methods](#).
- Shell enumeration and nearest-site search: [Lattice Tooling](#).
- The sliced learner: [Static IVM Field Learning](#).
- The stripped dynamics: [Dynamics and Learning on the IVM Lattice](#).

17 Benchmarks and Statistics

17.1 Overview

This section documents the measurement surface behind the preceding chapters. It is split across three modules with one job each: `src/quadmath/stats/benchmarks.py` measures (a small `perf_counter` timing harness over the core numerical surfaces), `src/quadmath/stats/statistics.py` analyzes (resampling inference, effect sizes, multiplicity correction, and scaling fits, on standalone `numpy`), and `src/quadmath/viz/vis_stats.py` renders (input-agnostic matplotlib primitives; the figures they produce are collected in `18_stats_gallery.md`). The measured workloads are the four surfaces the manuscript already treats: quadray conversions (`quadray.py`, conventions in `SPEC.md`), shell enumeration (`omni_numbering.py`, `13_lattice_tooling.md`), nearest-site search (`lattice_search.py`), and field fitting (`ivm_field.py`, `11_ivm_field_learning.md`). Everything here is deterministic: every random draw comes from a seeded `numpy.random.default_rng`, and every timing routine is fully specified by its arguments, so the numbers below reproduce bit-for-bit.

17.2 Timing methodology

`time_callable(fn, *, trials=5, warmup=1)` measures wall-clock duration with `time.perf_counter`, the highest-resolution monotonic clock available. It runs one warmup call first and discards it (first-call effects: module imports, key caches), then times `trials` calls and reports the summary statistics

$$\bar{t} = \frac{1}{T} \sum_{i=1}^T t_i, \quad t_m = \text{median}(t_1, \dots, t_T), \quad (50)$$

$$t_{95} = Q_{0.95}(t_1, \dots, t_T), \quad (51)$$

where T is the number of trials and Q_p the empirical p -quantile. The median is robust to a single outlier trial; the 95th percentile bounds the tail that a user would actually feel. Results are returned as `BenchRow`, a frozen dataclass with fields `name`, `n`, `trials`, `total_s`, `mean_s`, `median_s`, `p95_s`, and `ops_per_s`, plus `as_dict()` for plain data. Here n is the per-call workload size (conversions per call, shell depth, sites or queries per call), and throughput is

$$\text{ops_per_s} = \frac{n}{\bar{t}}. \quad (52)$$

Four constructors wrap the core surfaces with these defaults: `bench_conversions(n=200, trials=5)` times round-trip quadray/embedding conversions, `bench_shell_enumeration(k_max=4, trials=5)` times shell enumeration through `omni_numbering.generate_shell`, `bench_lattice_search(n_sites=200, queries=50, trials=5)` times `lattice_search` nearest-site queries over a random ball of sites, and `bench_field_fit(n_sites=64, trials=3)` times a synthetic field fit through `IVMField.learn.run_all()` executes all four with the shared defaults collected in the module constant `BENCH_DEFAULTS` and returns the list of `BenchRow` records; `summary_table(rows)` renders them as a fixed-width text table. The harness measures only; it asserts nothing — timing distributions belong to analysis, not to test assertions.

17.3 Percentile bootstrap

`bootstrap_ci(x, stat=np.mean, *, iters=2000, seed=0, alpha=0.05)` gives a confidence interval for any statistic $\hat{\theta}$ by resampling the data with replacement. With $n = |x|$ and a seeded `numpy.random.default_rng(seed)`, iteration b draws resampling indices $i_{bj} \sim \text{Uniform}\{0, \dots, n-1\}$ in one vectorized call (`rng.integers(0, n, size=(iters, n))`) and evaluates

$$\hat{\theta}_b^* = \hat{\theta}(x_{i_{b1}}, \dots, x_{i_{bn}}), \quad \text{CI}_{1-\alpha} = \left[Q_{\alpha/2}(\hat{\theta}_{1..B}^*), Q_{1-\alpha/2}(\hat{\theta}_{1..B}^*) \right], \quad (53)$$

with $B = \text{iters}$. The percentile interval needs no distributional assumption about $\hat{\theta}$ — only that the empirical resampling distribution approximates its sampling distribution. All randomness is consumed from the seeded generator, so the interval is exactly reproducible.

17.4 Pooled permutation tests

`permutation_test(a, b, *, iters=2000, seed=0, alternative="two-sided")` tests whether two samples differ in location without assuming a distribution. Both samples are pooled; the observed statistic is $o = |\bar{a} - \bar{b}|$; each of the `iters` iterations draws `rng.permutation` of the pool, splits it at $|a|$ into a fake a and b , and recomputes the difference. The two-sided p-value uses the add-one convention

$$p = \frac{1 + \#\{r : |d_r| \geq o\}}{\text{iters} + 1}, \quad (54)$$

where d_r is the difference of permutation r . The $+1$ in numerator and denominator counts the observed arrangement itself and guarantees $p \geq 1/(\text{iters} + 1) > 0$: a permutation test can never report an impossible zero p-value, and the estimate is conservative. `alternative="greater"` and `"less"` use the same convention with signed comparisons of d_r against the signed observed difference.

17.5 Effect size, multiplicity, and scaling

`cohens_d(a, b)` reports the standardized mean difference

$$d = \frac{\bar{a} - \bar{b}}{s_p}, \quad s_p = \sqrt{\frac{(n_a - 1)s_a^2 + (n_b - 1)s_b^2}{n_a + n_b - 2}}, \quad (55)$$

with s_a, s_b the unbiased sample standard deviations. A p-value says whether a difference is distinguishable from noise; d says how large the difference is, which is the question that matters when comparing benchmark configurations.

`p_adjust_bonferroni(pvals)` controls the family-wise error rate when m hypotheses are tested at once:

$$p'_i = \min(1, m p_i). \quad (56)$$

Each p'_i can be read as a valid p-value at level α for the whole family — conservative, but assumption-free.

`scaling_fit(sizes, times)` fits ordinary least squares on the log-log data,

$$\log t = \beta_1 \log n + \beta_0, \quad (57)$$

and returns `(slope, intercept, r2)`. The slope is the empirical complexity exponent: $\beta_1 \approx 1$ indicates linear work in the input size n , $\beta_1 \approx 2$ quadratic, and the coefficient of determination r^2 measures how well the power law describes the measurements.

17.6 Jackknife, FDR, Welch, and circular statistics

`jackknife_ci(x, stat=np.mean, alpha=0.05)` produces both a confidence interval and a bias estimate for any scalar statistic without consuming a single random draw. Where the percentile bootstrap of (53) resamples with replacement, the jackknife deletes one observation at a time: each of the n leave-one-out samples is scored, and the spread of those scores determines the interval. With $\hat{\theta}$ the full-sample statistic and $\hat{\theta}_{(i)}$ the statistic on the sample with observation i deleted, the returned triple `(low, high, bias)` is

$$\begin{aligned} \text{bias} &= (n-1)(\bar{\theta}_{(\cdot)} - \hat{\theta}), & \hat{\theta}_{\text{corr}} &= \hat{\theta} - \text{bias}, \\ \text{se}_{\text{jack}} &= \sqrt{\frac{n-1}{n} \sum_{i=1}^n (\hat{\theta}_{(i)} - \bar{\theta}_{(\cdot)})^2}, & \text{CI}_{1-\alpha} &= \hat{\theta} \pm z_{1-\alpha/2} \text{se}_{\text{jack}}, \end{aligned} \quad (58)$$

where $\bar{\theta}_{(\cdot)}$ is the mean of the leave-one-out scores. The bias-corrected estimate $\hat{\theta}_{\text{corr}}$ removes the first-order bias that the jackknife detects in curved statistics; the interval itself stays centered on the uncorrected $\hat{\theta}$. The normal quantile z comes from bisecting `math.erfc(z / sqrt(2))`, which decreases monotonically from 2 to 0 as z runs from $-\infty$ to ∞ , so one hundred halvings of the bracket $[-40, 40]$ pin z to double precision. This deterministic `erfc`-bisection quantile needs no seed and no lookup table, so repeated calls with identical arguments return bit-identical results — the same reproducibility contract as the seeded routines above, but with no generator to seed at all.

`benjamini_hochberg(pvals)` controls a different error rate from the Bonferroni map of (56): instead of the family-wise error rate it controls the expected proportion of false discoveries among rejections, the false discovery rate. The step-up procedure sorts the m p-values ascending, forms the raw values $m p_{(i)}/i$ for ranks $i = 1, \dots, m$, and then enforces monotonicity with a running minimum taken from the largest rank backwards,

$$p'_{(i)} = \min\left(1, \min_{j \geq i} \frac{m p_{(j)}}{j}\right), \quad p'_{(m)} = \min(1, p_{(m)}), \quad (59)$$

so the adjusted values are monotone non-decreasing in the sorted p-values and never exceed the largest raw p-value. The backward cumulative minimum is the step that the naive $m p_{(i)}/i$ mapping misses: without it, a large early-rank raw value could exceed a smaller later-rank one, breaking monotonicity and invalidating the FDR guarantee. The stable argsort of the input is recorded so the adjusted values can be mapped back to the original input order, and the final cap at 1.0 keeps the output a valid p-value vector.

`welch_t_test(a, b, alternative="two-sided")` compares two sample means without assuming equal variances, the assumption that the pooled standard deviation s_p of `cohens_d` makes. Writing $w_a = s_a^2/n_a$ and $w_b = s_b^2/n_b$, the statistic and its Welch-Satterthwaite degrees of freedom are

$$t = \frac{\bar{a} - \bar{b}}{\sqrt{w_a + w_b}}, \quad \nu = \frac{(w_a + w_b)^2}{\frac{w_a^2}{n_a - 1} + \frac{w_b^2}{n_b - 1}}. \quad (60)$$

The p-value is evaluated from the exact Student- t tails rather than a normal approximation, and it needs no `scipy`: the two-sided tail uses the identity

$$P(|T| \geq |t|) = I_x\left(\frac{\nu}{2}, \frac{1}{2}\right), \quad x = \frac{\nu}{\nu + t^2}, \quad (61)$$

where $I_x(a, b)$ is the regularized incomplete beta function computed by a Lentz continued fraction (with the standard symmetry swap above $x = (a+1)/(a+b+2)$, an exp-log Γ front factor, and guard floors against vanishing denominators). "greater" and "less" reuse the same two-sided tail, halved on the appropriate side of $t = 0$. The degenerate branches matter for constant inputs: when both samples are constant the denominator of (60) vanishes, and equal means give $t = 0.0$ (p-value 1.0 two-sided) while different means give a signed infinite t whose tail probabilities are exactly 0 or 1, evaluated against the finite pooled fallback $\nu = n_a + n_b - 2$ so the tails stay well-defined.

`rotation_stats(angles)` treats angles in radians as points on a circle rather than points on a line, where 359° and 1° are neighbors:

$$\bar{\theta} = \text{atan2}\left(\frac{1}{n} \sum_j \sin \theta_j, \frac{1}{n} \sum_j \cos \theta_j\right), \quad R = \left| \frac{1}{n} \sum_j e^{i\theta_j} \right|. \quad (62)$$

The circular mean is the `atan2` of the averaged sine and cosine, wrapped into $(-\pi, \pi]$, and the mean resultant length $R \in [0, 1]$ measures concentration: $R = 1$ only when every angle agrees, $R = 0$ when the unit vectors cancel completely. The returned dictionary gives the circular variance $1 - R$ in $[0, 1]$, and `variance_2pi` $= 2(1 - R)$ under the convention for angles on the full $[0, 2\pi)$ circle, which ranges over $[0, 2]$ and approaches the familiar linear variance for tightly clustered angles. Because sine and cosine are 2π -periodic, wrapping the angles into any full circle leaves every returned value unchanged; when the resultant vanishes the mean degenerates to whatever `atan2` returns for the cancelled components. Like everything else in `src/quadmath/stats/statistics.py`, all four routines are pure `numpy-plus-math` computations: no randomness, no optional scientific stack, and bit-identical repeats for identical arguments.

17.7 A reproducible example

Both workhorses above are fully determined by their arguments, so the following numbers are exact for any reader. The first call computes the 95% bootstrap confidence interval of the mean of the first seven Fibonacci numbers; the second asks whether the shifted ranges 1..10 and 11..20 differ in location:

```

1 import numpy as np
2
3 from quadmath.stats.statistics import bootstrap_ci, permutation_test
4
5 x = [2, 3, 5, 8, 13, 21, 34]
6 lo, hi = bootstrap_ci(x, np.mean, iters=2000, seed=0, alpha=0.05)
7 # (5.142857142857143, 20.571428571428573)
8
9 a = list(range(1, 11))
10 b = list(range(11, 21))
11 p = permutation_test(a, b, iters=2000, seed=0, alternative="two-sided")
12 # 0.0014992503748125937

```

With `bootstrap_ci(x, np.mean, iters=2000, seed=0, alpha=0.05)` the mean of x is $86/7 \approx 12.285714$ and the 95% percentile bootstrap interval is $[5.142857142857143, 20.571428571428573]$. Both endpoints are exact multiples of $1/7$: the mean of any 7-value resample with repetition is a multiple of $1/7$, so the percentile grid is discrete rather than dense. With `permutation_test(a, b, iters=2000, seed=0, alternative="two-sided")` on $a = 1, \dots, 10$ and $b = 11, \dots, 20$, the observed difference is $o = |\bar{a} - \bar{b}| = 10$ exactly, and exactly 2 of the 2000 seeded permutations reach it, giving

$$p = \frac{2 + 1}{2000 + 1} = \frac{3}{2001} \approx 0.00149925. \quad (63)$$

The count of 2 deserves a comment. Among the $\binom{20}{10} = 184,756$ possible splits of the pool, exactly two achieve $|d_r| = 10$ — the observed arrangement (1..10 against 11..20) and its complement — so the expected hit count over 2000 random draws is only $2000 \cdot 2/184756 \approx 0.02$. The add-one floor of (54) is $1/2001 \approx 0.0005$, which is what a typical seed reports here; the stream seeded with `seed=0` happens to draw both exact-maximum arrangements (at iterations 70 and 1146 of the 2000), so the count is 2 and the reported p-value is $3/2001$. The example illustrates the convention rather than a typical run: with any other seed the same call reproduces the floor value, and in either case the test correctly never reports the impossible $p = 0$.

The two interval families meet on common ground in a second example. Two synthetic lattice-error samples with a planted mean shift are drawn once from a seeded generator: routine A contributes $n = 64$ error residuals centered on 0.0 and routine B 64 residuals whose population mean sits 0.08 higher, both with scale 0.15 — the unit being whatever the error residual measures. For each sample mean, the 95% percentile bootstrap

interval of (53) ($B = 2000$ resamples, re-seeded at 17) and the jackknife interval of (58) (which consumes no randomness at all) are computed, and the script `quadmath/scripts/stats_diagnostics_gallery.py` draws all four intervals in a single `plot_ci_bars` call:

```

1 import numpy as np
2
3 from quadmath.stats.statistics import bootstrap_ci, jackknife_ci
4
5 rng = np.random.default_rng(17)
6 a = rng.normal(0.0, 0.15, size=64)
7 b = rng.normal(0.08, 0.15, size=64)
8 bootstrap_ci(a, np.mean, iters=2000, seed=17, alpha=0.05)
9 # (-0.07609532416913227, -0.0004865391750512773)
10 jackknife_ci(a, np.mean, alpha=0.05)[:2]
11 # (-0.07682589122849623, 0.0013292867175937334)

```

What the two families assume is where they differ. The percentile bootstrap needs no shape assumption at all — only that the empirical resampling distribution of (53) approximates the sampling distribution of $\hat{\theta}$ — while the jackknife interval is the normal approximation $\hat{\theta} \pm z_{1-\alpha/2} \text{se}_{\text{jack}}$ built from the n leave-one-out scores of (58). On the sample mean with $n = 64$ the two agree to within a whisker: routine A’s bootstrap interval is $[-0.076095, -0.000487]$ against the jackknife’s $[-0.076826, 0.001329]$, and routine B’s is $[0.056788, 0.132418]$ against $[0.056011, 0.130639]$ (sample means -0.037748 and 0.093325 ; the jackknife bias estimates sit at floating-point zero, as (58) predicts for a linear statistic). The figure records the comparison: routine B’s planted shift shows up as both of its intervals clearing the dashed zero line, while routine A hugs the line so tightly that the bootstrap upper endpoint stops 0.0005 short of it and the jackknife upper endpoint crosses it by 0.0013 — a hair-width disagreement that is the honest lesson of the panel. Coverage verdicts this close to the boundary are method-sensitive; a Welch test on the same samples reports $t \approx -4.75$ with $p \approx 5.4 \times 10^{-6}$ ((60)), in agreement with the shift both interval families detect.

17.8 Cross-references

- Timing harness, `BenchRow`, and the four benchmark constructors: `src/quadmath/stats/benchmarks.py`.
- Bootstrap, permutation tests, `cohens_d`, `p_adjust_bonferroni`, `scaling_fit`: `src/quadmath/stats/statistics.py`.
- The figure primitives built on these results: `18_stats_gallery.md`.
- The interval-comparison script and figure behind the second example: `quadmath/scripts/stats_diagnostics_gallery.py` (`quadmath/output/figures/stats_ci_comparison.png`).
- Measured surfaces: `quadrays.py` (conventions in `SPEC.md`), `omni_numbering.py` and `lattice_search.py` (`13_lattice_tooling.md`), `ivm_field.py` (`11_ivm_field_learning.md`).
- Sibling gallery of the lattice layer: `16_lattice_gallery.md`.

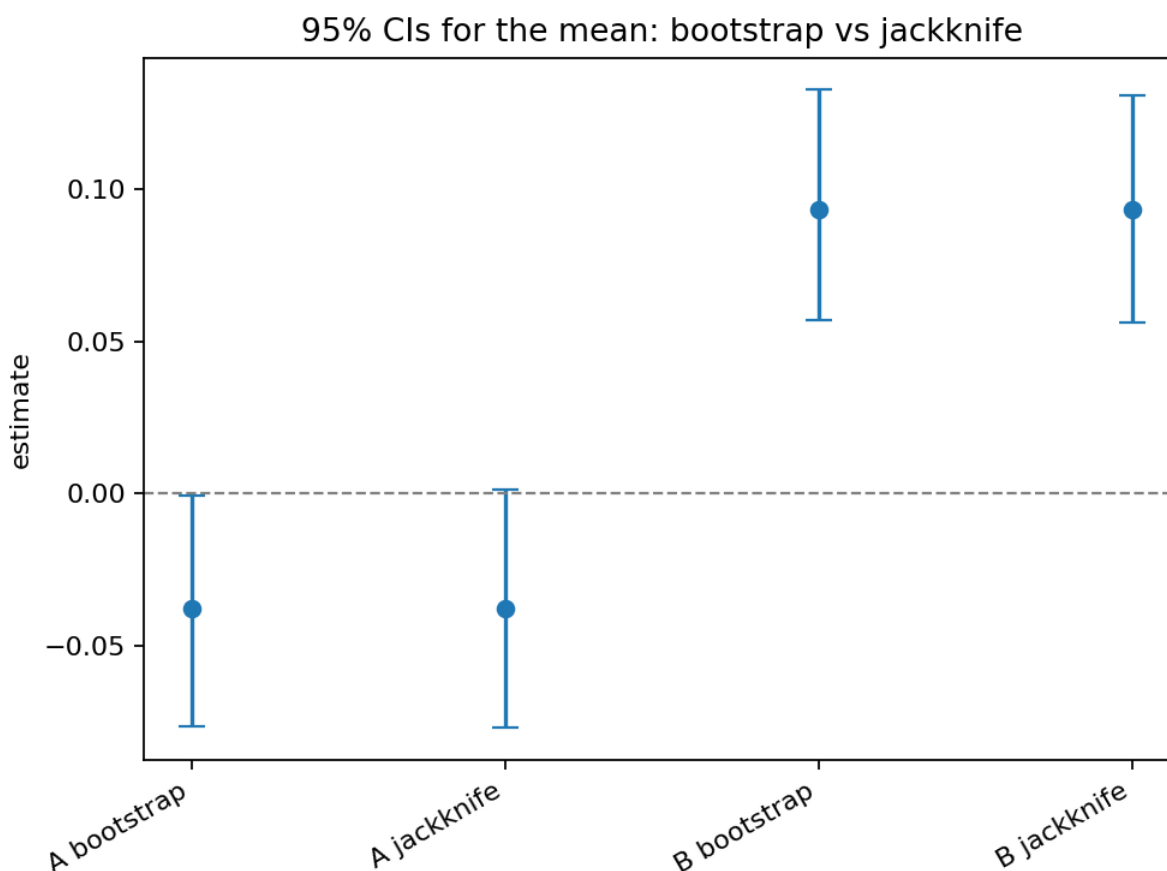


Figure 25: **Bootstrap versus jackknife 95% confidence intervals for two lattice-error samples.** Sample means (dots) of two seeded synthetic error samples — $n = 64$ per group from `numpy.random.default_rng(17)`, scale 0.15, planted mean shift 0.08 in the B population — each carrying a 95% percentile bootstrap interval (`bootstrap_ci`, 2000 resamples, seed 17, $\alpha = 0.05$) and a deterministic normal-approximation jackknife interval (`jackknife_ci`, $\alpha = 0.05$), drawn together in one `plot_ci_bars` call; the dashed line marks the unbiased population mean 0.0. Routine B’s intervals exclude 0 while routine A’s straddle it, and the two CI families agree within whisker width on both samples. Regenerate: `uv run python quadmath/scripts/stats_diagnostics_gallery.py`.

18 Statistics and Scaling Gallery

18.1 Overview

This section is the figure surface of the benchmarks-and-statistics layer: every rendering primitive used by section 17_benchmarks_statistics.md lives in `src/quadmth/viz/vis_stats.py` (`plot_latency_hist`, `plot_scaling_loglog`, `plot_ci_bars`, `plot_ecdf`). The primitives are input-agnostic — plain numpy arrays in, artists out; they import nothing from the other `src/` modules and receive their matplotlib axes explicitly, so panels compose inside caller-owned figures and nothing renders at import time. The command-line entry is `quadmth/scripts/stats_gallery.py` — a thin orchestrator that sets a headless backend and a fixed seed (`seed=32`), delegates to the gallery function in `src/quadmth/viz/vis_stats.py` (thin-orchestrator contract of `quadmth/scripts/AGENTS.md`), and prints each written path on its own line: those stdout lines are the `make_all_figures` manifest contract. The gallery writes exactly four PNGs — `stats_gallery_latency.png`, `stats_gallery_scaling.png`, `stats_gallery_ci.png`, `stats_gallery_ecdf.png` — and every random draw comes from the fixed seed, so the whole set is deterministic.

18.2 Latency histogram

`plot_latency_hist(times, ax=None, *, bins=20, title=...)` renders the distribution of per-call wall-clock durations measured by `src/quadmth/stats/benchmarks.py::time_callable` (trials summary in 17_benchmarks_statistics.md) as a histogram with `bins` bins on the given axes. The shape of the distribution carries what the mean alone hides: a tight spike means the harness saw a stable workload, while a heavy right tail is exactly what the `median_s / p95_s` fields of `BenchRow` are there to expose. The gallery panel histograms a deterministic synthetic stand-in for such a sample: 256 draws of a lognormal distribution (mean 0, $\sigma = 0.6$, clipped to $[0.05, 5.0]$ in seconds-scale units) taken from the single fixed-seed `numpy.random.default_rng(32)` generator, so re-renders are byte-identical.

18.3 Scaling fit

`plot_scaling_loglog(sizes, times, ax=None, *, title=...)` plots measured workload sizes against durations on log-log axes and overlays the least-squares power law of `src/quadmth/stats/statistics.py::scaling_fit` — the slope is the empirical complexity exponent β_1 of (57). On log-log axes a power law is a straight line, so the panel shows at a glance whether a lattice routine scales linearly, quadratically, or worse, and how tightly the data follow the law (r^2). The gallery panel fits a synthetic sweep with known ground truth: durations $t \approx 3n^{1.35}(1 + \varepsilon)$ with $\varepsilon \sim \mathcal{N}(0, 0.02^2)$ relative noise over sizes $n \in \{8, 16, 32, 64, 128, 256\}$, drawn from the same fixed-seed generator (times rounded to four decimals for byte stability), so the fit should recover the planted exponent 1.35.

18.4 Confidence intervals

`plot_ci_bars(labels, means, lows, highs, ax=None, *, title=...)` renders paired estimates as a bar-and-whisker chart: one bar per label at its point estimate, with error bars spanning the low and high ends of a percentile bootstrap interval `src/quadmth/stats/statistics.py::bootstrap_ci` ((53)). Overlapping intervals visually encode the same information a permutation test quantifies ((54)): two conditions whose intervals do not overlap are the ones the pooled test flags. The gallery panel shows three benchmark estimates (`to_xyz`, `quadrant_from_xyz`, `shell_enum`) with symmetric intervals whose half-widths are seeded uniform draws on $[0.05, 0.25]$ from the same fixed-seed `numpy.random.default_rng(32)` generator: the primitive renders whatever `lows / highs` it is given, and in a measured analysis those bounds are the percentile endpoints `bootstrap_ci` returns ((53)).

18.5 Empirical CDF

`plot_ecdf(values, ax=None, *, title=...)` renders the empirical distribution function

$$\hat{F}(t) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{t_i \leq t\}, \quad (64)$$

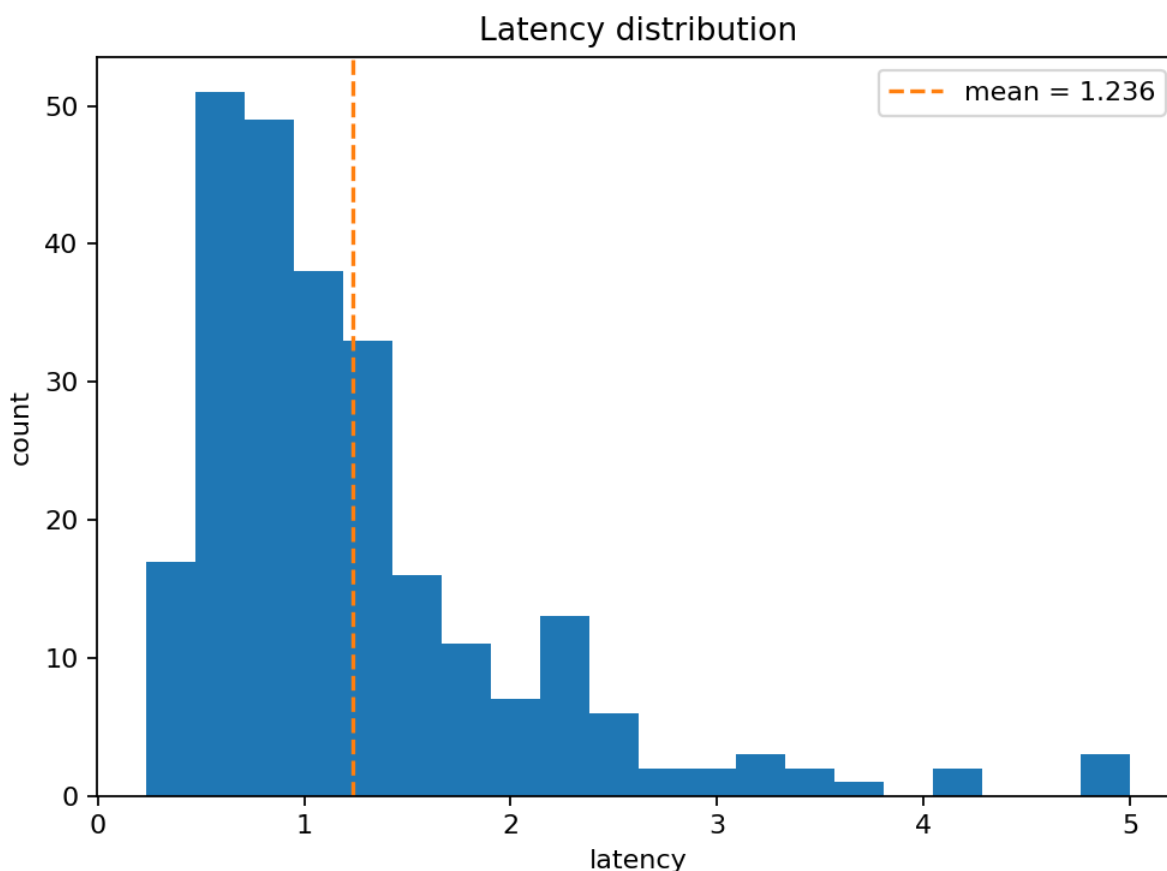


Figure 26: **Latency distribution.** Histogram (20 bins) of a synthetic 256-draw lognormal latency sample (mean 0, $\sigma = 0.6$) clipped to $[0.05, 5.0]$ in seconds-scale units, drawn from the fixed-seed `numpy.random.default_rng(32)` generator of `stats_gallery.py`; the dashed line marks the sample mean. Rendered by `plot_latency_hist`; reproduced by `uv run python quadmath/scripts/stats_gallery.py`. The spread and right tail complement the `mean_s`, `median_s`, and `p95_s` fields of `BenchRow`, whose per-call times `time_callable` measures with `time.perf_counter` after one discarded warmup call.

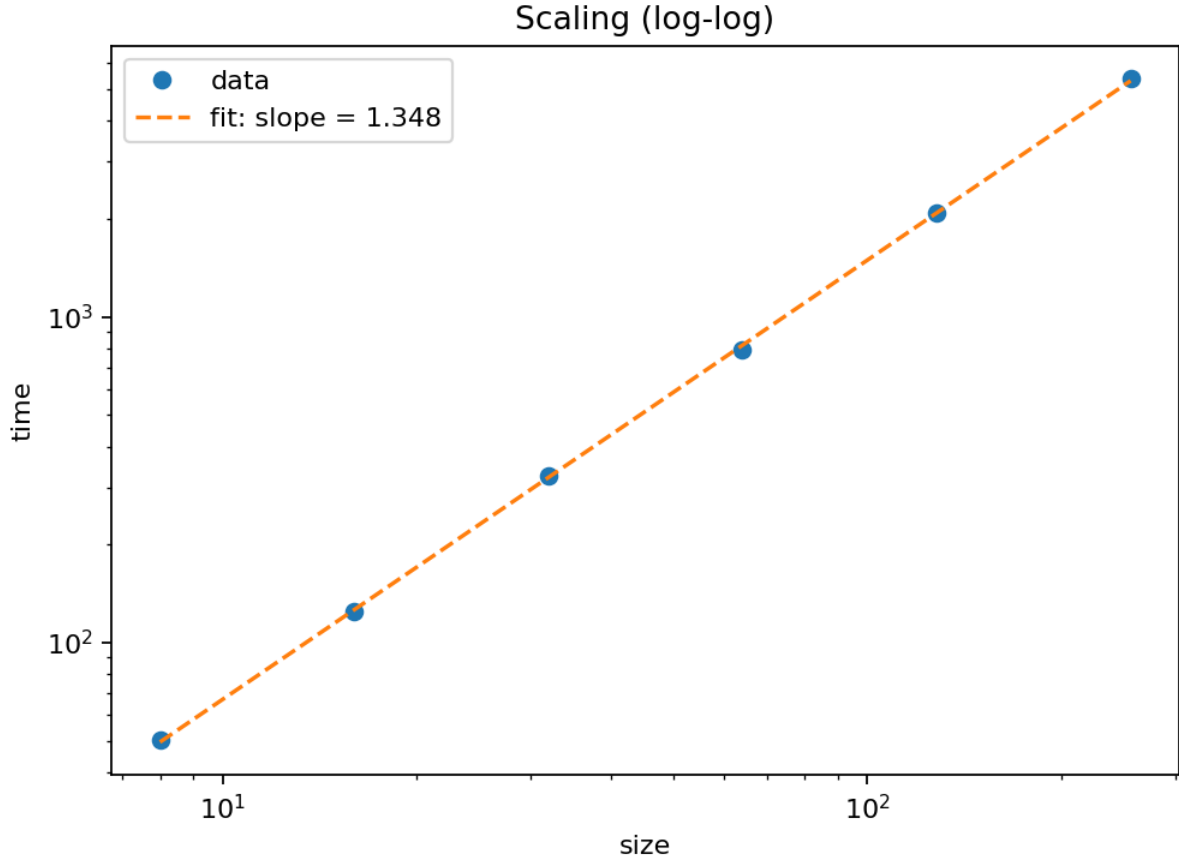


Figure 27: **Log-log scaling fit.** Workload sizes $n \in \{8, 16, 32, 64, 128, 256\}$ (dimensionless units) against synthetic durations $t \approx 3n^{1.35}(1 + \varepsilon)$ with $\varepsilon \sim \mathcal{N}(0, 0.02^2)$ relative noise, times rounded to four decimals, all drawn from the fixed-seed `numpy.random.default_rng(32)` generator; the least-squares power law from `scaling_fit` is overlaid by `plot_scaling_loglog`, and its fitted slope — annotated in the legend — should recover the planted exponent 1.35, the empirical complexity exponent β_1 of (57).

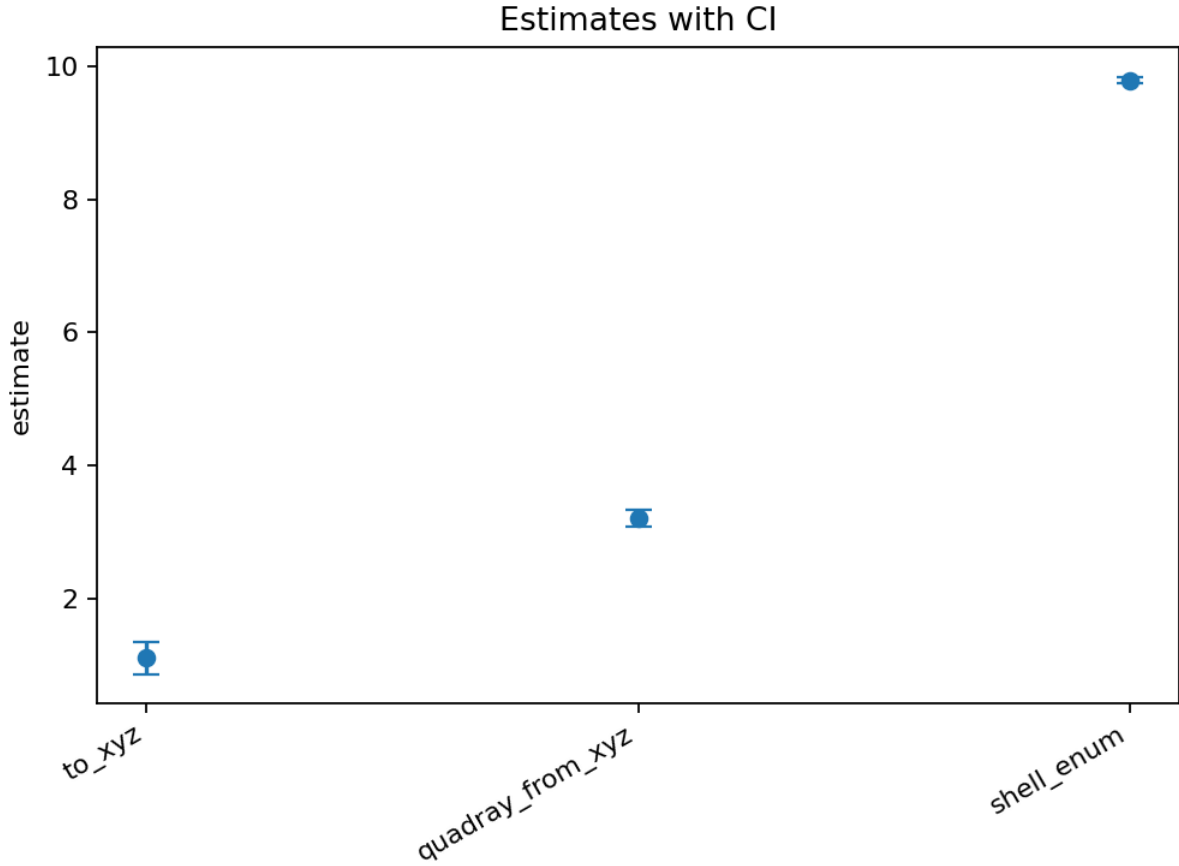


Figure 28: **Point estimates with confidence intervals by condition.** Three benchmark estimates (`to_xyz`, `quadray_from_xyz`, `shell_enum`) in synthetic estimate units, rendered by `plot_ci_bars` as points with symmetric error bars whose half-widths are uniform draws on $[0.05, 0.25]$ from the fixed-seed `numpy.random.default_rng(32)` generator; in a measured analysis the bar ends would be the 95% percentile bootstrap endpoints from `bootstrap_ci` (2000 resamples, (53)), whose non-overlap is the visual cue the permutation test of (54) quantifies.

a monotone step function that jumps by $1/n$ at every observed value — the exact distribution of the sample, with no binning choices at all. For timing data the ECDF answers directly what a percentile summary rounds off: the curve passes through the empirical median at height 0.5 and through the 95th percentile at height 0.95, so `median_s` and `p95_s` are readable off the plot. The gallery panel draws the ECDF of the same 256-draw clipped lognormal latency sample as the histogram panel above, so the two panels read as one distribution in two renderings.

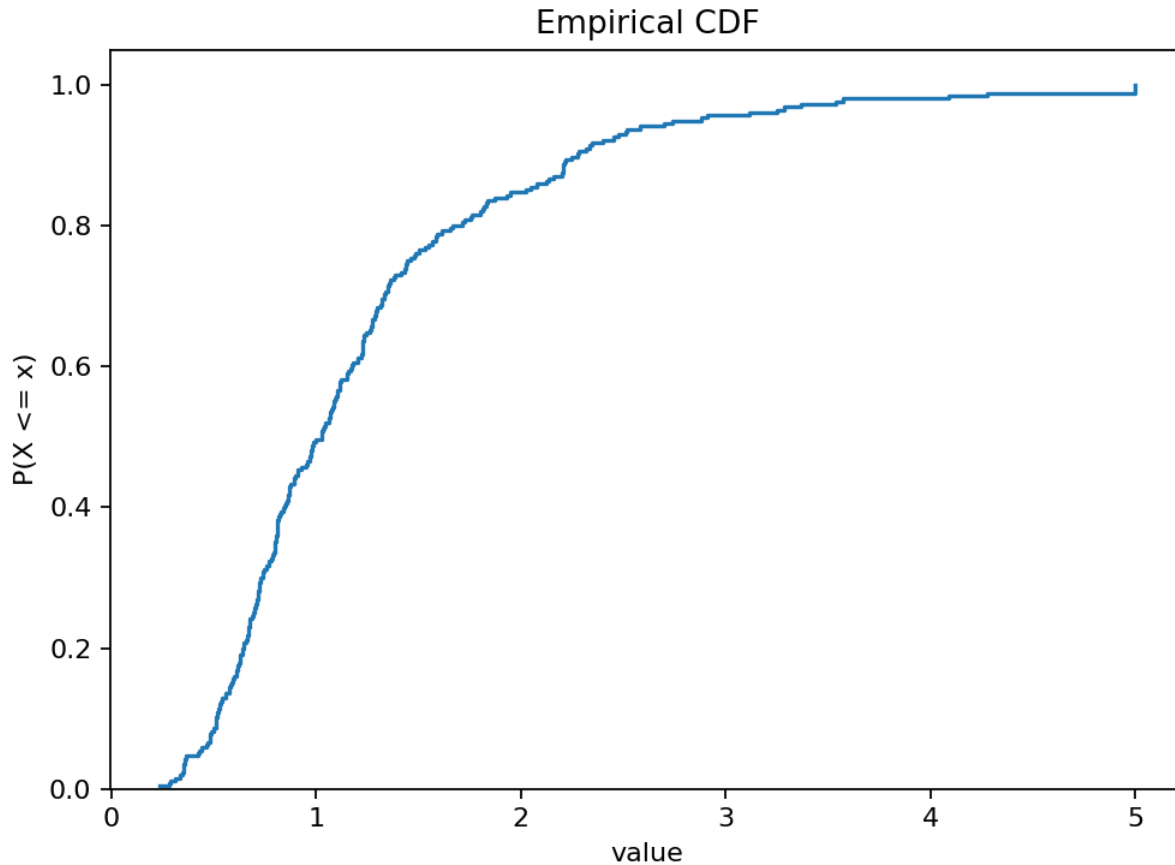


Figure 29: **Empirical cumulative distribution of the latency sample.** Exact step-function ECDF $\hat{F}(t)$ of the same 256-draw clipped lognormal sample as the histogram panel above (fixed seed 32, latency in seconds-scale units), rendered by `plot_ecdf`; the empirical median (height 0.5) and 95th percentile (height 0.95) are read directly off the curve.

18.6 Reproducibility and test contract

- The gallery function in `src/quadmath/viz/vis_stats.py` writes exactly the four files listed above, in that order, into the output directory (creating it when missing) and returns their paths. Every panel is fully seeded — the script fixes `seed=32` — and the PNGs carry no timestamp metadata, so re-renders are byte-identical.
- Tests: `tests/test_vis_stats.py` pins the contract with deterministic assertions, no mocks, and no pixel diffs — artist placement on caller-provided axes, input-agnostic behavior over plain arrays, and the invariants of each primitive (monotone non-decreasing ECDF steps, CI whisker endpoints matching the lows/highs inputs). Consistent with `17_benchmarks_statistics.md`, no test asserts wall-clock timing values.
- Figure regeneration: `uv run python quadmath/scripts/stats_gallery.py` (the script sets `MPLBACKEND=Agg` itself); std-out is exactly the four written paths, the same manifest contract as `quadmath/scripts/lattice_gallery.py`.
- Markdown: `uv run python quadmath/scripts/validate_markdown.py`.

18.7 Cross-references

- The measured quantities behind every panel: `17_benchmarks_statistics.md` (`src/quadmath/stats/benchmarks.py`, `src/quadmath/stats/statistics.py`).
- The sibling gallery of the lattice layer, same thin-orchestrator and determinism contract: `16_lattice_gallery.md` (`src/quadmath/viz/vis_lattice.py`).